

Universidad de Castilla-La Mancha
Escuela Superior de Ingeniería Informática

Trabajo Procesos de Ingeniería Software
Grado en Ingeniería Informática
Ingeniería del Software

La Grada Colchonera: juego web survivor en pixel art sobre una aplicación SaaS en capas

Javier Vela González-Román

Octubre, 2026

TRABAJO DE PROCESOS DE

INGENIERÍA DEL SOFTWARE

Grado en Ingeniería Informática

Ingeniería del Software

La Grada Colchonera: juego web survivor en pixel art sobre una aplicación SaaS en capas

Autor: Javier Vela González-Román

Octubre, 2026

Declaración de autoría

Yo, Javier Vela González-Román , declaro que soy el autor del trabajo titulado “La Grada Colchonera: juego web survivor en pixel art sobre una aplicación SaaS en capas”, que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual, y que todo el material no original contenido en dicho trabajo está apropiadamente atribuido a sus legítimos autores, detallándose asimismo el uso responsable de la inteligencia artificial en el Anexo D.

Albacete, Octubre, 2026

Fdo.: Javier Vela González-Román

Resumen

Esta memoria describe el diseño, la implementación y el proceso de desarrollo completo del proyecto *La Grada Colchonera*, una aplicación SaaS organizada en capas con gestión de usuarios y un juego web de tipo *survivor* en pixel art, desarrollado como parte de la asignatura Procesos de Ingeniería del Software (curso 2026-2027). El trabajo abarca dos sprints: el Sprint 1 (29/09–27/10/2026, Hitos 1–4) construye la base SaaS con cuentas, sesión, roles, PostgreSQL, correo, Google OAuth, errores, logs, CI/CD y despliegue en Google Cloud Run; el Sprint 2 (23/10–29/10/2026) implementa la lógica del juego: motor survivor con oleadas, jefes, armas automáticas, mejoras, mundo procedural, club persistente en servidor con validación anti-trampas, logros con historia del Atlético de Madrid, música chiptune y sonidos de grada. Se detalla exhaustivamente la arquitectura en capas, el modelo de datos, la gestión segura de sesiones y contraseñas mediante bcrypt, la metodología Scrum en DoneTonic, GitHub Flow, trazabilidad requisito → PBI → tarea → PR, y la evaluación con 276 pruebas automatizadas y revisión frase a frase contra el enunciado.

Palabras clave: SaaS, Scrum, GitHub Flow, Node.js, PostgreSQL, Google Cloud Run, survivor, pixel art, anti-cheat, Ingeniería del Software.

Abstract

This memoir describes the design, implementation, and development process of the *La Grada Colchonera* project, a layered SaaS application with user management and a web *survivor*-style game in pixel art, developed for the Software Engineering Processes course (2026-2027). The work spans two sprints: Sprint 1 (29/09–27/10/2026, Milestones 1–4) builds the SaaS foundation with accounts, sessions, roles, PostgreSQL, email, Google OAuth, error handling, logging, CI/CD, and deployment on Google Cloud Run; Sprint 2 (23/10–29/10/2026) implements the game logic: survivor engine with waves, bosses, automatic weapons, upgrades, procedural world, server-persisted club with anti-cheat validation, achievements tied to Atlético de Madrid history, chiptune music, and crowd sounds. The memoir thoroughly details the layered architecture, data model, secure session and password management with bcrypt, Scrum methodology in DoneTonic, GitHub Flow, requirement → PBI → task → PR traceability, and evaluation with 276 automated tests and phrase-by-phrase review against the specification.

Keywords: SaaS, Scrum, GitHub Flow, Node.js, PostgreSQL, Google Cloud Run, survivor, pixel art, anti-cheat, Software Engineering.

Índice general

1 Introducción 1

1.1 Contexto y motivación 1

1.2 Objetivos 1

1.2.1 Objetivo principal 1

1.2.2 Objetivos específicos 2

1.3 Alcance y cambio de enfoque 2

1.3.1 Por qué un juego y no una red social 2

1.4 Estructura de la memoria 3

2 Propuesta de Solución, Metodología y Herramientas 5

2.1 Propuesta de solución 5

2.2 Metodología 5

2.2.1 Gestión con Scrum en DoneTonic 6

2.2.2 Flujo de versiones: GitHub Flow adaptado 6

2.2.3 Pipeline de CI/CD 7

2.3 Herramientas 9

3 Descripción de la aplicación 11

3.1 Requisitos 11

3.1.1 Requisitos funcionales del Sprint 1 (Hitos 1–4) 11

3.1.2 Requisitos funcionales del juego (Sprint 2) 13

3.1.3 Requisitos no funcionales 15

3.2 Arquitectura y decisiones de diseño 16
3.2.1 Vista general: arquitectura en capas *16*
3.2.2 Decisiones de diseño *17*
3.3 Modelo de datos 19
3.3.1 Módulos del backend *19*
3.3.2 Frontend *20*
3.3.3 API *21*
3.4 Flujos con diagramas de secuencia 22
3.4.1 Registro con email y contraseña *22*
3.4.2 Confirmación con el enlace del correo *22*
3.4.3 Inicio de sesión local *23*
3.4.4 Inicio de sesión con Google OAuth 2.0 . . . . . *23*
3.4.5 Darse de baja *23*
3.5 Seguridad 23
3.6 Resumen de los sprints 23
3.7 La aplicación: manual de usuario (SaaS) 28
3.7.1 Inicio y registro *28*
3.7.2 Perfil y administración *28*
3.7.3 Hito 4: correo y Google *30*
3.7.4 Producción *31*
3.7.5 Prueba técnica *31*
3.7.6 Gestión en DoneTonic *31*
4 El juego (Sprint 2) 45
4.1 Concepto 45
4.2 Cómo se juega (manual de usuario) 45
4.2.1 Entrada y menú principal *45*
4.2.2 Controles *45*
4.2.3 Partida: oleadas, experiencia y mejoras *46*
4.2.4 Jefes *46*
4.2.5 Ambientes *46*
4.2.6 Fin de partida *49*
4.2.7 Vestuario, Entrenamiento, Contratar, Logros, Combinaciones, Dirección, Ranking, Ajustes, Pausa *49*

4.3 Sistemas 54
4.4 Arquitectura del juego 56
4.4.1 Determinismo *56*
4.4.2 Eventos *58*
4.4.3 Estadísticas *58*
4.4.4 Rendimiento *58*
4.4.5 Dibujo *58*
4.4.6 Audio *58*
4.5 Servidor del juego y anti-trampas 58
4.6 Diagramas de secuencia del juego 59
4.6.1 Una partida del juego (inicio, juego y fin con validación) *59*
4.6.2 Contratar con una ficha (concurrencia optimista) *59*
4.7 Logros 61
4.8 Evolución por fases 62

5 Evaluación 65
5.1 Diseño de la evaluación 65
5.2 Estrategia de pruebas y resultados 65
5.2.1 Pruebas automáticas *65*
5.2.2 Comprobaciones manuales y visuales *67*
5.3 Integración y despliegue continuos (CI/CD) 67
5.4 Registro de actividad y manejo de errores 69
5.4.1 Registro de actividad (logs) *69*
5.4.2 Manejo de errores *69*
5.5 Revisión frente al enunciado 70
5.6 Prueba técnica (RF-13) 70
5.7 Comprobación en producción 70
5.8 Métricas del proyecto 71
5.9 Resultados y discusión 73

6 Conclusiones y Trabajo Futuro 75
6.1 Conclusiones 75
6.1.1 Lecciones aprendidas 76
6.2 Trabajo futuro 76

A Desarrollo del Proyecto 77
A.1 Sprint 1: la base SaaS (29/09/2026–27/10/2026) 77
A.1.1 Planificación 77
A.1.2 Desarrollo: problemas encontrados y soluciones 80
A.1.3 Revisión 81
A.1.4 Retrospectiva 82
A.2 Sprint 2: la lógica del juego (23/10/2026–29/10/2026) 83
A.2.1 Planificación 83
A.2.2 Desarrollo: problemas encontrados y soluciones 91
A.2.3 Revisión 91
A.2.4 Retrospectiva 91

B Trazabilidad 95

C Material Adicional 101
C.1 Enlaces del proyecto 101
C.2 Capturas de la gestión en DoneTonic 102
C.3 Credenciales de administrador para la revisión 102

D Uso Responsable de la Inteligencia Artificial 105
D.1 Uso de la inteligencia artificial en el desarrollo del software 105
D.2 Uso de la inteligencia artificial en la redacción de la memoria 105

Referencia bibliográfica 108

Índice de figuras

2.1 Tablero del Sprint 1 en DoneTonic: cada fila es un PBI con sus tareas por columnas de estado; los PBIs en verde están hechos. 7
2.2 Pipeline CI/CD: push/PR -> CI -> si main y verde -> Cloud Run + Render. . . . 8

3.1 Arquitectura en capas: cliente (navegador) y servidor (Node.js/Express). . . 16
3.2 Modelo de datos entidad-relación: usuarios, sesiones y tablas del juego. . . 21
3.3 Registro con email y contraseña. 22
3.4 Confirmación con el enlace del correo. 23
3.5 Inicio de sesión local. 24
3.6 Inicio de sesión con Google OAuth 2.0. 25
3.7 Darse de baja. 27
3.8 Página de inicio sin sesión: botones para crear cuenta o entrar. 28
3.9 Registro con email y contraseña no válidos: campos en rojo y errores explicados. 29
3.10 Inicio de sesión con contraseña incorrecta: «Email o contraseña incorrectos». 29
3.11 Mi cuenta: mote, avatar, email, rol, estado, fecha de alta, cerrar sesión y darse de baja. 30
3.12 Diálogo de confirmación antes de darse de baja. 30
3.13 Un usuario normal intenta abrir #/admin y vuelve a Mi cuenta con el aviso de que la página es solo para administradores. 31
3.14 Panel de administración: contadores, buscador y tabla con email, rol, estado, alta y acciones. 32
3.15 Panel de administración filtrado por email. 33
3.16 Panel de administración en un móvil: cada usuario es una tarjeta. 34
3.17 Mi cuenta en un móvil con el menú plegado. 35
3.18 Ruta inexistente: «Fuera de juego». 36
3.19 Pantalla de entrar con el botón Entrar con Google. 36
3.20 Aviso: cuenta creada y correo de confirmación enviado. 37
3.21 Error «Debes confirmar tu correo» con el botón Reenviar el correo de confirmación. 37

3.22 Aviso verde: cuenta confirmada, ya puedes entrar. 38
3.23 Mi cuenta de una cuenta nueva de Google con el aviso «Cuenta creada con tu cuenta de Google». 38
3.24 Panel de administración con todos los estados de cuenta. 39
3.25 Página de política de privacidad. 40
3.26 Correo de confirmación real recibido en Gmail. 40
3.27 GitHub Actions: CI y despliegue en Cloud Run en verde tras los PRs del Hito 4. 41
3.28 Google Auth Platform: aplicación publicada en producción para usuarios externos. 41
3.29 Prueba técnica: respuesta permitido: true en la app desplegada. 41
3.30 Tablero del Sprint 1 en DoneTonic al 100 %: los 19 PBIs hechos. 42
3.31 Sprint Burndown y Sprint Burnup del Sprint 1 en DoneTonic. 42
3.32 Product Backlog de DoneTonic con los dos sprints. 43
3.33 Tablero del Sprint 2 en DoneTonic al 100 %: los 29 PBIs hechos. 43
3.34 Detalle del PBI P44 en DoneTonic con historia de usuario y criterios de aceptación. 44

4.1 Menú del juego (inicio con sesión): el jugador paseando por el campo, la ficha del aficionado, tus tiempos y el menú lateral. 46
4.2 Partida en curso: mundo procedural, rivales, armas automáticas y HUD. . . 47
4.3 Subida de nivel: elegir una de tres mejoras. 47
4.4 Jefe Vinicius Jr. en el campo con su barra de vida. 48
4.5 Ambiente de la Final de la Copa del Rey (a partir del minuto 5). 48
4.6 Fin de la partida: tiempo, estadísticas, daño por habilidad y logro nuevo. . . 49
4.7 Vestuario: lista de jugadores con filtros, ficha de Griezmann y equipaciones. 50
4.8 Entrenamiento: puntos para subir vida, defensa, disparo y suerte; botón modo pruebas (admin). 50
4.9 Mercado de fichajes con las fichas de contrato. 51
4.10 Vitrina de logros, cada uno con un dato de la historia del Atleti. 51
4.11 Las metas de la grada: ultras y combinaciones con lo que pide cada una. . . 52
4.12 Dirección deportiva: elegir entrenador (Cholo Simeone o Luis Aragonés) y su ulti. 52
4.13 Ranking de la grada: mote y avatar, nunca el correo. 53
4.14 Pantalla “Tu ficha”: mote y avatar. 53
4.15 Ajustes de volumen, gráficos, cámara y controles. 54
4.16 Menú de pausa de la partida. 57
4.17 Arquitectura del juego: simulación pura (izquierda), presentación (centro), servidor (derecha). 57
4.18 Una partida del juego con validación en el servidor. 59
4.19 Contratar con concurrencia optimista. 60

5.1 Pipeline de integración y despliegue continuos. 68
5.2 GitHub Actions: flujos de CI y despliegue en Cloud Run en verde tras los PRs del Hito 4. 69

5.3 Prueba técnica: validación afirmativa de dominio permitido. 71
5.4 Prueba técnica: rechazo de subdominio no autorizado. 71
5.5 Correo de confirmación transaccional recibido en un buzón de Gmail. . . . 71
5.6 Consola de Google Cloud: aplicación OAuth publicada en producción para usuarios externos. 72

A.1 Burndown del Sprint 1: PBIs pendientes del alcance de la entrega según se cerraban los hitos. 82
A.2 Sprint Burndown y Sprint Burnup del Sprint 1 en DoneTonic: los 19 PBIs completados, con el escalón del cambio de alcance (P19). 83
A.3 Product Backlog de DoneTonic con los dos sprints (Sprint 1 desplegado con sus PBIs). 84
A.4 Tablero del Sprint 1 en DoneTonic al cierre: los 19 PBIs en verde (hechos). . 84
A.5 Detalle de las gráficas del Sprint 1 en DoneTonic (vista ampliada del burndown y burnup). 85
A.6 Burndown del Sprint 2: recta porque el backlog se registró al cerrar el sprint (29/10/2026). 92
A.7 Gráficas del Sprint 2 en DoneTonic: recta porque el backlog se registró al cerrar el sprint (29/10/2026). 92
A.8 Product Backlog de DoneTonic con el Sprint 2 cerrado: 29 PBIs (P20–P48) en verde. 93
A.9 Tablero de DoneTonic al cierre de los dos sprints: Sprint 1 y Sprint 2 al 100 %. 93

C.1 Tablero del Sprint 1 en DoneTonic al 100 %: los 19 PBIs (P1–P19) hechos. . . 102
C.2 Sprint Burndown y Sprint Burnup del Sprint 1 en DoneTonic. 103
C.3 Product Backlog de DoneTonic con los dos sprints del proyecto. 103
C.4 Tablero del Sprint 2 en DoneTonic al 100 %: los 29 PBIs (P20–P48) con sus etiquetas de fase. 104
C.5 Sprint Burndown y Sprint Burnup del Sprint 2 en DoneTonic. 104

Índice de tablas

2.1 Clasificación de herramientas empleadas en el proyecto. 9

3.1 Requisitos funcionales del Sprint 1. 11
3.2 Requisitos funcionales del juego (Sprint 2). 13
3.3 Requisitos no funcionales. 15
3.4 Responsabilidades por capa. 17
3.5 Decisiones de diseño y justificación. 17
3.6 Módulos del backend. 19
3.7 Rutas principales de la API. 21
3.8 Amenazas y medidas de seguridad. 26
3.9 Resumen de los dos sprints. 27

4.1 Sistemas del juego. 54
4.2 Logros y equipaciones. 61
4.3 Evolución del juego por fases. 62

5.1 Pruebas por fichero. 66
5.2 Workflows de CI/CD. 68
5.3 Métricas globales del proyecto. 72

A.1 Sprint Backlog del Sprint 1 (19 PBIs, 32 tareas). 77
A.2 Trazabilidad del Sprint 1: tarea $\rightarrow$ rama $\rightarrow$ PR. 78
A.3 Problemas de desarrollo del Sprint 1 y su solución. 80
A.4 Problemas de despliegue y servicios externos. 80
A.5 Problemas de ramas y GitHub. 81
A.6 Cambio de alcance durante el sprint. 81
A.7 Cronología del proyecto. 81
A.8 Evolución del Sprint 1 (PBIs del alcance de la entrega). 82
A.9 Sprint Backlog del Sprint 2 (29 PBIs, 50 tareas). 85
A.10 Historias de usuario y criterios de aceptación del Sprint 2. 87
A.11 Problemas del Sprint 2 (juego y cuenta) y su solución. 91

B.1 Trazabilidad completa: requisito → PBI → tarea → PR. 95

1. Introducción

1.1. Contexto y motivación

Esta memoria describe el trabajo de la asignatura Procesos de Ingeniería del Software (PIS), curso 2026-2027, del Grado en Ingeniería Informática de la Escuela Superior de Ingeniería Informática de Albacete (Universidad de Castilla-La Mancha). El trabajo se plantea como si fuera un Trabajo Fin de Grado: un producto software real, desarrollado con un proceso de ingeniería (Scrum, control de versiones, integración y despliegue continuos y pruebas automáticas) y documentado con una memoria.

El enunciado del Sprint 1 fija cuatro hitos comunes a todos los proyectos. Son la base de cualquier aplicación SaaS: gestión de usuarios, autenticación, persistencia, roles, seguridad y despliegue. Sobre esa base, cada proyecto construye su propio producto. En este caso el producto es *La Grada Colchonera*, un juego web para aficionados del Atlético de Madrid. Es un proyecto de aficionado, sin ninguna relación oficial con el Club Atlético de Madrid.

La motivación es doble. Por un lado, practicar el proceso completo de un TFG con un proyecto que se pueda usar de verdad: una URL pública, cuentas reales y datos guardados en una base de datos. Por otro, que el proceso se pueda revisar entero desde el repositorio: el backlog, las ramas, los pull requests, las pruebas y los despliegues dejan rastro, y la memoria se apoya en ese rastro.

1.2. Objetivos

1.2.1. Objetivo principal

Construir y desplegar una aplicación web pública en la que los aficionados del Atlético de Madrid se registran, entran (con email o con Google) y juegan a un juego propio, con su progreso guardado en el servidor, siguiendo un proceso de ingeniería del software que se pueda revisar de principio a fin.

1.2.2. Objetivos específicos

1. **Producto.** Ofrecer la base SaaS que pide el enunciado (cuentas, sesión, roles, persistencia, correo, Google y manejo de errores) y, encima, el juego: una partida *survivor* con un club que se guarda en el servidor, ranking, fichajes, entrenamiento y logros.
2. **Proceso.** Aplicar Scrum con un backlog en DoneTonic, GitHub Flow con una rama y un pull request por tarea, integración continua con pruebas automáticas y despliegue continuo en la nube.
3. **Calidad.** Usar una arquitectura en capas con responsabilidades separadas, probar los casos de error y no solo el camino feliz, aplicar seguridad desde el diseño (hash de contraseñas, secretos fuera del código y permisos comprobados en el servidor) y hacer una interfaz accesible y adaptada al móvil.
4. **Documentación.** Que todo el trabajo, incluida la gestión, se pueda revisar desde el repositorio: README, carpeta `docs/` y esta memoria.

1.3. Alcance y cambio de enfoque

El proyecto se ha desarrollado en dos sprints:

- **Sprint 1** (29/09/2026 – 27/10/2026): los cuatro hitos del enunciado, con 19 PBIs (P1–P19) y 32 tareas. Los Hitos 1–3 (repositorio, CI, despliegue continuo, lógica de usuarios con pruebas, API REST, frontend separado en interfaz y cliente, registro e inicio de sesión, sesión, rutas protegidas, PostgreSQL, secretos, roles, panel de administración y registro de actividad) se entregaron el 12/10/2026. El Hito 4 (confirmación de la cuenta por correo, inicio de sesión con Google, manejo de errores de principio a fin y política de privacidad) se completó el 23/10/2026. Al cierre del sprint se añadió el despliegue en Google Cloud Run.
- **Sprint 2** (23/10/2026 – 29/10/2026): la lógica del juego, con 29 PBIs (P20–P48) y 50 tareas: el rediseño de la interfaz y el juego, construido en seis fases.

1.3.1. Por qué un juego y no una red social

La idea inicial era una red social para aficionados del Atlético. Cerrado el Sprint 1, el profesor dejó libertad para la lógica y el diseño del producto (y fijó después que el Sprint 2 sería esa lógica), y se decidió hacer un juego por tres motivos:

1. Es más original que otro clon de red social.
2. Obliga a resolver problemas técnicos más ricos: simulación determinista, rendimiento al dibujar en *canvas*, generación procedural del mapa, sonido sintetizado y validación de las partidas en el servidor para evitar trampas.
3. Reutiliza entera la base del Sprint 1. Un juego con cuentas, progreso en el servidor, ranking y roles necesita exactamente lo que piden los hitos.

Quedan fuera del alcance, por ahora, el multijugador en tiempo real, los amigos y las partidas en grupo, y las aplicaciones nativas (Sección 6.2).

1.4. Estructura de la memoria

- El Capítulo 2 presenta la propuesta, la metodología (Scrum en DoneTonic y GitHub Flow) y las herramientas.
- El Capítulo 3 describe la aplicación: requisitos, arquitectura, modelo de datos, API, los flujos principales con diagramas de secuencia, la seguridad y el uso de la aplicación con capturas.
- El Capítulo 4 explica el juego del Sprint 2: concepto, cómo se juega, sistemas, arquitectura, servidor y anti-trampas, logros y evolución por fases.
- El Capítulo 5 recoge la evaluación: pruebas, integración y despliegue continuos, registro de actividad y errores, revisión frente al enunciado, prueba técnica, comprobación en producción y métricas.
- El Capítulo 6 repasa los objetivos, las lecciones aprendidas y el trabajo futuro.
- El Anexo A cuenta el desarrollo de cada sprint; el Anexo B, la trazabilidad requisito – PBI – tarea – pull request; el Anexo C, los enlaces al material; y el Anexo D, el uso de la inteligencia artificial.

2. Propuesta de Solución, Metodología y Herramientas

2.1. Propuesta de solución

La propuesta de solución responde a la necesidad de construir una base sólida para una plataforma SaaS comunitaria con capacidad de ampliación futura. Antes de abordar las particularidades de implementación o la elección concreta de librerías, la solución se concibe como un sistema distribuido estructurado bajo el principio de separación de responsabilidades en tres capas: una capa de interfaz de usuario con navegación ligera, una capa intermedia que encapsula las reglas de negocio y políticas de autorización, y una capa de persistencia encargada de conservar el estado del sistema.

En el plano funcional, la solución propone un mecanismo de autenticación en el que los usuarios pueden registrarse y acceder al servicio de forma segura, garantizando la preservación de la sesión durante su navegación mediante identificadores almacenados en el servidor. Asimismo, se plantea un modelo de autorización diferenciado que distinga a los usuarios estándar de aquellos con privilegios administrativos. Los administradores disponen de visibilidad global sobre las cuentas existentes y ostentan la capacidad de revocar accesos, mientras que los usuarios conservan el control para solicitar su propia baja del sistema. Para asegurar una experiencia coherente, la solución postula que ninguna decisión de seguridad ni comprobación de permisos dependa exclusivamente de la interfaz del cliente: cualquier petición debe ser validada en el servidor antes de modificar el estado persistente.

2.2. Metodología

Para articular el desarrollo y garantizar la transparencia del progreso, se adoptó el marco de trabajo ágil Scrum [Schwaber and Sutherland, 2020], adaptado a las condiciones de un proyecto individual de ingeniería del software.

2.2.1. Gestión con Scrum en DoneTonic

La planificación y seguimiento del proyecto se centralizaron en la plataforma DoneTonic [DoneTonic, sf], dentro del espacio de trabajo *TFG* y el proyecto *Colchoneros*. El ciclo de trabajo corresponde al denominado **Sprint 1**, planificado entre el 29/09/2026 y el 27/10/2026.

A partir del análisis detallado del enunciado de la asignatura [Universidad de Castilla-La Mancha, 2026], se elaboró un *Product Backlog* compuesto por 19 elementos de la lista de producto (PBIs, numerados de P1 a P19), desglosados en un total de 32 tareas técnicas (T1 a T32). Puesto que todos los requisitos recogidos en el enunciado poseen carácter prescriptivo para la evaluación del software, la totalidad de los PBIs se catalogó con prioridad obligatoria (*MUST*).

Para velar por la integridad técnica del producto se definió una estricta Definición de Hecho (*Definition of Done*, DoD). Se consideró que una tarea técnica estaba completada exclusivamente cuando satisfacía los siguientes cuatro criterios concurrentes:

1. El código fuente reside en su rama de trabajo específica y se vincula formalmente a un pull request que referencia la tarea.
2. Todas las pruebas automatizadas superan satisfactoriamente la validación en el entorno de integración continua (verificación del check *Pruebas*).
3. El pull request correspondiente ha sido revisado y fusionado en la rama de integración del hito.
4. La documentación de soporte (como guías de despliegue o ficheros de configuración de ejemplo) ha sido actualizada coherentemente si la tarea lo requería.

Por su parte, un hito del sprint se considera completado cuando su rama integradora se ha fusionado en la rama principal, el software se encuentra desplegado de forma operativa en la nube y se ha emitido la correspondiente etiqueta de versión (*release*).

La Figura 2.1 muestra el tablero del Sprint 1 en DoneTonic al cierre de la entrega, con los 19 PBIs (P1–P19, Hitos 1–4) en verde (hechos).

2.2.2. Flujo de versiones: GitHub Flow adaptado

El desarrollo colaborativo y la gestión del código fuente siguieron las directrices de GitHub Flow [GitHub, sfb], estructuradas de forma jerárquica para reflejar los hitos pedagógicos:

- **Rama principal (`main`):** Representa en todo momento el código estable y listo para producción. Se encuentra protegida en GitHub, imposibilitando modificaciones directas y exigiendo obligatoriamente la resolución favorable del pipeline de integración continua.
- **Ramas de integración de hito (`hito-N/integracion`):** Actúan como puntos de convergencia para el conjunto de tareas pertenecientes a un mismo bloque temático semanal antes de su incorporación definitiva a la rama principal.

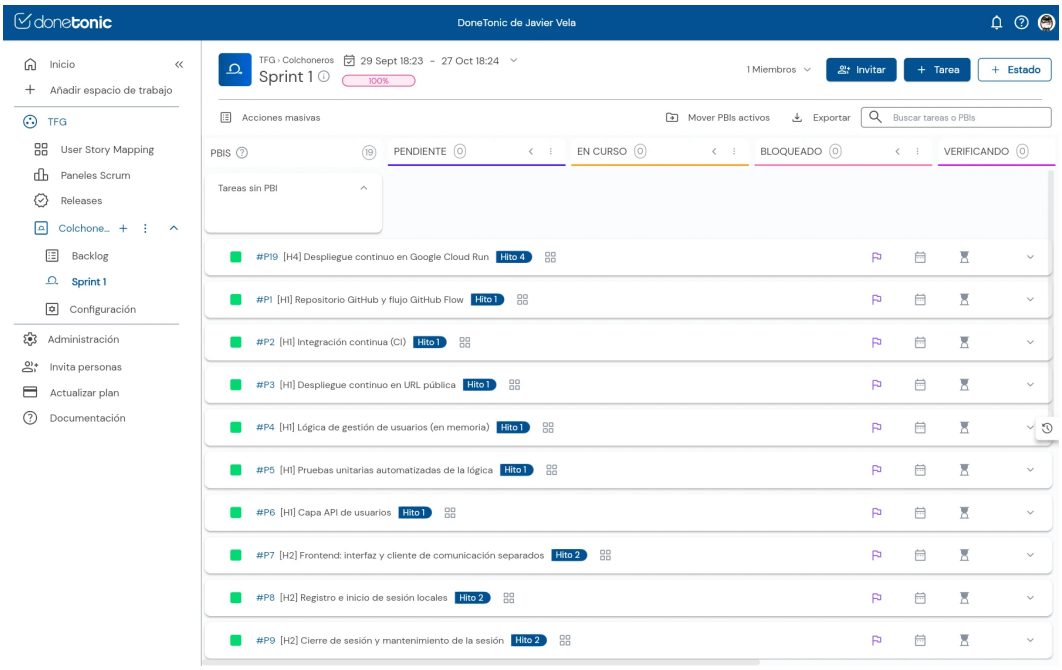


Figura 2.1: Tablero del Sprint 1 en DoneTonic: cada fila es un PBI con sus tareas por columnas de estado; los PBIs en verde están hechos.

- **Ramas de tarea (`hito-N/Tx-descripcion`):** Creadas a partir de la rama de integración correspondiente para acometer de forma aislada una única tarea del backlog.

El ciclo operativo de cada cambio se rigió por las normas estipuladas en la guía de contribución del repositorio:

1. Selección de la tarea `Tx` en el tablero de DoneTonic y creación de la rama técnica asociada.
2. Generación de commits con mensajes descriptivos en tiempo imperativo y prefijo identificativo (por ejemplo, `[Tx] Descripción del cambio`).
3. Ejecución local de la batería de pruebas unitarias y de integración (`npm test`) previo a la publicación de cambios.
4. Apertura de un pull request hacia la rama de integración del hito y verificación del paso exitoso del CI en GitHub Actions.
5. Fusión del pull request y transición automática de la tarea a estado *HECHO* en el tablero ágil.

2.2.3. Pipeline de CI/CD

El pipeline de integración y despliegue continuo, implementado con GitHub Actions, se describe en la Figura 2.2. El flujo ejecuta las pruebas contra PostgreSQL 17 real, construye la imagen Docker y, solo en `main` con CI en verde, despliega automáticamente en Google Cloud Run mediante Workload Identity Federation.

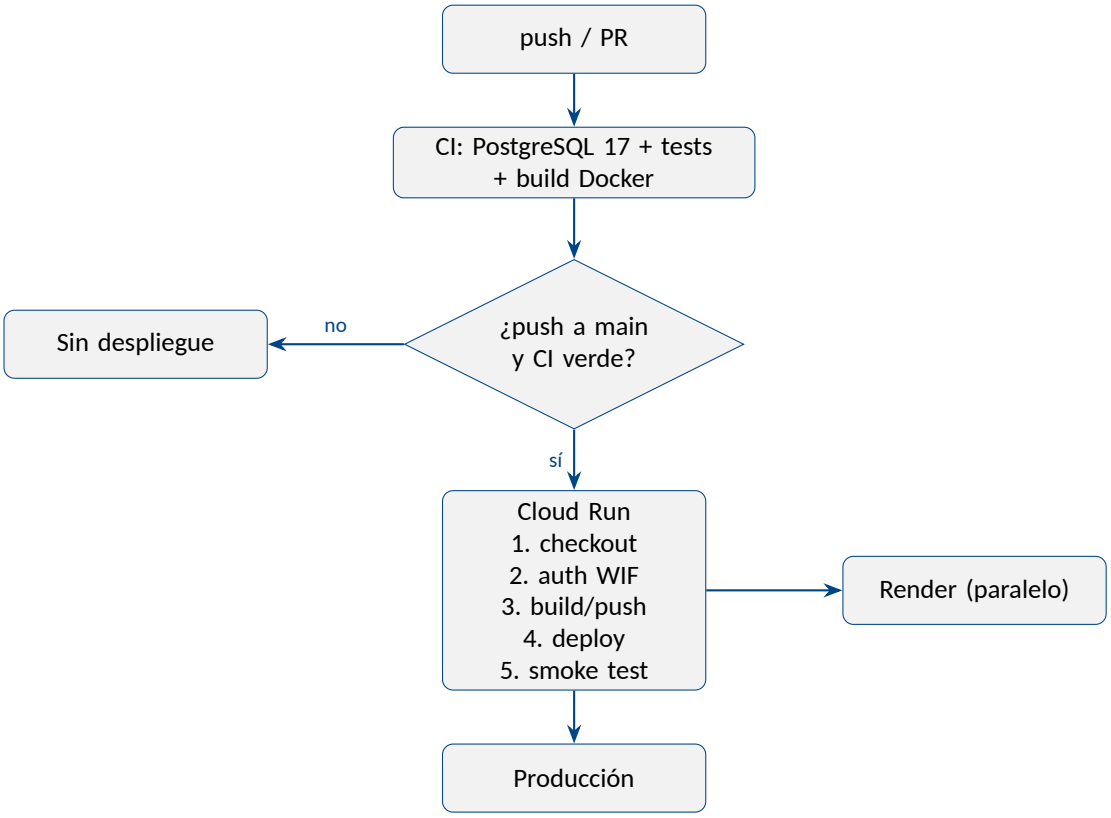


Figura 2.2: Pipeline CI/CD: push/PR -> CI -> si main y verde -> Cloud Run + Render.

2.3. Herramientas

Para sustentar las distintas dimensiones del proyecto se seleccionó un conjunto de herramientas y tecnologías de acuerdo con los requisitos técnicos exigidos en la asignatura. La Tabla 2.1 sintetiza las herramientas empleadas, estructuradas según su ámbito de aplicación en el ciclo de vida del software.

Categoría	Herramienta	Propósito en el proyecto
Gestión ágil	DoneTonic	Backlog, sprint, tablero de tareas y gráficas [Schwaber and Sutherland, 2020].
	GitHub	Repositorio público, ramas protegidas, pull requests, releases y GitHub Actions [GitHub, sfb].
Desarrollo	Node.js 22	Entorno de ejecución para unificar JavaScript en cliente y servidor [OpenJS Foundation, sf].
	Express 5	Framework web minimalista con gestión nativa de errores asíncronos [Express, sf].
	PostgreSQL (Neon)	Base de datos relacional serverless con capa gratuita en la nube [The PostgreSQL Global Development Group, sf].
	Playwright	Automatización de navegador para generar capturas de interfaz homogéneas.
Despliegue y CI/CD	GitHub Actions	Pipeline de pruebas automáticas, empaquetado Docker y entrega [GitHub, sfa].
	Google Cloud Run	Plataforma PaaS de contenedores serverless para el despliegue principal [Google Cloud, sf].
	Render	Plataforma PaaS alternativa configurada con `render.yaml`.
Documentación	LuaLaTeX	Motor tipográfico para la maquetación de la memoria técnica.
	Claude (Cowork)	Asistente inteligente para la generación de código, pruebas y documentación.

Tabla 2.1: Clasificación de herramientas empleadas en el proyecto.

La combinación de estas herramientas permitió orquestar un entorno homogéneo y plenamente automatizado, reduciendo al mínimo las dependencias manuales y proporcionando trazabilidad completa desde la definición de requisitos hasta la verificación del despliegue en producción.

3. Descripción de la aplicación

3.1. Requisitos

Esta sección presenta los requisitos funcionales y no funcionales del proyecto, organizados por sprints, con su trazabilidad a PBIs y ubicación en el código.

3.1.1. Requisitos funcionales del Sprint 1 (Hitos 1–4)

La Tabla 3.1 recoge los 13 requisitos funcionales del Sprint 1.

Tabla 3.1: Requisitos funcionales del Sprint 1.

Id	Requisito	Hito	PBI	Dónde se cumple	Pruebas
RF-01	Registro con email y contraseña (validación, email único, contraseña de 8 o más caracteres)	2	P8	`userService.register`, `POST /api/auth/register`, vista `registro.js`	`userService`, `api.auth`
RF-02	Inicio de sesión local, mismo mensaje si el email no existe o la contraseña es incorrecta	2	P8	`userService.authenticate`, `POST /api/auth/login`	`api.auth`
RF-03	Mantener la sesión al recargar y cerrarla en el servidor	2	P9	`express-session` + tabla `session`, `GET /api/auth/me`, `POST /api/auth/logout`	`api.auth`, `api.database` (CI)
RF-04	Rutas protegidas en el servidor (401 sin sesión)	2	P10	`requireAuth`	`api.users`, `api.auth`

Id	Requisito	Hito	PBI	Dónde se cumple	Pruebas
RF-05	Persistencia en base de datos real	3	P11	PostgreSQL (Neon), `postgresUserRepository`, migraciones	`userRepository`, `migrate`, `api.database`
RF-06	Roles usuario y administrador, con un primer administrador	3	P13	`requireRole`, `requireSelfOrRole`, `userService.ensureAdmin`	`userService`, `api.users`, `bootstrap`
RF-07	Listar usuarios, ver si un usuario está activo y cambiar roles (solo administrador)	3	P13	`GET /api/users`, `GET /api/users/:id/active`, `PATCH /api/users/:id/role`, vista `admin.js`	`api.users`
RF-08	Darse de baja (y que el administrador pueda eliminar cuentas); nunca eliminar al último administrador; tras la baja se puede volver con el mismo email	3	P13	`DELETE /api/users/:id` (baja lógica), vistas `perfil.js` y `admin.js`	`userService`, `api.users`
RF-09	Registro de la actividad relevante	3	P14	`src/logger.js`, `requestLogger`	`logger`
RF-10	Confirmación de la cuenta por correo, con reenvío	4	P16	`accountConfirmationService`, `tokenSigner`, `mailer`, `GET /api/auth/confirm`, `POST /api/auth/resend-confirmation`	`accountConfirmation`, `tokenSigner`, `mailer`, `api.auth`
RF-11	Inicio de sesión con proveedor externo (Google, OAuth 2.0) [Hardt, 2012]	4	P15	`googleOAuth.js`, `GET /api/auth/google`, `GET /api/auth/google/callback`, `userService.loginWithOAuth`	`googleOAuth`, `userService`
RF-12	Manejo de errores con información clara al usuario	4	P17	`errorHandler.js`, `componentes.js`, `avisos.js`, `api.js` (timeout)	`api.*`, `frontend`

Id	Requisito	Hito	PBI	Dónde se cumple	Pruebas
RF-13	Prueba técnica: solo dominios de correo permitidos	Prueba	—	`domainPolicy.js`, `POST /api/prueba/dominio`	`domainPolicy` (13)

3.1.2. Requisitos funcionales del juego (Sprint 2)

La Tabla 3.2 recoge los 20 requisitos funcionales del juego, derivados de los PBIs P20–P48.

Tabla 3.2: Requisitos funcionales del juego (Sprint 2).

Id	Requisito	PBI	Dónde se cumple	Pruebas
RJ-01	Con sesión, el inicio es el menú del juego; sin sesión no se juega y el progreso va ligado a la cuenta	P20, P29	`ui.js` (ruta `#/jugar`), `requireAuth` en `/api/juego`, tablas `juego_perfiles` y `juego_records`	`api.juego`, `frontend`
RJ-02	Partida *survivor* infinita: oleadas, experiencia, mejoras al subir de nivel y dificultad creciente	P21, P24, P34	`mundo.js`, `oleadas.js`, `mejoras.js`, `config.js`	`game`, `juego-personajes`, `juego-club`
RJ-03	Gráficos pixel art generados por código y cámara 3/4	P22, P45	`sprites.js`, `render.js`, `render-mundo.js`	`juego-movilidad`, `juego-habilidades`
RJ-04	Elegir jugador (36, con estadísticas por posición, habilidades propias y rareza por historia) y entrenador (2, con ulti)	P23, P38	Vestuario y Dirección (`juego.js`), `config.js`, `ulti.js`	`juego-personajes`, `juego-club`
RJ-05	Armas automáticas con 5 niveles, pasivas y habilidades que solo suben a partir de cierto minuto	P24, P39	`armas.js`, `mejoras.js`, `combate.js`	`juego-habilidades`
RJ-06	Rivales con comportamiento propio (embisten, disparan, rodean obstáculos, élites)	P25	`rivales.js`, `oleadas.js`	`juego-movilidad`, `juego-personajes`

Id	Requisito	PBI	Dónde se cumple	Pruebas
RJ-07	Jefes con tres fases, avisos de ataque, escolta, furia y jugada estrella	P26, P46	`jefes.js`, `oleadas.js`	`juego-movilidad`, `juego-logros`
RJ-08	Mundo procedural con terreno, obstáculos macizos y colisiones exactas	P27, P35, P41	`terreno.js`, `mundo.js`	`juego-movilidad`
RJ-09	Ambientes que cambian cada 5 minutos (de la pretemporada a la Final del Mundial)	P36	`config.js`, `render-mundo.js`, `musica.js`	`juego-movilidad`
RJ-10	Jugable con teclado, ratón y pantalla táctil; pausa, pantalla completa y ajustes	P28	`entrada.js`, `ajustes.js`, `juego.js`	`game`, `frontend`
RJ-11	Club guardado en el servidor con validación anti-trampas de cada partida	P29	`juegoService.finalizarPartida`, migración 003	`api.juego`, `juegoRepository`
RJ-12	Fichar jugadores con fichas de contrato (una por cada 3 jefes), sorteadas por rareza	P30, P38	`club.js`, `POST /api/juego/contratar`, `firma.js`	`juego-club`, `api.juego`, `frontend`
RJ-13	Entrenar a los jugadores con puntos ganados por bajas y jefes	P31	`club.js`, `POST /api/juego/entrenar`	`juego-club`, `api.juego`
RJ-14	Ranking de la grada con la mejor partida de cada usuario (sin mostrar el correo)	P32	`juego_records`, `GET /api/juego/ranking`	`api.juego`, `juegoRepository`
RJ-15	Combinaciones y ULTRAS, con una pantalla que muestra lo que pide cada una	P33, P40	`ultras.js`, `render-ultras.js`, pantalla Combinaciones	`juego-ultras`, `juego-habilidades`, `frontend`
RJ-16	Modo pruebas solo para administradores	P34	`POST /api/juego/desbloquear`, `scripts/desbloquear-juego.js`	`api.juego`
RJ-17	Mote único y avatar, en la cabecera, Mi cuenta y el ranking	P37	`PUT /api/juego/identidad`, migración 004	`api.juego`, `juegoRepository`, `frontend`

Id	Requisito	PBI	Dónde se cumple	Pruebas
RJ-18	Logros con datos de la historia del Atleti, validados en el servidor, y equipaciones desbloqueables	P42	`logros.js`, `juegoService`	`juego-logros`, `api.juego`
RJ-19	Pantalla de fin con estadísticas y daño por habilidad	P43	`fin-ui.js`, `combate.js`	`juego-logros`
RJ-20	Música chiptune, efectos y la grada, con ajustes de volumen	P44	`audio.js`, `musica.js`, `ajustes.js`	`juego-logros`

3.1.3. Requisitos no funcionales

La Tabla 3.3 recoge los 12 requisitos no funcionales.

Tabla 3.3: Requisitos no funcionales.

Id	Requisito	Cómo se cumple
RNF-01	**Seguridad** de credenciales y sesiones	Hash bcrypt [Provos and Mazières, 1999], cookie `httpOnly` + `secure` + `sameSite`, regeneración de la sesión, permisos en el servidor, consultas parametrizadas
RNF-02	**Secretos fuera del código**	Variables de entorno; Secret Manager en producción; la app no arranca si falta un secreto obligatorio
RNF-03	**Separación en capas**	API → lógica → datos/integraciones, con inyección de dependencias
RNF-04	**Pruebas automáticas** y CI obligatorio	305 pruebas con `node:test`; las ramas protegidas exigen el CI en verde
RNF-05	**Despliegue continuo** en una URL pública	Cloud Run (principal) y Render (alternativo), sin intervención manual
RNF-06	**Disponibilidad y coste**	Escala a cero (capas gratuitas y créditos de educación); si una versión no arranca, sigue sirviendo la anterior
RNF-07	**Accesibilidad**	HTML semántico, foco gestionado al navegar, `aria-live` en los avisos, errores de formulario enlazados con `aria-describedby`, manejo con teclado, `prefers-reduced-motion`

Id	Requisito	Cómo se cumple
RNF-08	**Responsive**	Diseño adaptable: el panel de administración pasa a tarjetas en el móvil; joystick táctil en el juego
RNF-09	**Rendimiento del juego**	60 FPS en un portátil normal: simulación de paso fijo, búsqueda espacial por celdas, texturas por celda cacheadas y precargadas
RNF-10	**Trazabilidad**	Requisito → PBI → tarea → rama → PR → release
RNF-11	**Privacidad**	Política de privacidad pública; el ranking nunca muestra el correo; los logs ocultan los datos sensibles
RNF-12	**Mantenibilidad**	Sin frameworks ni compilación en el frontend; todo el equilibrio del juego en un fichero de configuración; módulos pequeños y comentados

3.2. Arquitectura y decisiones de diseño

3.2.1. Vista general: arquitectura en capas

La Figura 3.1 muestra la arquitectura cliente-servidor en capas. El servidor (Node.js + Express) sirve también el frontend estático, por lo que todo vive en el mismo origen.

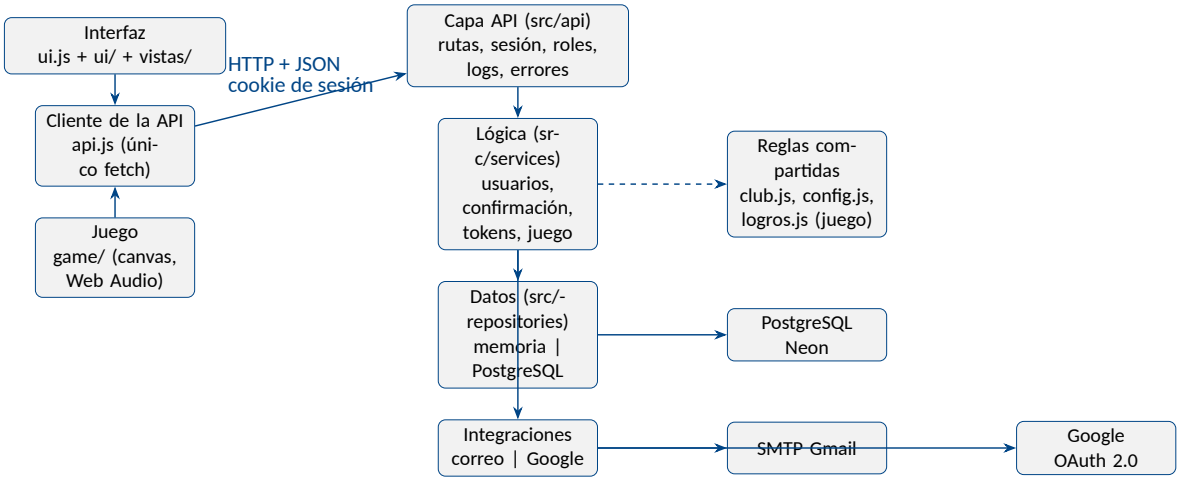


Figura 3.1: Arquitectura en capas: cliente (navegador) y servidor (Node.js/Express).

La separación de responsabilidades por capas se detalla en la Tabla 3.4.

Capa	Carpeta	Responsabilidad	No hace
Presentación	`public/`	Pintar vistas, validar formularios, mostrar avisos, jugar	No habla con la BD ni decide permisos
Cliente API	`public/js/api.js`	Peticiones HTTP, timeout 15 s, traducir errores a `ApiError`, avisar si caduca la sesión	No pinta nada
API	`src/api/`	Rutas HTTP, sesión, autenticación y autorización, registro de peticiones, formato común de errores	No contiene reglas de negocio
Lógica	`src/services/`	Reglas de negocio	No conoce HTTP ni SQL
Datos	`src/repositories/`	Guardar y leer (memoria o PostgreSQL) con el mismo contrato	No valida reglas
Integraciones	`src/integrations/`	Correo y Google	No decide nada
Composición	`src/app.js`, `src/bootstrap.js`	Conectar las capas, elegir la BD, migrar, crear el primer administrador	—

Tabla 3.4: Responsabilidades por capa.

3.2.2. Decisiones de diseño

La Tabla 3.5 resume las principales decisiones de diseño con su justificación.

Tabla 3.5: Decisiones de diseño y justificación.

Decisión	Alternativas	Por qué
Express 5 sin framework de frontend	React, Vue, Next.js	Proyecto individual y didáctico: las capas se ven como ficheros, no hay paso de compilación, el backend sirve el frontend (sin CORS) y la frontera interfaz/cliente que pide el Hito 2 queda en dos ficheros. Express 5 pasa los errores de las funciones `async` al manejador común

Decisión	Alternativas	Por qué
Repositorio con dos implementaciones (memoria y PostgreSQL)	Solo PostgreSQL; un ORM	Se puede arrancar y probar sin base de datos, y las **mismas pruebas** se pasan a las dos, lo que demuestra que la lógica no depende del almacenamiento (patrón Repository)
Inyección de dependencias por parámetro	Módulos globales, singletons	Cada servicio recibe sus dependencias (`userRepository`, `passwordHasher`, `mailer`, `tokenSigner`, reloj, azar), así que las pruebas usan versiones falsas sin trucos
Sesiones en el servidor (`express-session` + PostgreSQL)	JWT en el navegador	Cerrar sesión o eliminar una cuenta la invalida al momento; la cookie es `httpOnly` y no contiene datos; las sesiones sobreviven a los reinicios y despliegues
Bajas lógicas (`status = deleted`) con el email libre	Borrar la fila; bloquear el email para siempre	Se conserva el historial y la auditoría; la cuenta deja de poder entrar y su sesión deja de valer, pero su email y su mote quedan libres para que la persona pueda volver con una cuenta nueva (índice único parcial)
Token de confirmación firmado (HMAC) sin guardar en la BD	Token aleatorio en una tabla	No hace falta tabla ni limpieza; caduca solo; no se puede fabricar ni modificar sin el secreto
OAuth 2.0 escrito con `fetch`	Passport.js u otra librería	El flujo es corto y se entiende entero (`state`, intercambio del código, email verificado); menos dependencias y fácil de probar con un cliente falso [Hardt, 2012]
Primer administrador por variables de entorno	Script SQL, insertar a mano	No deja contraseñas en el código ni en scripts; es idempotente (si ya existe, no se toca)
Migraciones SQL propias	ORM con migraciones	Pocas tablas; SQL legible y versionado, con un registro de las aplicadas
Cloud Run como principal y Render como alternativo	Solo uno	Cloud Run aprovecha los créditos de educación, escala a cero y permite Secret Manager y acceso sin claves (WIF); Render fue el primer despliegue y queda de respaldo

Decisión	Alternativas	Por qué
Reglas del juego compartidas cliente/servidor	Reglas duplicadas	El servidor importa `club.js`, `config.js` y `logros.js`: una sola fuente de verdad para fichajes, entrenamiento y logros
Concurrencia optimista en el perfil del juego (`version`)	Bloqueos, transacciones largas	Dos peticiones simultáneas no pueden gastar dos veces la misma ficha; la segunda se reintenta con los datos nuevos
Simulación del juego pura y determinista	Lógica mezclada con el dibujo	Paso fijo de 1/60 s y azar con semilla: se prueba en Node sin navegador y el resultado no depende de los FPS
Gráficos y sonido generados con código	Imágenes y ficheros de audio	Sin problemas de derechos (nada oficial), carga instantánea y fácil de variar (equipaciones, selecciones, intensidad de la música)

3.3. Modelo de datos

El diagrama entidad-relación de la Figura 3.2 muestra las tablas principales, incluyendo las del juego.

3.3.1. Módulos del backend

La Tabla 3.6 resume los módulos del backend.

Tabla 3.6: Módulos del backend.

Módulo	Qué hace
`server.js`	Punto de entrada: carga la configuración, llama a `bootstrap` y escucha en `PORT`; para con orden al recibir `SIGTERM`
`bootstrap.js`	Elige PostgreSQL (si hay `DATABASE_URL`) o memoria, aplica las migraciones, crea o promociona el primer administrador y crea el correo (SMTP o registro)
`app.js`	Construye la app Express: JSON, sesión, `/api` y los estáticos de `public/`
`config.js`, `loadConfig.js`	Leen y validan las variables de entorno; en producción exigen los secretos
`logger.js`	Logs JSON por línea, con niveles y ocultación de datos sensibles
`api/index.js`	Monta el router: `requestLogger`, `currentUser`, `/health`, `/auth`, `/prueba`, `/users` y `/juego` (con `requireAuth`), 404 y manejador de errores
`api/auth.js`	Registro, confirmación, reenvío, login, Google, logout y `me`

Módulo	Qué hace
`api/users.js`	Administración de usuarios (`requireRole('admin')`, `requireSelfOrRole('admin')`)
`api/juego.js`	Perfil, partidas, contratos, entrenamiento, identidad, ranking y modo pruebas
`api/currentUser.js`	Middlewares de identidad, autenticación y autorización
`api/errorHandler.js`	Traduce los errores tipados a HTTP (`{ error: { code, message } }`)
`services/userService.js`	Reglas de usuarios: registro, autenticación, OAuth, roles, bajas, último administrador y primer administrador
`services/accountConfirmationService.js`	Crea y envía el enlace de confirmación, confirma y reenvía
`services/tokenSigner.js`	Tokens firmados con HMAC-SHA256 y caducidad
`services/passwordHasher.js`	bcrypt (coste 10; en las pruebas, 4)
`services/juegoService.js`	Reglas del servidor del juego: perfiles, partidas, validación anti-trampas, recompensas, logros, contratos, entrenamiento, identidad y ranking
`services/domainPolicy.js`	Prueba técnica: dominios permitidos
`services/errors.js`	Errores tipados: `ValidationError`, `UnauthorizedError`, `ForbiddenError`, `AccountPendingError`, `NotFoundError`, `ConflictError`, `ServiceUnavailableError`
`repositories/*`	`memoryUserRepository`, `postgresUserRepository`, `memoryJuegoRepository`, `postgresJuegoRepository`
`integrations/mailer.js`	Correo por SMTP (`nodemailer`) o, sin `SMTP_URL`, escrito en el registro
`integrations/googleOAuth.js`	URL de autorización de Google e intercambio del código por el perfil
`db/migrate.js`, `db/migrations/*.sql`	Migraciones versionadas (001 usuarios, 002 sesiones, 003 juego, 004 identidad, 005 email libre tras la baja)

3.3.2. Frontend

El frontend es una SPA con rutas por hash (`public/js/ui.js`): `#/` inicio, `#/entrar`, `#/registro`, `#/perfil`, `#/jugar` y `#/admin`, y una vista 404 («Fuera de juego»). Cada ruta declara si es protegida, solo para invitados o solo para administradores; la interfaz redirige, pero la protección real está en el servidor.

Las vistas (`ui/vistas/*.js`) exportan `montar(raiz, contexto)` y devuelven una función de limpieza. El estado (`ui/estado.js`) guarda el usuario actual; la cabecera (`ui/cabecera.js`) se repinta al cambiar (mote, avatar, enlaces según el rol). Los formularios (`ui/formulario.js`) validan al escribir y al enviar, con errores enlazados mediante

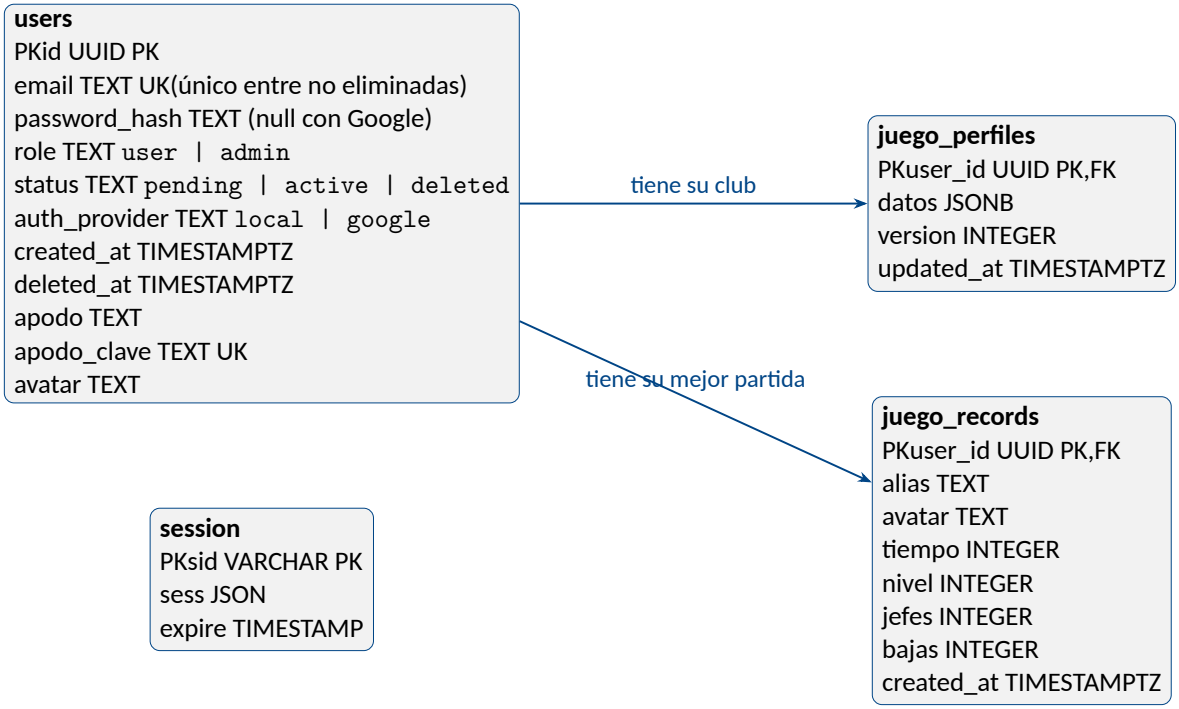


Figura 3.2: Modelo de datos entidad-relación: usuarios, sesiones y tablas del juego.

`aria-describedby`. Los avisos (`ui/avisos.js`) usan una zona `aria-live`; los errores usan `role="alert"`. El cliente (`api.js`) es el único sitio con `fetch`; timeout de 15 s con `AbortController`; convierte respuestas de error en `ApiError`.

3.3.3. API

La lista completa de rutas está en el `README` del repositorio. Resumen en la Tabla 3.7.

Tabla 3.7: Rutas principales de la API.

Grupo	Rutas	Acceso
Salud	`GET /api/health`	Público
Autenticación	`POST register`, `POST login`, `POST logout`, `GET me`, `GET confirm`, `POST resend-confirmation`, `GET providers`, `GET google`, `GET google/callback`	Público (salvo `me`)
Usuarios	`GET /users`, `GET /users/:id/active`, `PATCH /users/:id/role`, `DELETE /users/:id`	Admin (la baja, propio o admin)

Grupo	Rutas	Acceso
Juego	`GET perfil`, `POST partidas`, `POST partidas/:id/fin`, `POST contratar`, `POST entrenar`, `PUT identidad`, `GET ranking`, `POST desbloquear`	Sesión (`desbloquear`, admin)
Prueba técnica	`POST /api/prueba/dominio`	Público

3.4. Flujos con diagramas de secuencia

Esta sección muestra los diagramas de secuencia de los flujos principales. Todos los participantes están declarados en cada diagrama.

3.4.1. Registro con email y contraseña

La Figura 3.3 muestra el flujo de registro.

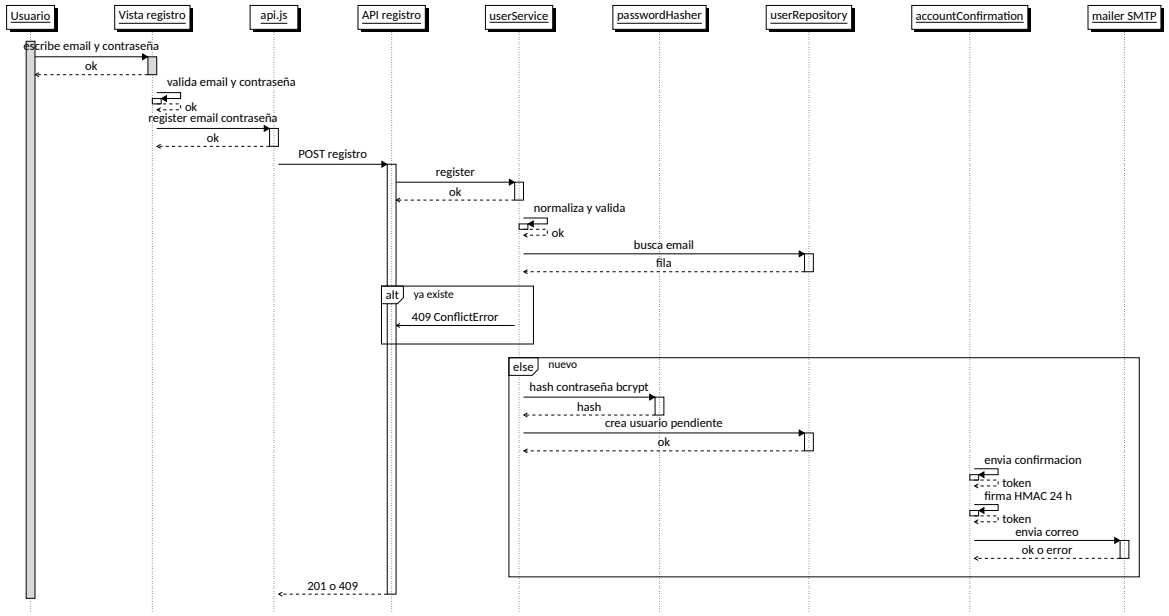


Figura 3.3: Registro con email y contraseña.

3.4.2. Confirmación con el enlace del correo

La Figura 3.4 muestra el flujo de confirmación.

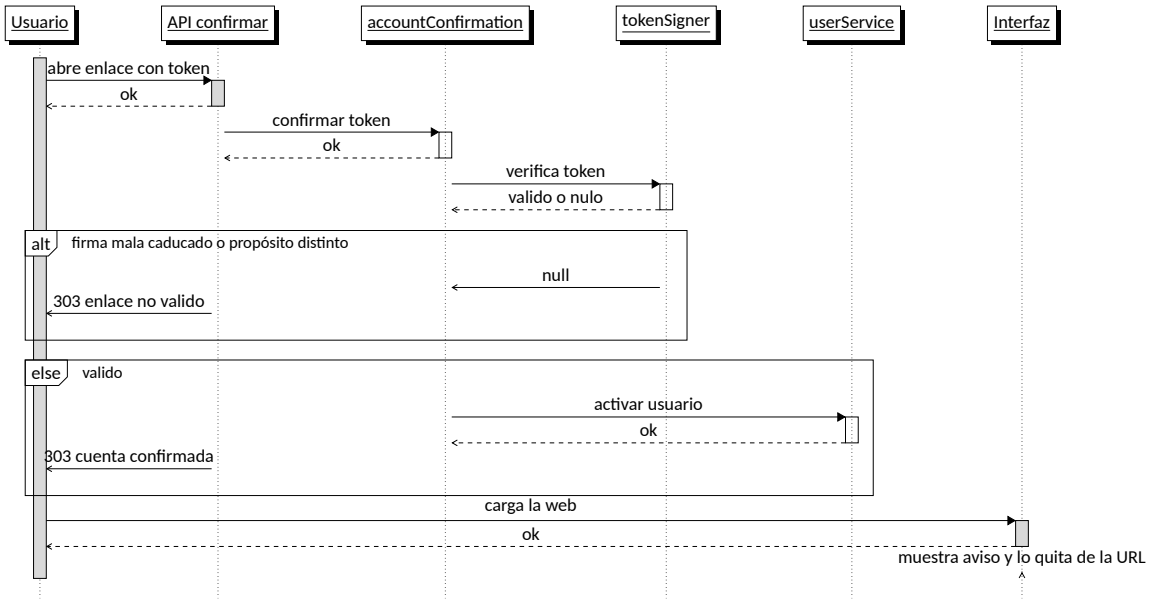


Figura 3.4: Confirmación con el enlace del correo.

3.4.3. Inicio de sesión local

La Figura 3.5 muestra el flujo de inicio de sesión local.

3.4.4. Inicio de sesión con Google OAuth 2.0

La Figura 3.6 muestra el flujo de inicio de sesión con Google.

3.4.5. Darse de baja

La Figura 3.7 muestra el flujo de baja de cuenta.

3.5. Seguridad

La Tabla 3.8 resume las principales amenazas y las medidas aplicadas.

3.6. Resumen de los sprints

La Tabla 3.9 resume los dos sprints del proyecto.

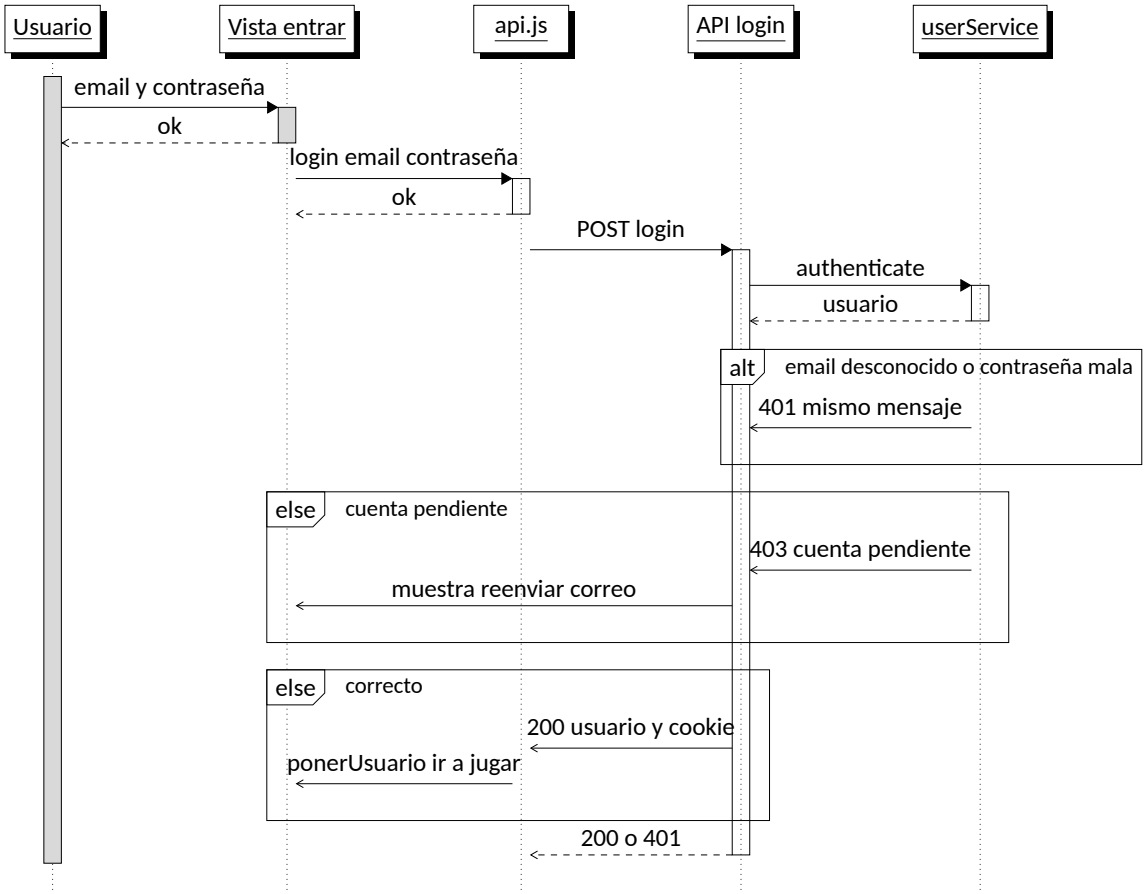


Figura 3.5: Inicio de sesión local.

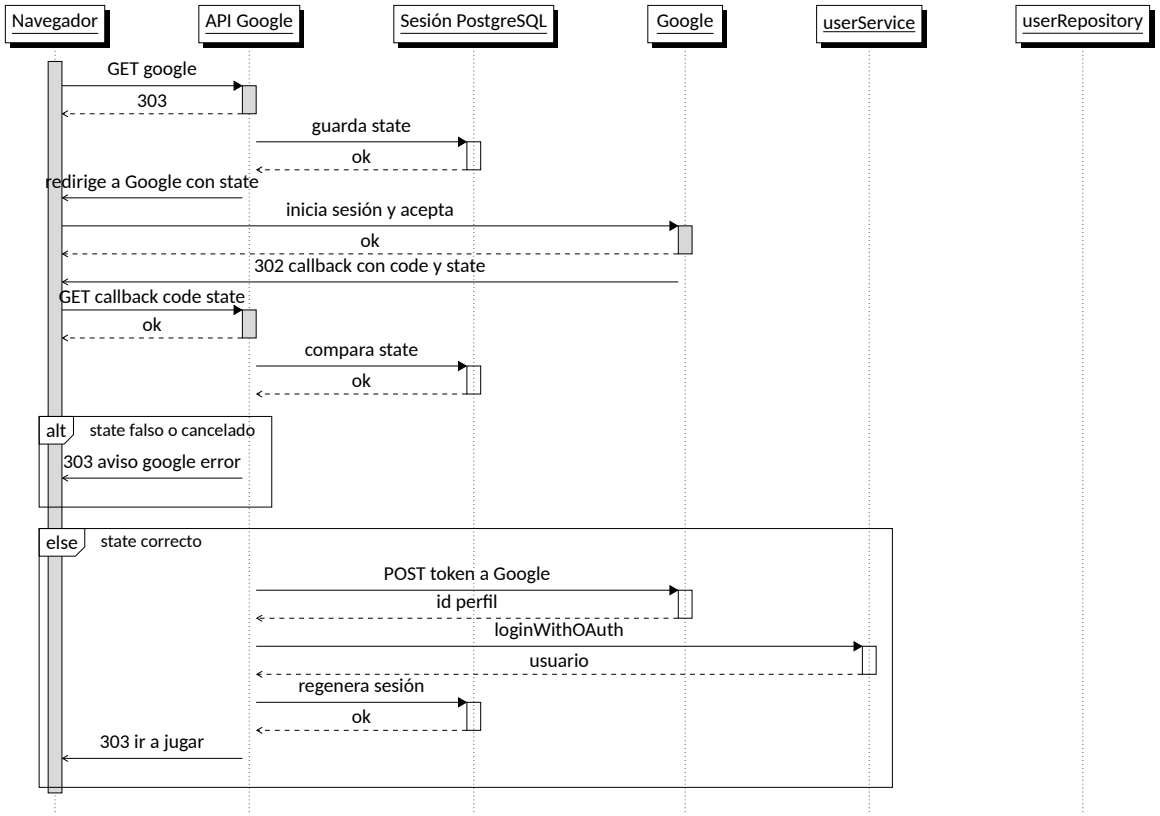


Figura 3.6: Inicio de sesión con Google OAuth 2.0.

Amenaza	Medida	Dónde
Robo de contraseñas	Solo se guarda el hash bcrypt (sal, coste 10). Nunca la contraseña ni un cifrado reversible	`passwordHasher.js`
Robo o fijación de sesión	Cookie `httpOnly`, `sameSite=lax`, `secure` en producción; la sesión se regenera al entrar y se destruye al salir	`app.js`, `auth.js`
Sesión de una cuenta eliminada	En cada petición se vuelve a cargar el usuario; si ya no está activo, la sesión deja de valer	`currentUser.js`
Escalada de privilegios	Roles comprobados en el servidor (`401`/`403`); siempre queda un administrador	`currentUser.js`, `userService.js`
Enlaces falsificados	Token HMAC-SHA256 con clave derivada, comparación en tiempo constante, propósito y caducidad (24 h)	`tokenSigner.js`
CSRF en OAuth	`state` aleatorio, guardado en la sesión, de un solo uso	`auth.js`
Inyección SQL	Consultas parametrizadas	`postgres*Repository.js`
XSS	El DOM se crea con `textContent` (helper `h()`); una prueba lo comprueba	`dom.js`, `frontend.test.js`
Secretos expuestos	Solo variables de entorno; Secret Manager con cuenta de servicio de mínimo privilegio; `.env` ignorado	`loadConfig.js`, workflows
Datos sensibles en logs	Contraseñas, hashes, cookies, tokens y secretos se sustituyen por `[oculto]`	`logger.js`
Trampas en el juego	Reloj del servidor, límites creíbles, logros verificables, concurrencia optimista	`juegoService.js`

Tabla 3.8: Amenazas y medidas de seguridad.

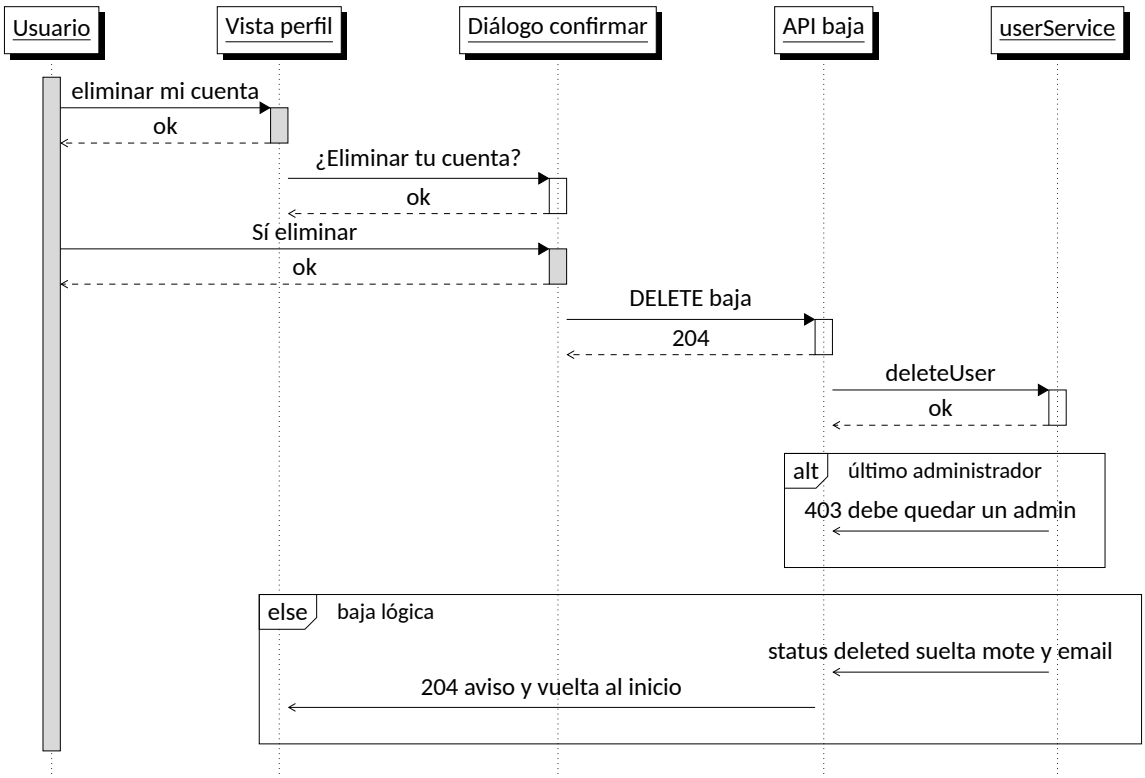


Figura 3.7: Darse de baja.

Sprint	Fechas	PBIs / tareas	Alcance
Sprint 1	29/09/2026 – 27/10/2026	19 / 32	Base SaaS: cuentas, sesión, roles, PostgreSQL, correo, Google, errores, logs, CI/CD, despliegue en Cloud Run
Sprint 2	23/10/2026 – 29/10/2026	29 / 50	Lógica del juego: survivor en pixel art, club, ranking, logros, música y sonido

Tabla 3.9: Resumen de los dos sprints.

3.7. La aplicación: manual de usuario (SaaS)

Esta sección describe el uso de la aplicación SaaS (cuentas, administración, Hito 4) con capturas de la aplicación real.

3.7.1. Inicio y registro

La figura 3.8 muestra la página de inicio sin sesión: botones para crear cuenta o entrar.

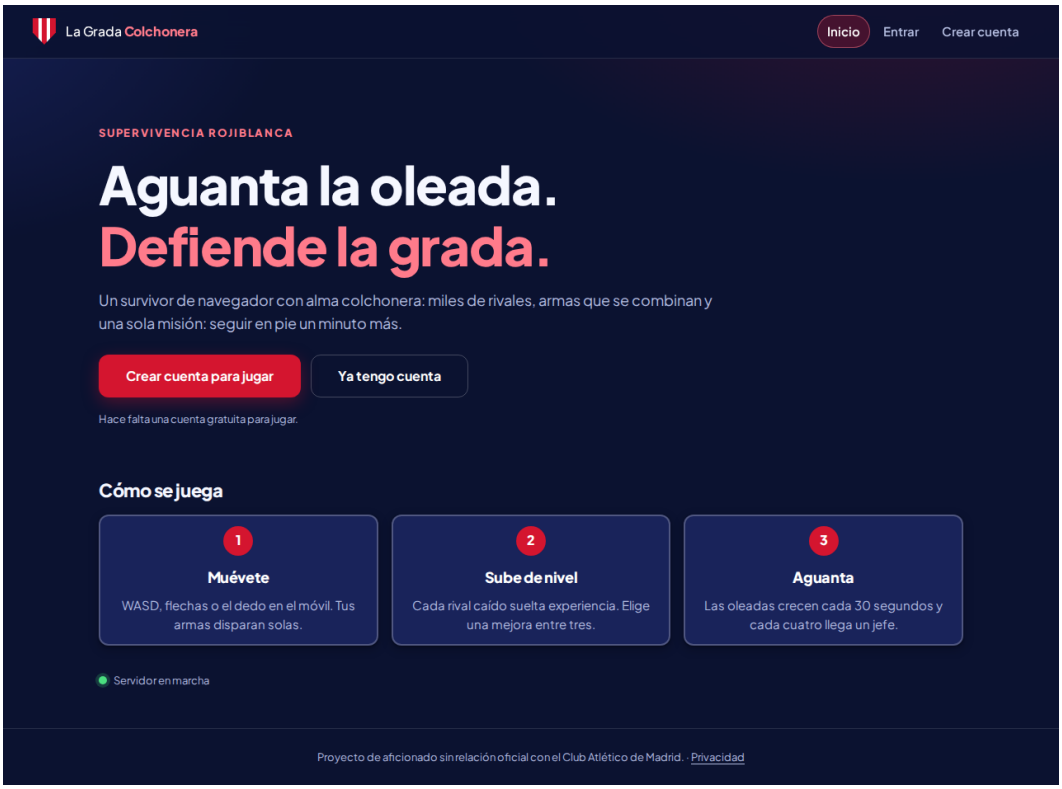


Figura 3.8: Página de inicio sin sesión: botones para crear cuenta o entrar.

La figura 3.9 muestra el registro con email y contraseña no válidos: campos en rojo y errores explicados.

La figura 3.10 muestra el inicio de sesión con contraseña incorrecta: «Email o contraseña incorrectos».

3.7.2. Perfil y administración

La figura 3.11 muestra la pantalla Mi cuenta: mote, avatar, email, rol, estado, fecha de alta, cerrar sesión y darse de baja.

La figura 3.12 muestra el diálogo de confirmación antes de darse de baja.

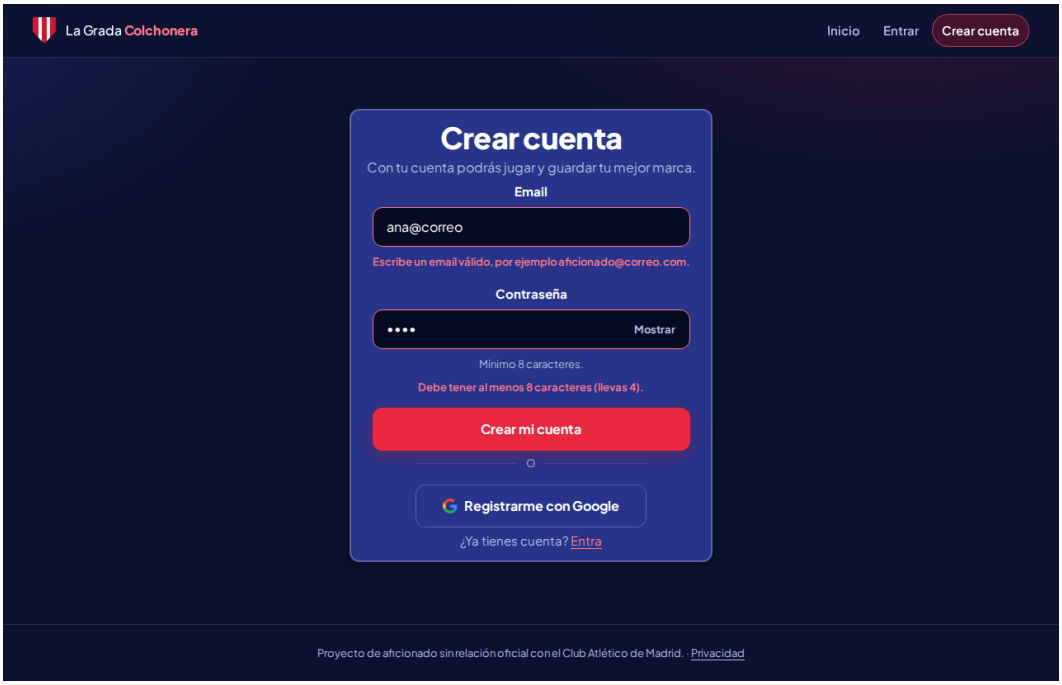


Figura 3.9: Registro con email y contraseña no válidos: campos en rojo y errores explicados.

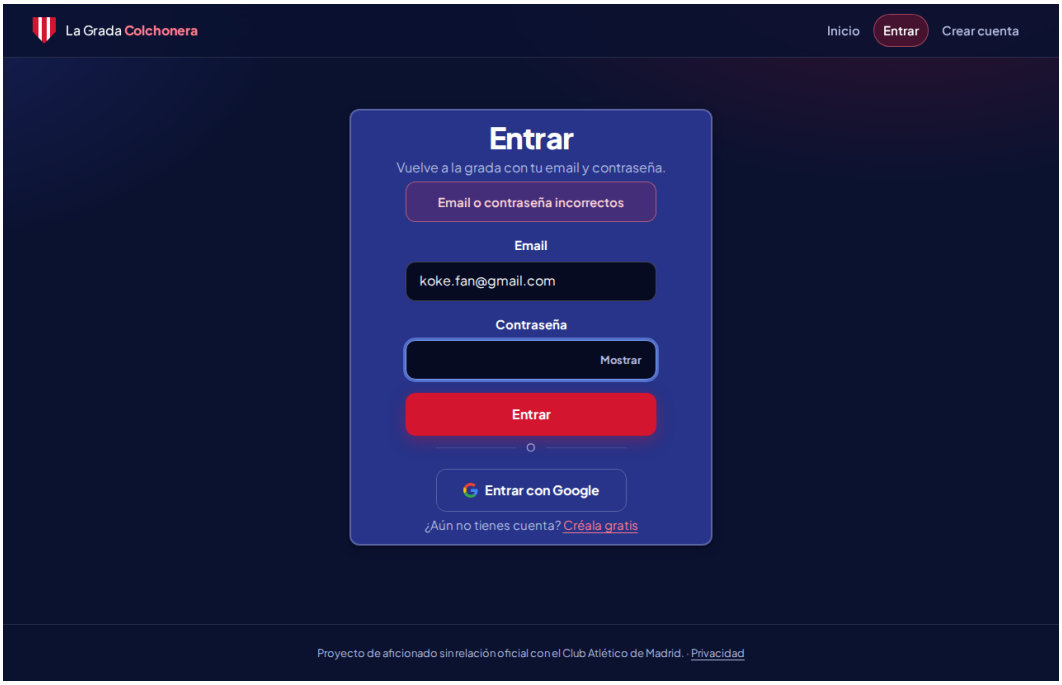


Figura 3.10: Inicio de sesión con contraseña incorrecta: «Email o contraseña incorrectos».

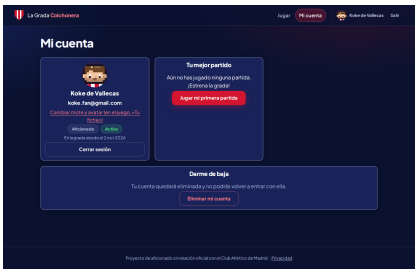


Figura 3.11: Mi cuenta: mote, avatar, email, rol, estado, fecha de alta, cerrar sesión y darse de baja.

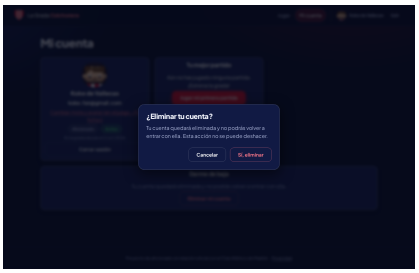


Figura 3.12: Diálogo de confirmación antes de darse de baja.

En la figura 3.13, un usuario normal intenta abrir #/admin y vuelve a Mi cuenta con el aviso de que la página es solo para administradores.

La figura 3.14 muestra el panel de administración: contadores, buscador y tabla con email, rol, estado, alta y acciones.

La figura 3.15 muestra el panel de administración filtrado por email.

La figura 3.16 muestra el panel de administración en un móvil: cada usuario es una tarjeta.

La figura 3.17 muestra la pantalla Mi cuenta en un móvil con el menú plegado.

La figura 3.18 muestra la ruta inexistente: «Fuera de juego».

3.7.3. Hito 4: correo y Google

La figura 3.19 muestra la pantalla de entrar con el botón Entrar con Google.

La figura 3.20 muestra el aviso: cuenta creada y correo de confirmación enviado.

La figura 3.21 muestra el error «Debes confirmar tu correo» con el botón Reenviar el correo de confirmación.

La figura 3.22 muestra el aviso verde: cuenta confirmada, ya puedes entrar.

La figura 3.23 muestra la pantalla Mi cuenta de una cuenta nueva de Google con el aviso «Cuenta creada con tu cuenta de Google».

La figura 3.24 muestra el panel de administración con todos los estados de cuenta.

La figura 3.25 muestra la página de política de privacidad.

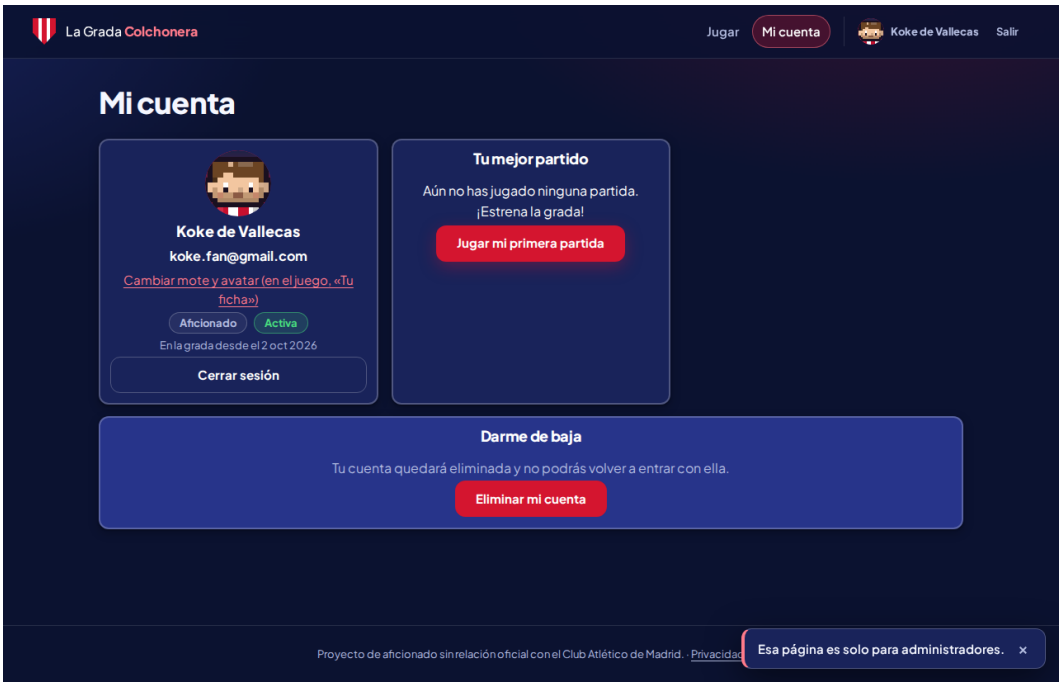


Figura 3.13: Un usuario normal intenta abrir #/admin y vuelve a Mi cuenta con el aviso de que la página es solo para administradores.

3.7.4. Producción

La figura 3.26 muestra el correo de confirmación real recibido en Gmail.

La figura 3.27 muestra GitHub Actions: CI y despliegue en Cloud Run en verde tras los PRs del Hito 4.

La figura 3.28 muestra Google Auth Platform: aplicación publicada en producción para usuarios externos.

3.7.5. Prueba técnica

La figura 3.29 muestra la prueba técnica: respuesta permitido: true en la app desplegada.

3.7.6. Gestión en DoneTonic

La figura 3.30 muestra el tablero del Sprint 1 en DoneTonic al 100 %: los 19 PBIs hechos.

La figura 3.31 muestra las gráficas Sprint Burndown y Sprint Burnup del Sprint 1 en DoneTonic.

La figura 3.32 muestra el Product Backlog de DoneTonic con los dos sprints.

La figura 3.33 muestra el tablero del Sprint 2 en DoneTonic al 100 %: los 29 PBIs hechos.

La figura 3.34 muestra el detalle del PBI P44 en DoneTonic con historia de usuario y criterios de aceptación.

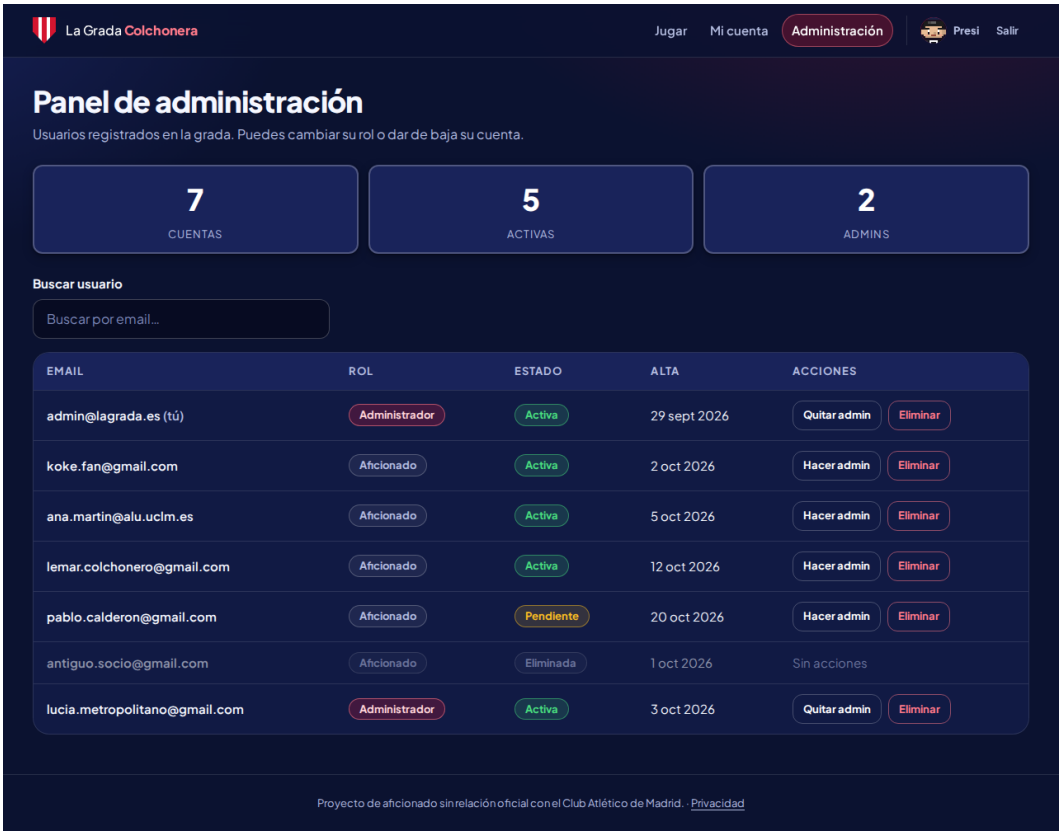


Figura 3.14: Panel de administración: contadores, buscador y tabla con email, rol, estado, alta y acciones.

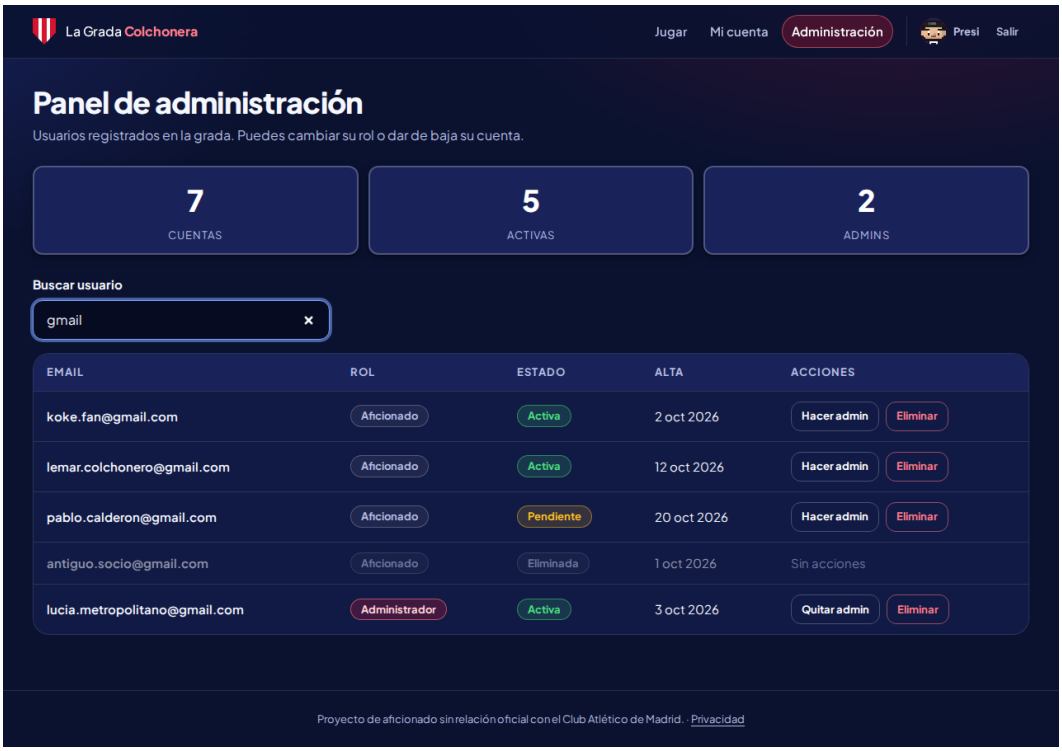


Figura 3.15: Panel de administración filtrado por email.

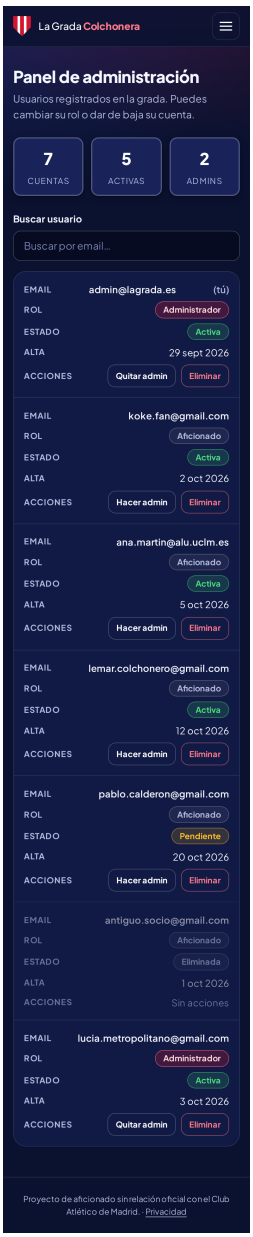


Figura 3.16: Panel de administración en un móvil: cada usuario es una tarjeta.



Figura 3.17: Mi cuenta en un móvil con el menú plegado.

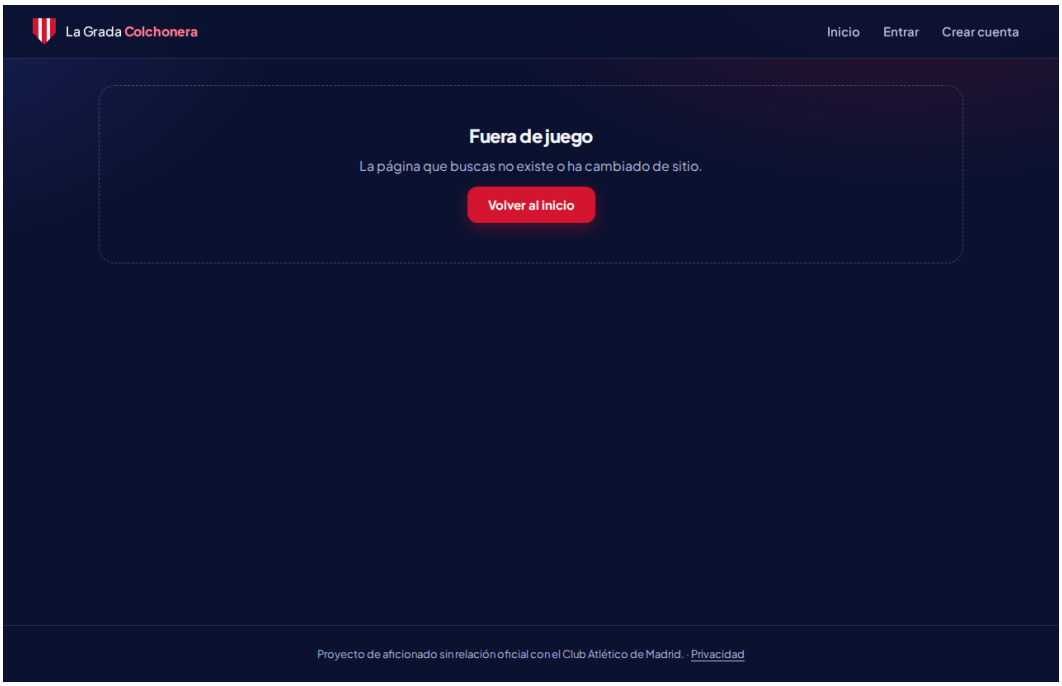


Figura 3.18: Ruta inexistente: «Fuera de juego».

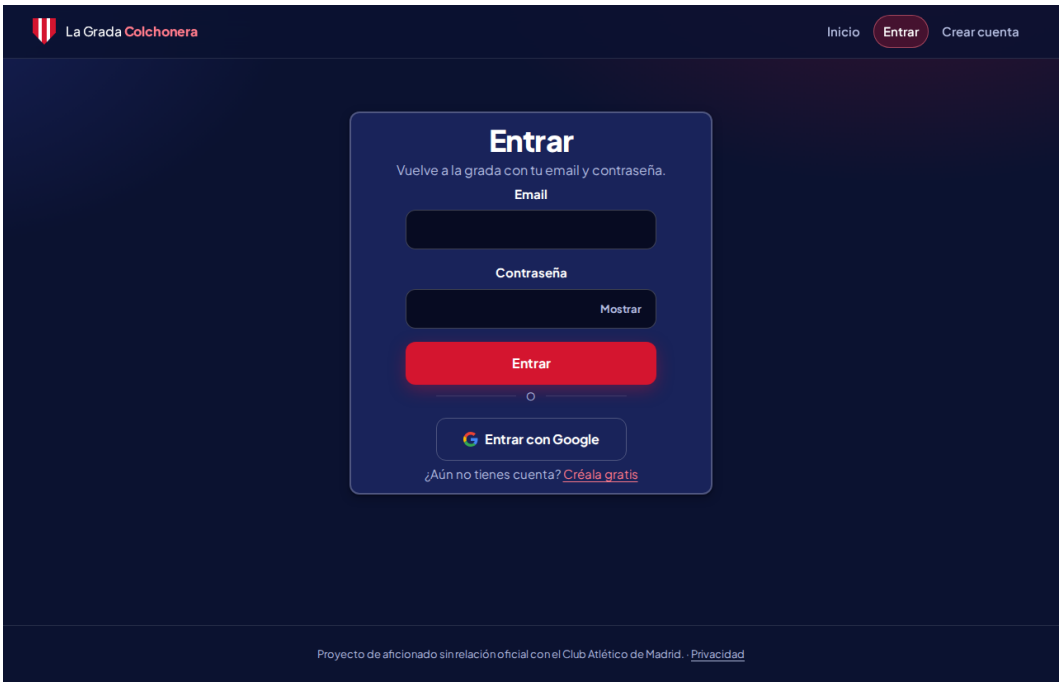


Figura 3.19: Pantalla de entrar con el botón Entrar con Google.

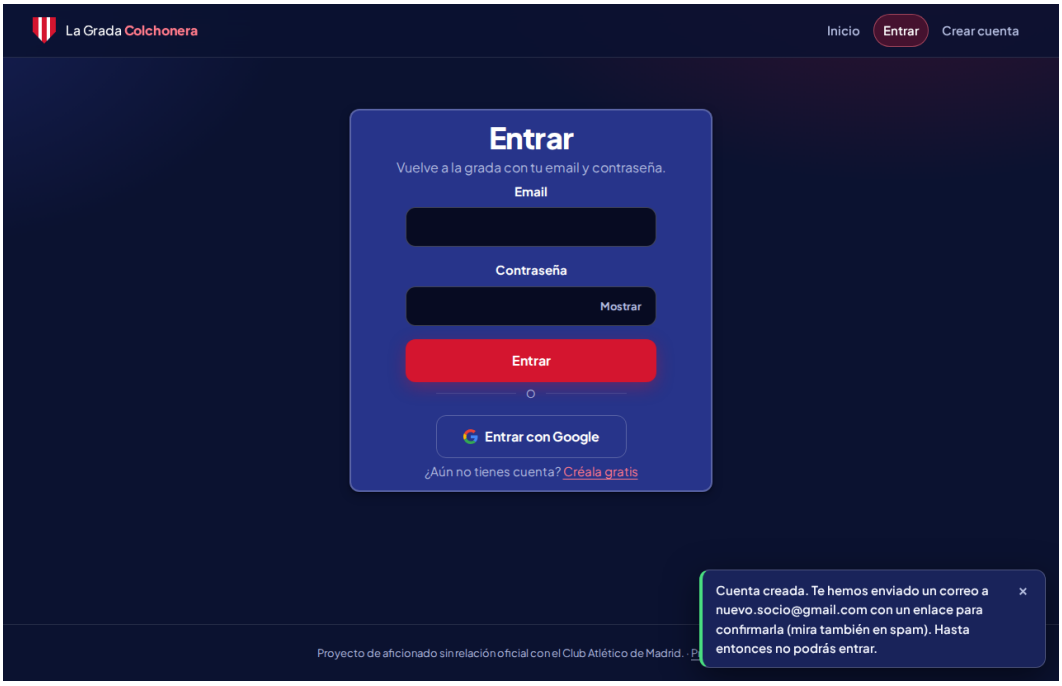


Figura 3.20: Aviso: cuenta creada y correo de confirmación enviado.

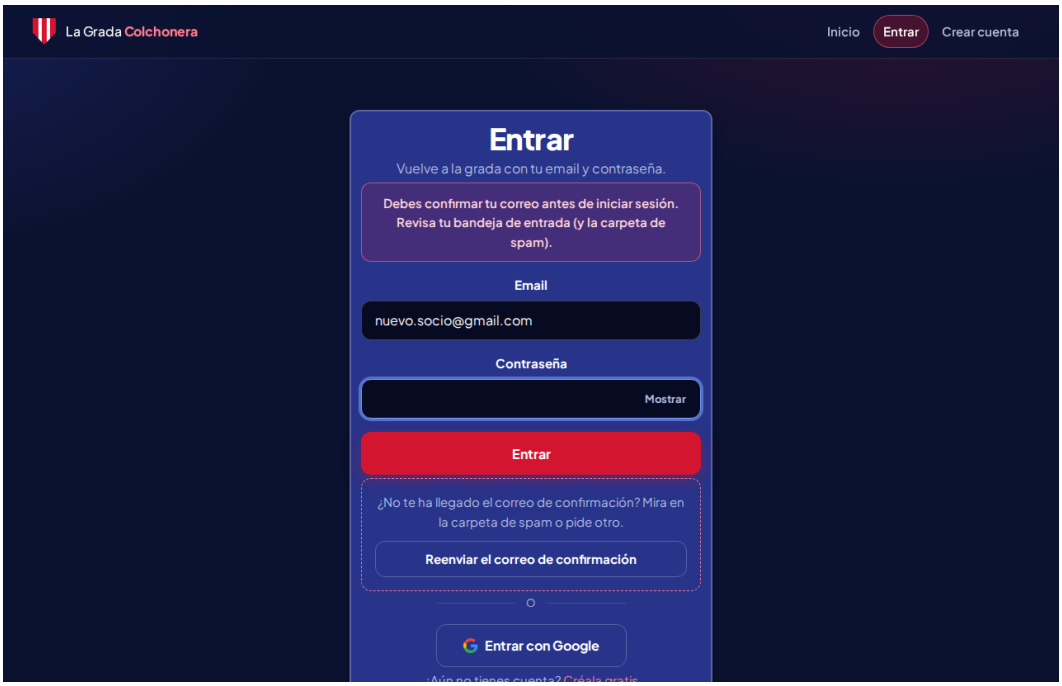


Figura 3.21: Error «Debes confirmar tu correo» con el botón Reenviar el correo de confirmación.

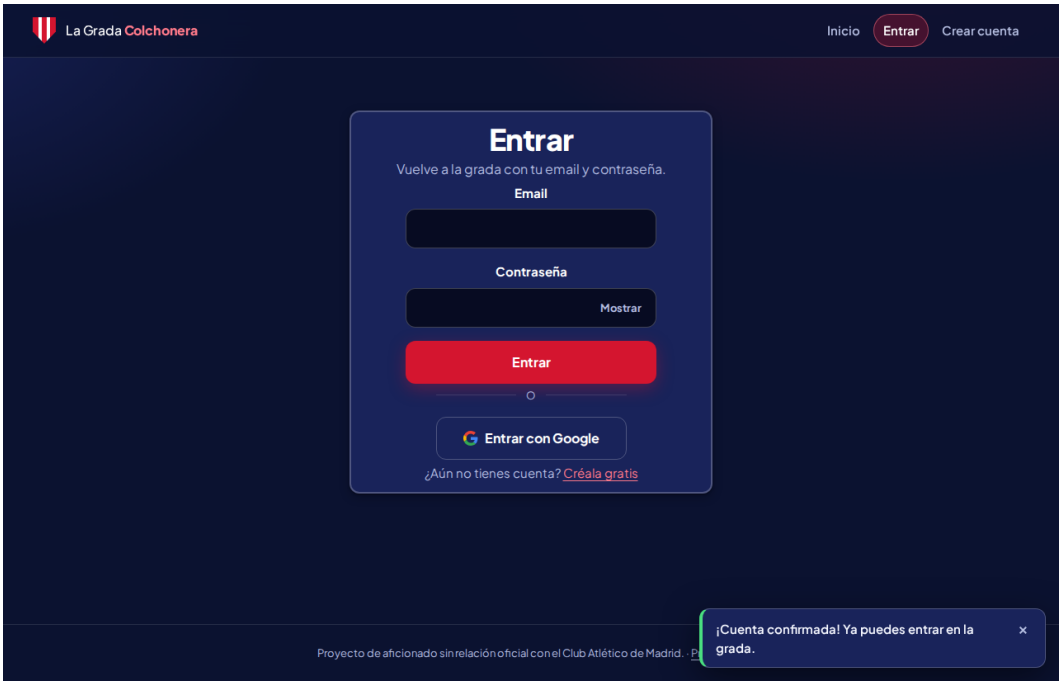


Figura 3.22: Aviso verde: cuenta confirmada, ya puedes entrar.

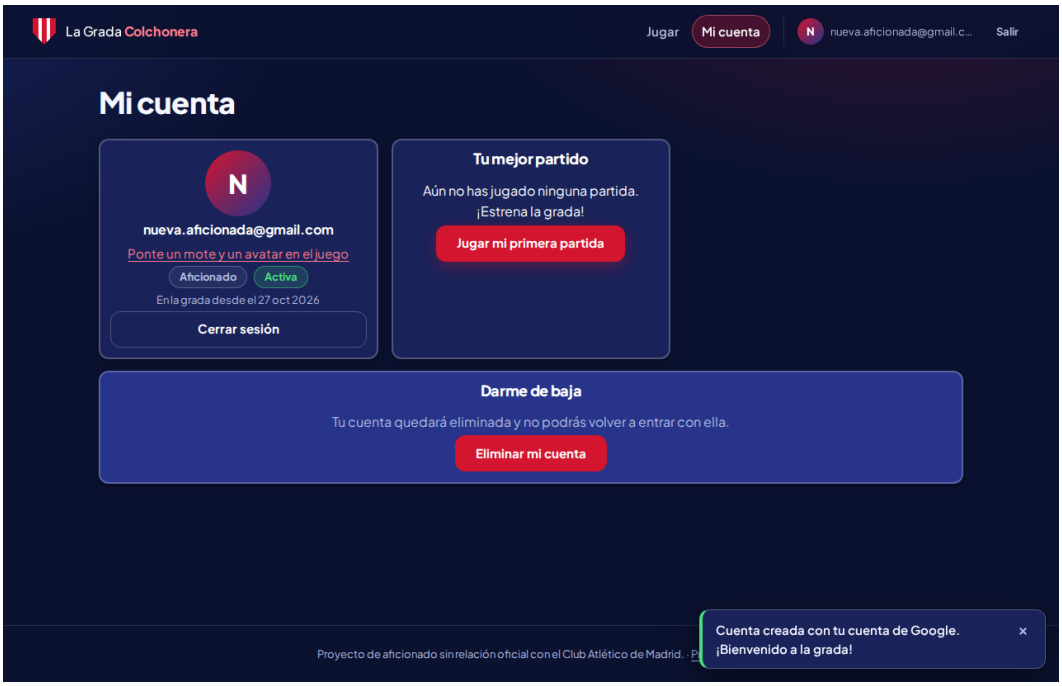


Figura 3.23: Mi cuenta de una cuenta nueva de Google con el aviso «Cuenta creada con tu cuenta de Google».

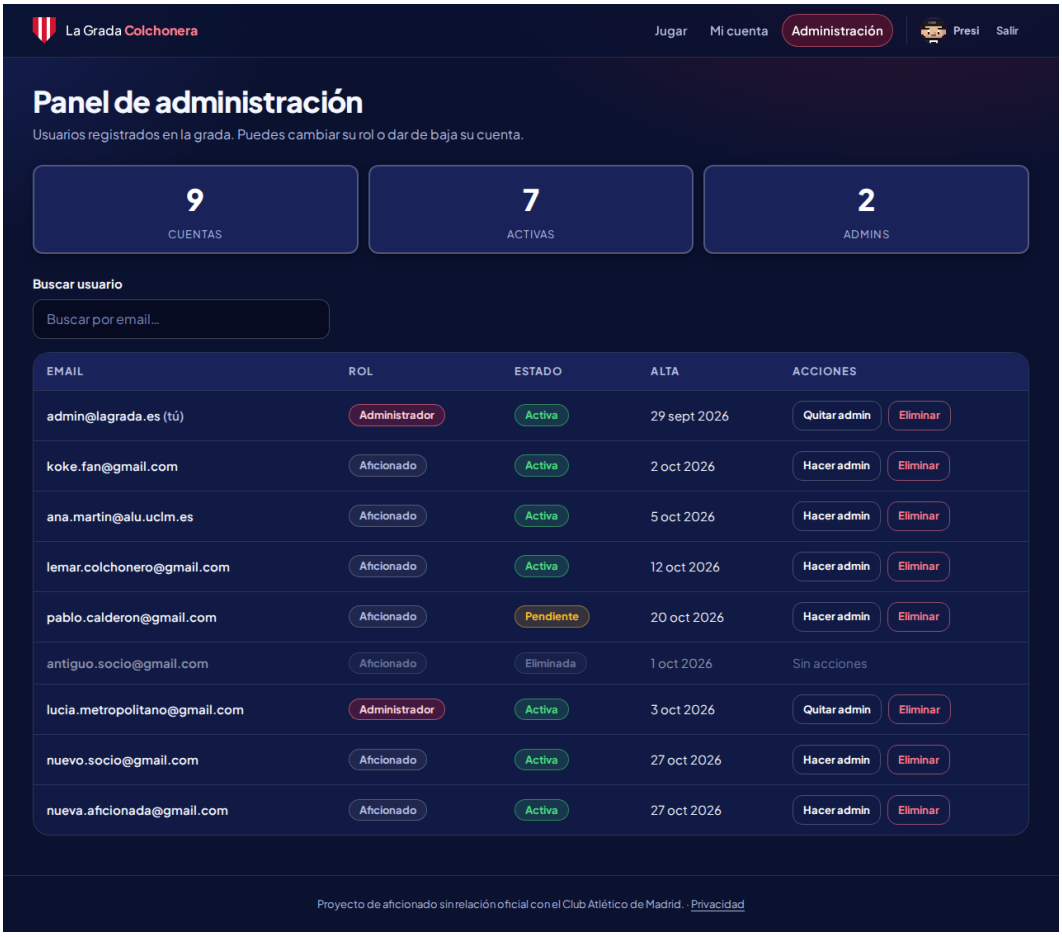


Figura 3.24: Panel de administración con todos los estados de cuenta.



Figura 3.25: Página de política de privacidad.

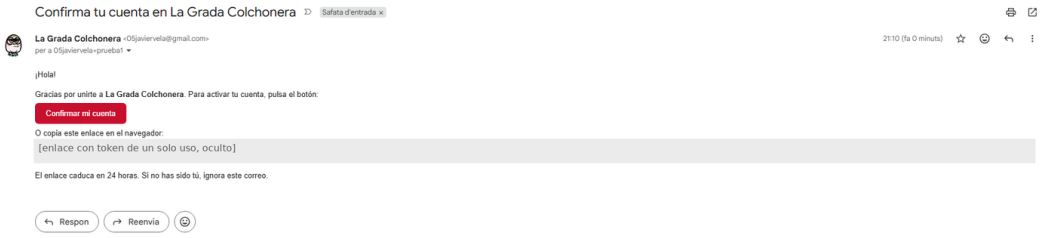


Figura 3.26: Correo de confirmación real recibido en Gmail.

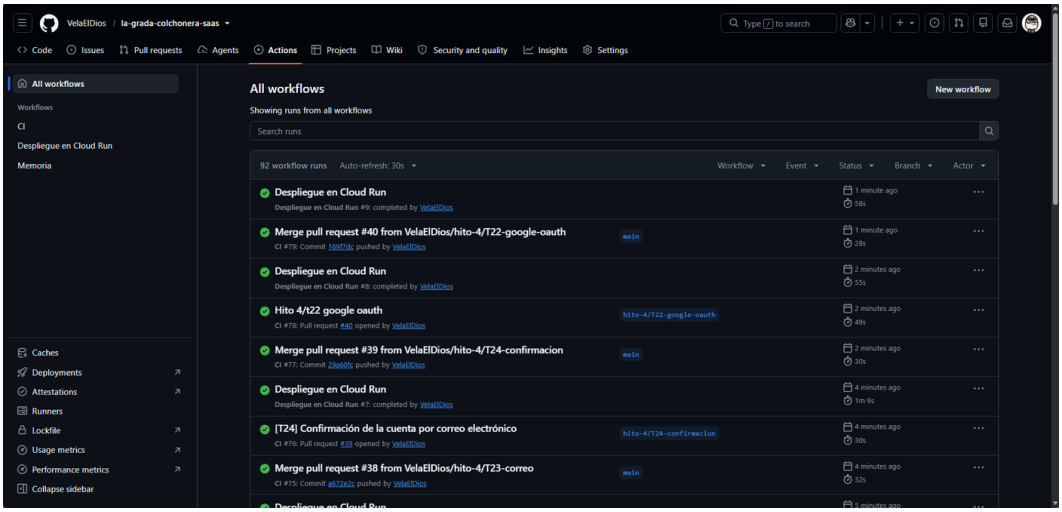


Figura 3.27: GitHub Actions: CI y despliegue en Cloud Run en verde tras los PRs del Hito 4.

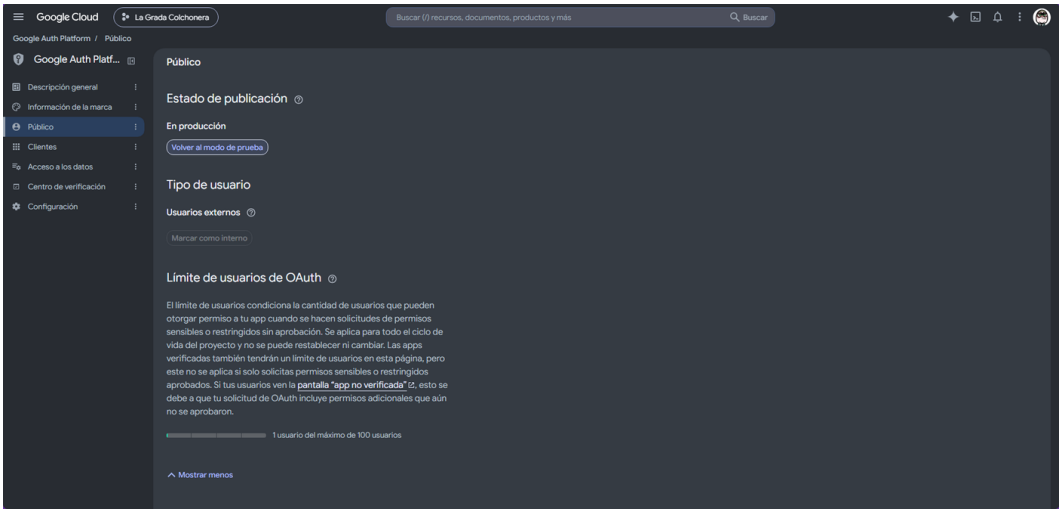


Figura 3.28: Google Auth Platform: aplicación publicada en producción para usuarios externos.

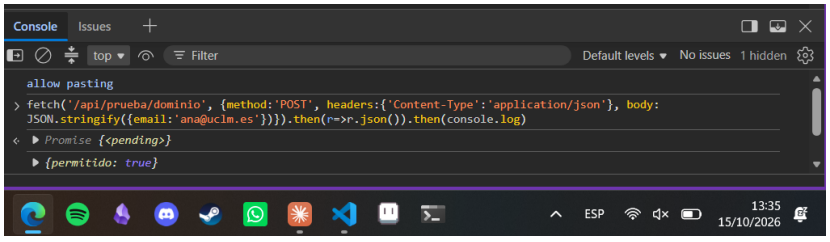


Figura 3.29: Prueba técnica: respuesta permitido: true en la app desplegada.

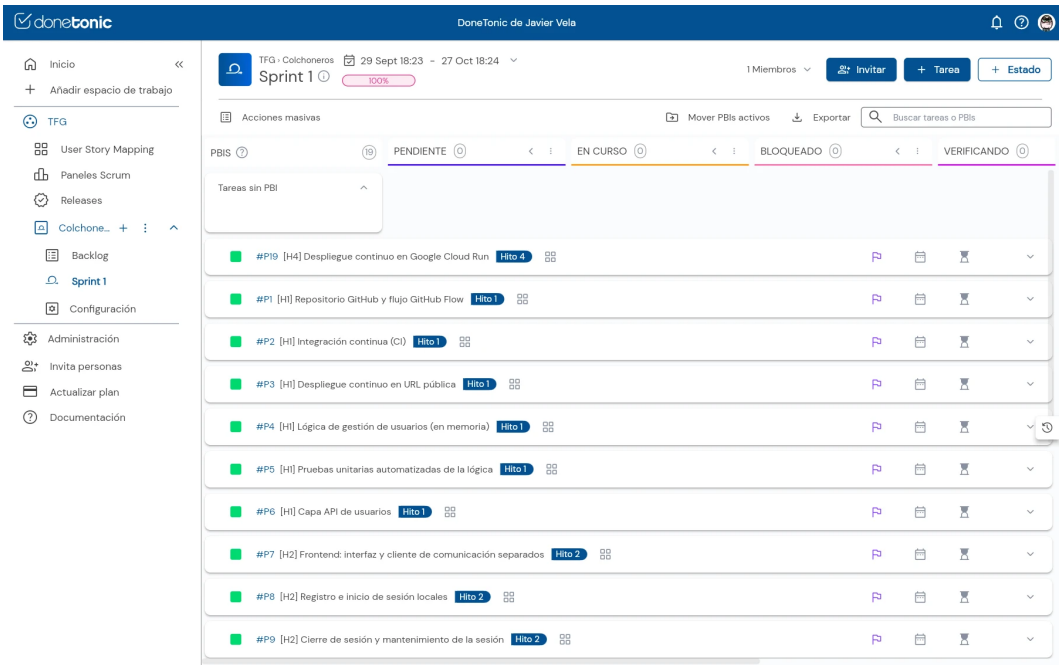


Figura 3.30: Tablero del Sprint 1 en DoneTonic al 100 %: los 19 PBIs hechos.

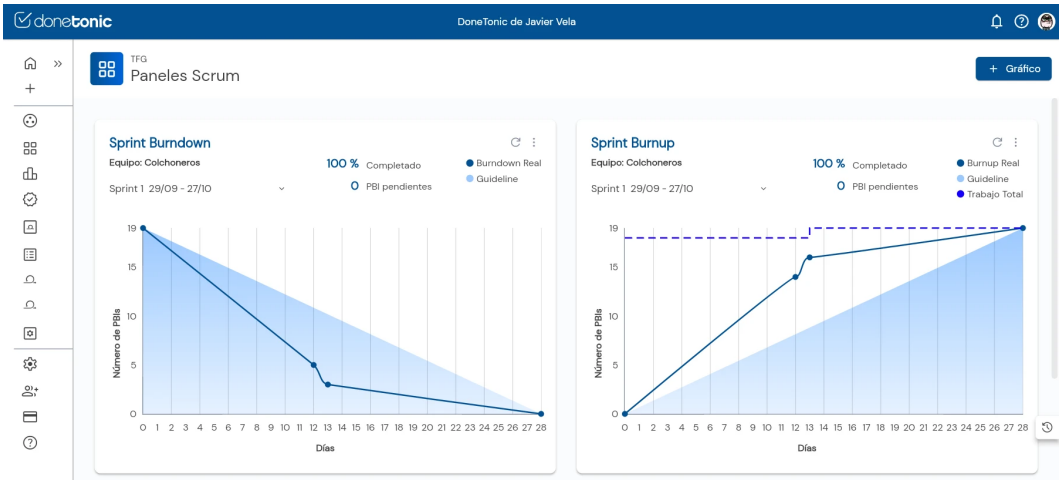


Figura 3.31: Sprint Burndown y Sprint Burnup del Sprint 1 en DoneTonic.

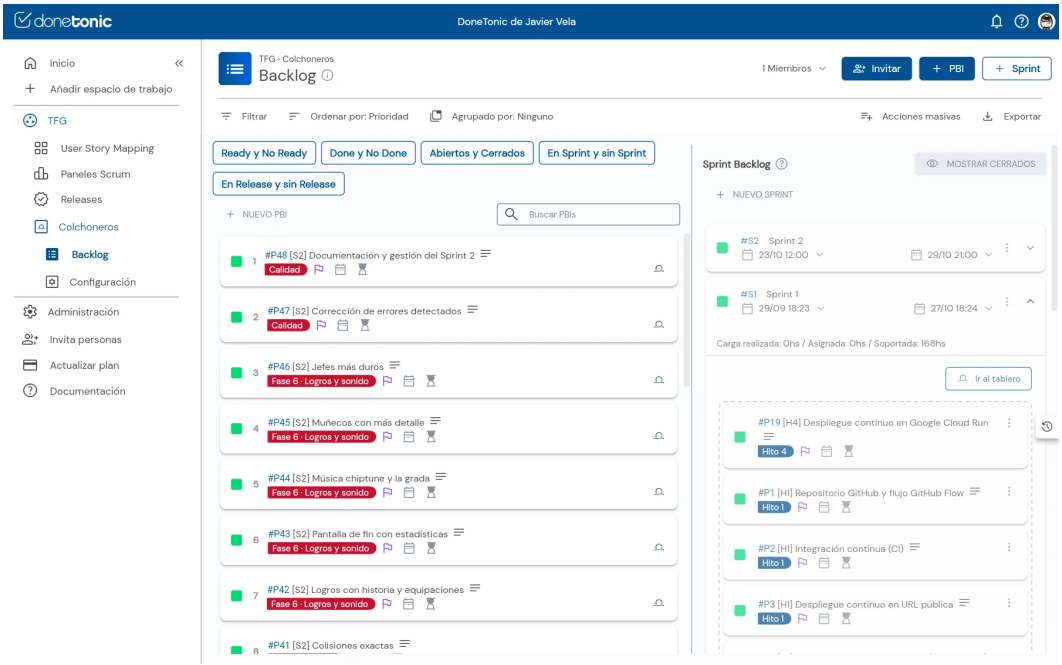


Figura 3.32: Product Backlog de DoneTonic con los dos sprints.

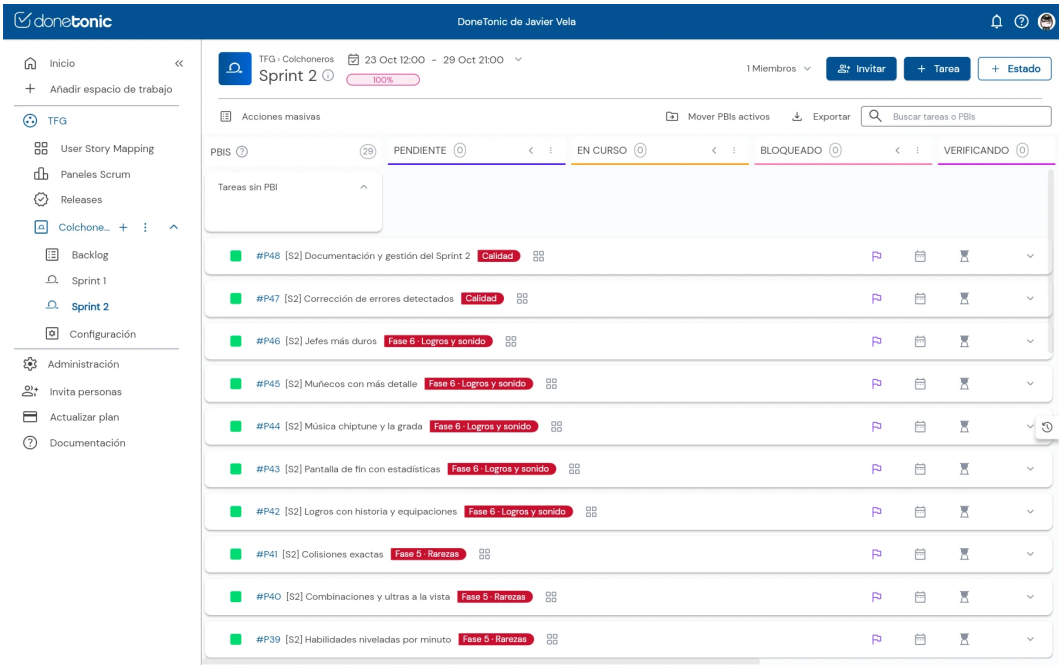


Figura 3.33: Tablero del Sprint 2 en DoneTonic al 100 %: los 29 PBIs hechos.

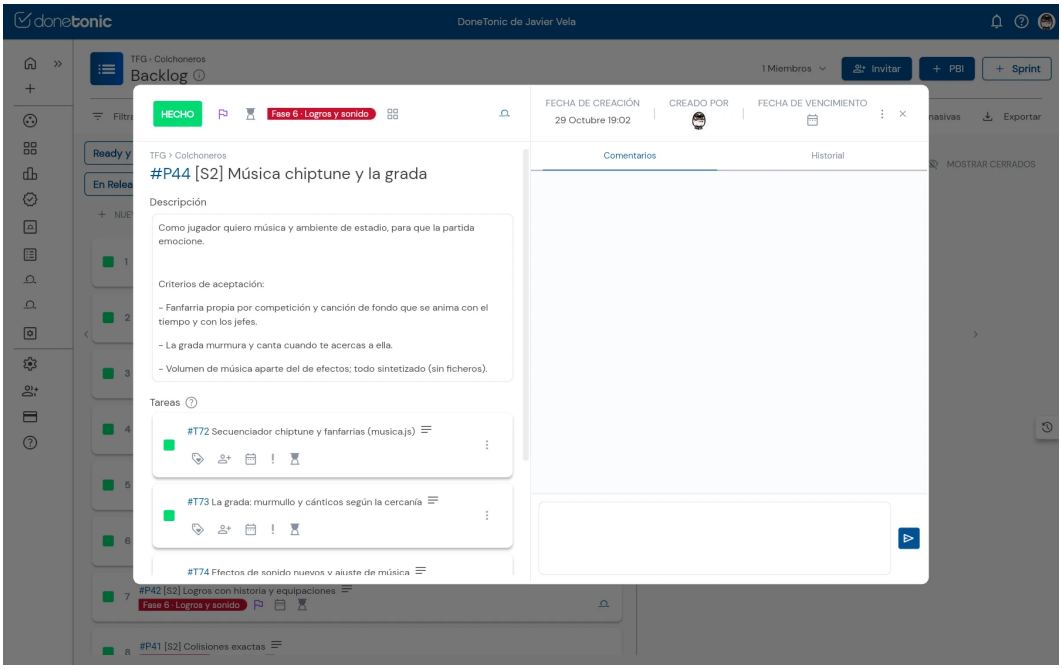


Figura 3.34: Detalle del PBI P44 en DoneTonic con historia de usuario y criterios de aceptación.

4. El juego (Sprint 2)

4.1. Concepto

La Grada Colchonera incluye un videojuego web de género *survivor* (popularizado por *Vampire Survivors* [poncle, 2022]) con estética de píxeles "chibi". El jugador controla a un futbolista de la historia del Atlético de Madrid; las armas atacan solas y la tarea del jugador es moverse, esquivar y elegir mejoras para aguantar oleadas infinitas de rivales y jefes. La progresión entre partidas (club, fichajes, entrenamiento, logros) se guarda en el servidor y se valida con mecanismos anti-trampas.

4.2. Cómo se juega (manual de usuario)

4.2.1. Entrada y menú principal

Con sesión iniciada, la ruta `#/jugar` abre el menú del juego (Figura 4.1). La primera vez se pide un **mote** (3 a 16 caracteres, único) y una **cara** para el avatar.

El menú ofrece: Jugar, Vestuario, Entrenamiento, Contratar, Logros, Combinaciones, Dirección, Ranking y Ajustes; a la izquierda, tu ficha y tus tiempos.

4.2.2. Controles

- **Teclado:** WASD o flechas para moverse; `Espacio`/`E` para la ulti; `Esc`/`P` para pausar; `F` para pantalla completa; `1`–`3` para elegir mejora al subir de nivel.
- **Móvil:** joystick táctil flotante al arrastrar.
- **Ajustes:** volumen de música y efectos, gráficos, cámara, controles; se guardan en `localStorage`.



Figura 4.1: Menú del juego (inicio con sesión): el jugador paseando por el campo, la ficha del aficionado, tus tiempos y el menú lateral.

4.2.3. Partida: oleadas, experiencia y mejoras

La partida es infinita: cada 30 segundos una oleada nueva y un jefe cada cuatro oleadas (el primero a 1:30). Los rivales sueltan experiencia; al subir de nivel se elige una de tres mejoras (las que completan una combinación salen marcadas con ⍰). La Figura 4.2 muestra una partida en curso.

La pantalla de subida de nivel (Figura 4.3) permite elegir una mejora.

4.2.4. Jefes

Cada jefe tiene tres fases (66% y 33% de vida), escolta de élites desde el segundo, furia si se tarda en ganarles y una jugada estrella en la fase 3. La Figura 4.4 muestra al jefe Vinicius Jr.

4.2.5. Ambientes

Cada 5 minutos cambia el ambiente: pretemporada → Final Copa del Rey → Final Liga → Final Champions → Final Mundial (Figura 4.5). Cambian la luz, el césped, la afición, los carteles y los rivales salen más fuertes; en el Mundial todos llevan la camiseta de su selección.



Figura 4.2: Partida en curso: mundo procedural, rivales, armas automáticas y HUD.



Figura 4.3: Subida de nivel: elegir una de tres mejoras.



Figura 4.4: Jefe Vinicius Jr. en el campo con su barra de vida.



Figura 4.5: Ambiente de la Final de la Copa del Rey (a partir del minuto 5).

4.2.6. Fin de partida

Al caer, se muestra la pantalla de estadísticas (tiempo, ambiente, bajas, jefes, daño por habilidad, golpe más fuerte, daño recibido, vida recuperada, logros nuevos) y las recompensas del club (Figura 4.6).



Figura 4.6: Fin de la partida: tiempo, estadísticas, daño por habilidad y logro nuevo.

4.2.7. Vestuario, Entrenamiento, Contratar, Logros, Combinaciones, Dirección, Ranking, Ajustes, Pausa

Las capturas 4.7 a 4.16 ilustran cada pantalla.

La figura 4.8 muestra el entrenamiento: puntos para subir vida, defensa, disparo y suerte; botón modo pruebas (admin).

La figura 4.9 muestra el mercado de fichajes con las fichas de contrato.

La figura 4.10 muestra la vitrina de logros, cada uno con un dato de la historia del Atleti.

La figura 4.11 muestra las metas de la grada: ultras y combinaciones con lo que pide cada una.

La figura 4.12 muestra la dirección deportiva: elegir entrenador (Cholo Simeone o Luis Aragonés) y su ulti.

La figura 4.13 muestra el ranking de la grada: mote y avatar, nunca el correo.

La figura 4.14 muestra la pantalla “Tu ficha”: mote y avatar.



Figura 4.7: Vestuario: lista de jugadores con filtros, ficha de Griezmann y equipaciones.



Figura 4.8: Entrenamiento: puntos para subir vida, defensa, disparo y suerte; botón modo pruebas (admin).

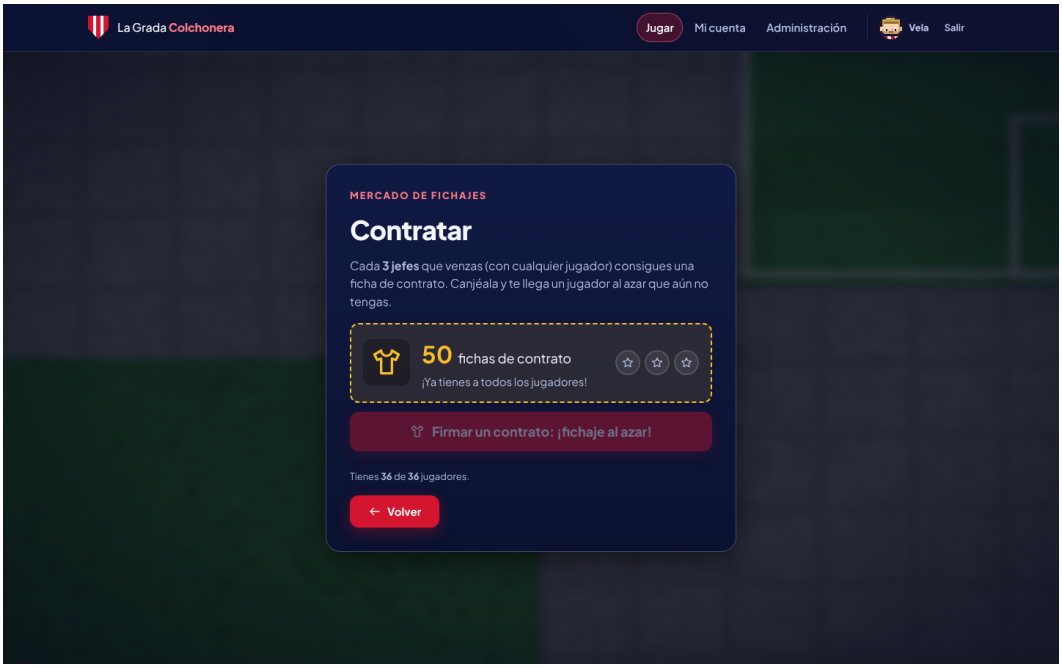


Figura 4.9: Mercado de fichajes con las fichas de contrato.

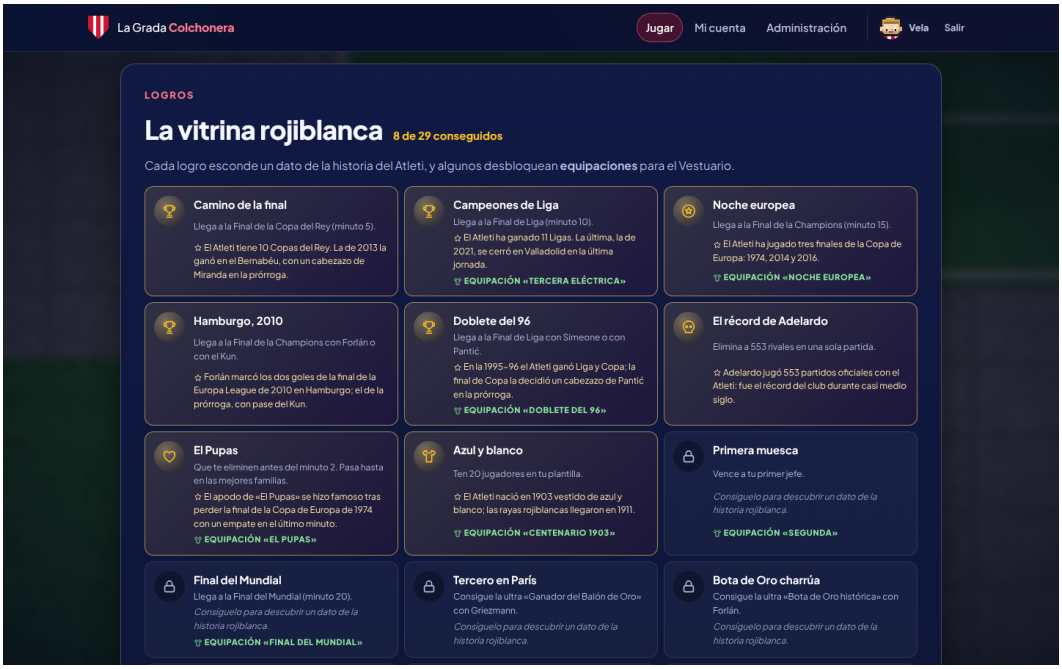


Figura 4.10: Vitrina de logros, cada uno con un dato de la historia del Atleti.



Figura 4.11: Las metas de la grada: ultras y combinaciones con lo que pide cada una.



Figura 4.12: Dirección deportiva: elegir entrenador (Cholo Simeone o Luis Aragonés) y su ulti.



Figura 4.13: Ranking de la grada: mote y avatar, nunca el correo.

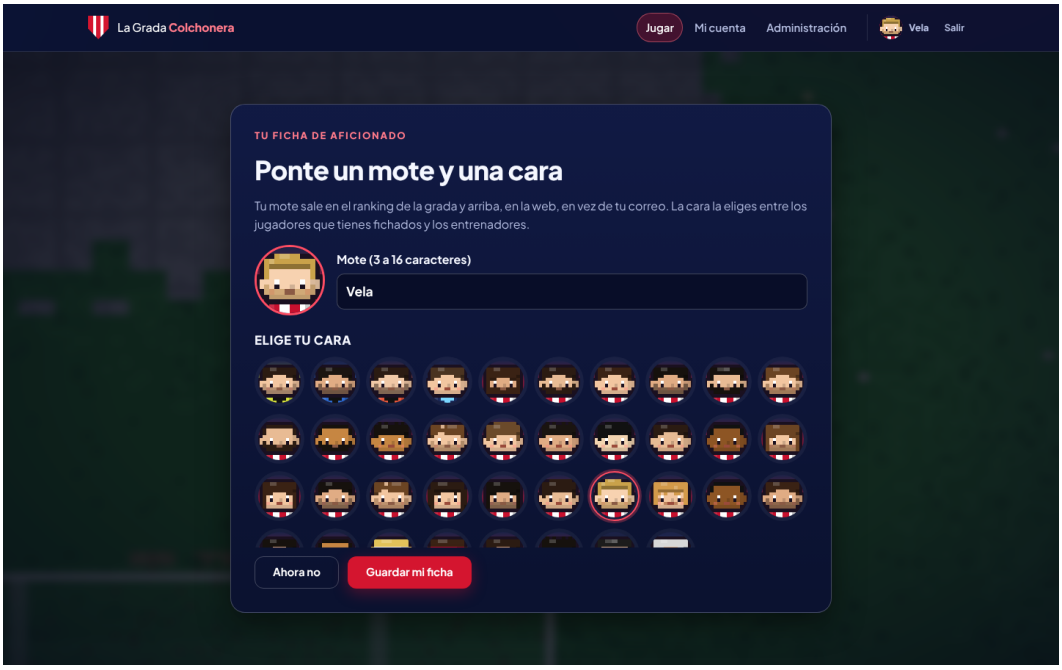


Figura 4.14: Pantalla “Tu ficha”: mote y avatar.

La figura 4.15 muestra los ajustes de volumen, gráficos, cámara y controles.



Figura 4.15: Ajustes de volumen, gráficos, cámara y controles.

4.3. Sistemas

La Tabla 4.1 resume los sistemas del juego.

Tabla 4.1: Sistemas del juego.

Sistema	Descripción	Configuración
Jugadores	36 (4 porteros, 9 defensas, 13 medios, 10 delanteros). Posición fija estadísticas base; rasgos propios y lista de habilidades aprendibles	`PERSONAJES`, `POSICIONES`
Rarezas	Según historia en el Atleti: Leyenda (8), Épico (8), Raro (8), Común (12). Más rareza = más habilidades (8 a 16) y menos probabilidad en contrato	`RAREZAS`

Sistema	Descripción	Configuración
Entrenadores	Cholo Simeone (+daño, +vida; ulti “¡Partido a partido!”) y Luis Aragonés (+velocidad, +experiencia; ulti “Furia Roja”)	`ENTRENADORES`
Armas y habilidades	24, con 5 niveles: disparo, cuerpo a cuerpo, movilidad, especiales. Las más fuertes no suben hasta cierto minuto (`desdeMinuto`) y tienen recarga mínima (`cdMin`)	`ARMAS`
Pasivas	14 (botas, imán, pasión, pizarra, cantera, masajista, espinilleras, pulmones, toque dorado, botas en llamas, paradón, brazalete, trébol, prórroga)	`PASIVAS`
Combinaciones	24 “metas” que se activan al juntar ciertas habilidades a cierto nivel y dan bonus (Muro rojiblanco, Tiki-taka, Pichichi, Catenaccio, Leyenda rojiblanca, Golazo de otro planeta, Remontada épica, Triple Corona, Infierno del Metropolitano, Capitán eterno…)	`SINERGIAS`
Ultras	6 que se consiguen solas al completar varias combinaciones: Premio Puskás, Ganador del Balón de Oro, MVP de la Champions, Trofeo Zamora, Bota de Oro histórica, Mejor jugador del Mundial. Efecto visual propio y daño en parte porcentual (menor contra jefes)	`ULTRAS`, `ultras.js`
Rivales	16 equipos con jugadores genéricos; unos embisten (Liverpool, Arsenal, PSG, Dortmund) y otros disparan (Bayern, City, Chelsea, Juventus, Milan, Inter, Nápoles); élites blindado, veloz y furioso	`EQUIPOS`, `ELITES`
Jefes	10: Guardiola (rondo, tiki-taka), Mourinho (autobús), Cristiano (“Siuuu”), Messi (eslalon), Vinicius (zigzag), Yamal (rosca), Neymar (lluvia de bicicletas), Bale (cañonazos), Benzema (precisión), Suárez (mordiscos). Tres fases, escolta, furia, jugada estrella. Desde oleada 24 pueden salir dos a la vez y cada vuelta completa los hace 25% más duros	`JEFES`, `jefes.js`

Sistema	Descripción	Configuración
Dificultad	Crecimiento continuo de vida, daño y velocidad; tramo final más duro desde minuto 9 y rivales “curtidos” desde 12; multiplicador por ambiente; ayuda inicial que se apaga en 8 minutos	`ESCALADO`, `TRAMO_FINAL`, `CURTIDOS`, `AMBIENTES`
Movimiento	Inercia, *ritmo* al correr (más daño y experiencia), castigo por quedarse quieto, premios por el campo	`FISICA`, `JUGADOR`
Mundo	Procedural e infinito por semilla, con ruidos suaves: distritos (ciudad, entrenamiento, parque, prado), estadios con gradas, plazas y tiendas, mesetas con escaleras, barro, lluvia y caminos. Colisión por los pies (vista 3/4)	`TERRENO`, `terreno.js`
Club	Plantilla inicial de 4 comunes (uno por posición); una ficha de contrato por cada 3 jefes de la cuenta; puntos de entrenamiento por jefes (1 cada 3) y por bajas; estadísticas Vida, Defensa, Disparo y Suerte (máx. 20)	`club.js`
Logros	29 con dato curioso de la historia del Atleti (tabla 9.6); 10 equipaciones desbloqueables	`logros.js`
Sonido	Efectos, música chiptune original (canción por ambiente que se anima con el tiempo y con los jefes, fanfarria por competición) y la grada (murmullo y cánticos según la distancia a las gradas)	`audio.js`, `musica.js`

4.4. Arquitectura del juego

La Figura 4.17 muestra la arquitectura del juego.

4.4.1. Determinismo

`mundo.js` avanza con paso fijo de 1/60 s y generador aleatorio con semilla. Si el navegador va lento, se hacen varios pasos por fotograma (con tope); el resultado no depende de los FPS y las pruebas son reproducibles.



Figura 4.16: Menú de pausa de la partida.

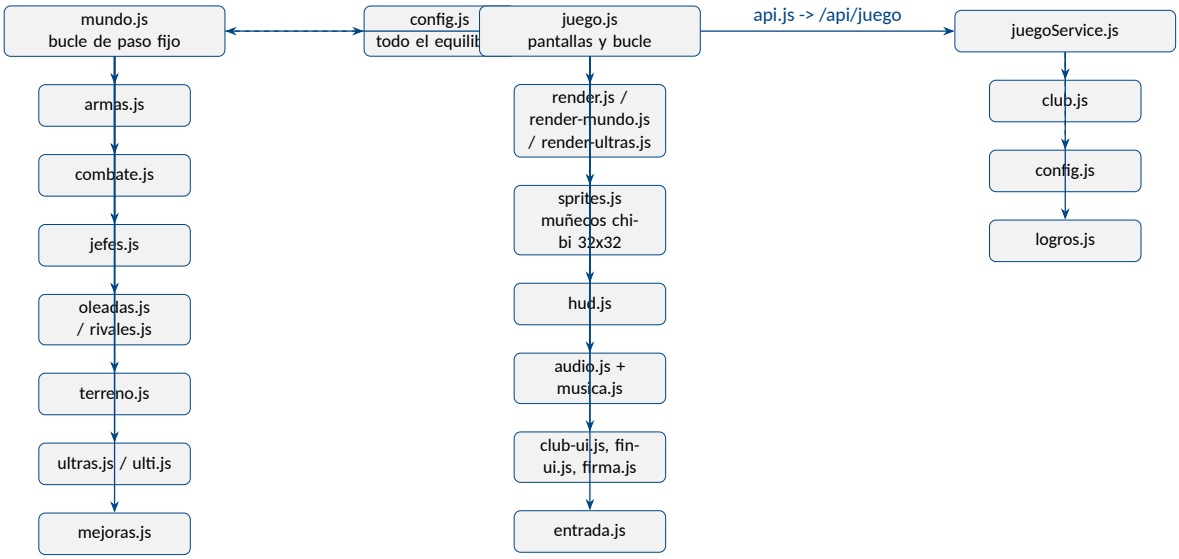


Figura 4.17: Arquitectura del juego: simulación pura (izquierda), presentación (centro), servidor (derecha).

4.4.2. Eventos

La simulación no toca el DOM ni el audio: deja eventos (`oleada`, `aviso`, `jefe-derrotado`, `ambiente`, `ultra`…) que `juego.js` traduce a sonido, avisos y anuncios para lectores de pantalla.

4.4.3. Estadísticas

Cada fuente de daño (arma, ultra, ulti, quemaduras) se apunta en `mundo.stats` para la pantalla de fin.

4.4.4. Rendimiento

Búsqueda de obstáculos por celdas con claves numéricas (unas 4,5 veces más rápida que la primera versión), una textura de suelo por celda y ambiente, pintada como mucho una por fotograma y por adelantado, y muñecos cacheados.

4.4.5. Dibujo

Canvas 2D con cámara 3/4 (el suelo se aplasta en vertical y los muñecos están de pie). Los muñecos se pintan con código en una cuadrícula de 32×32 en estilo chibi (16×20 con piernas hasta la Fase 6): cabeza grande, cuerpo y zapatones, sin piernas. Se pintan por capas y píxel a píxel: zapatos, pantalón, torso, brazos y cabeza, cada una con su contorno de color y luz desde arriba a la izquierda. Cada persona tiene sus rasgos (18 peinados, 6 tipos de barba, mandíbula, complexión, estatura, gesto, gafas, cinta, guantes, brazalete…) y caminan con 4 fotogramas. Así se generan las equipaciones y las camisetas de 26 selecciones sin imágenes. El mapa (Fase 7) se pinta con píxeles de 2 unidades en `render-suelo.js` y `render-obstaculos.js`, con las paletas de cada ambiente en `render-paletas.js`, y los efectos de las habilidades en `render-habilidades.js` (partículas con tope y azar propio en `efectos.js`, para que encenderlas o apagarlas no cambie la partida).

4.4.6. Audio

Todo sintetizado con Web Audio: ondas de pulso (12,5%, 25% y 50%), triángulo y ruido. La música es un secuenciador propio que programa las notas por adelantado; la grada usa voces con formantes de vocales y palmas.

4.5. Servidor del juego y anti-trampas

- Al **empezar**, el servidor guarda la partida en curso con su propio reloj.
- Al **terminar**, rechaza el resultado si la partida no es la que está en curso, si el tiempo supera lo transcurrido (con 15 s de margen) o 4 horas, o si los jefes, el nivel o las bajas no son creíbles para ese tiempo (`jefes  3 + t/50`, `nivel  8 + t/3`, `bajas  20·t + 50`).

- De los **logros** que manda el cliente, solo apunta los que puede comprobar con lo que sabe (tiempo, jefes, bajas, jugador y entrenador); los de plantilla los calcula él.
- Cada cambio del perfil usa **concurrencia optimista** (`UPDATE ... WHERE version = $n`) con reintentos.
- El **ranking** guarda la mejor partida de cada usuario con su mote y avatar, nunca el correo.

4.6. Diagramas de secuencia del juego

4.6.1. Una partida del juego (inicio, juego y fin con validación)

La Figura 4.18 muestra el flujo de una partida con validación en el servidor.

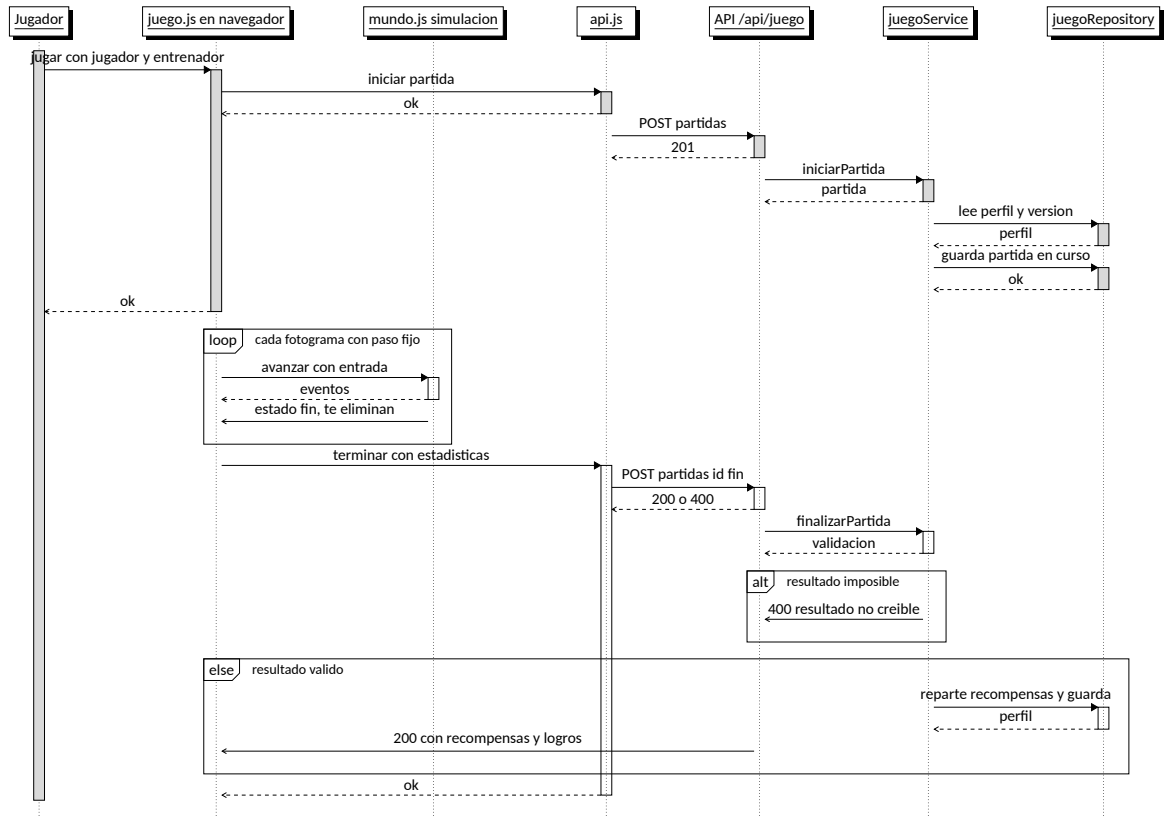


Figura 4.18: Una partida del juego con validación en el servidor.

4.6.2. Contratar con una ficha (concurrencia optimista)

La Figura 4.19 muestra el flujo de contratación con concurrencia optimista.

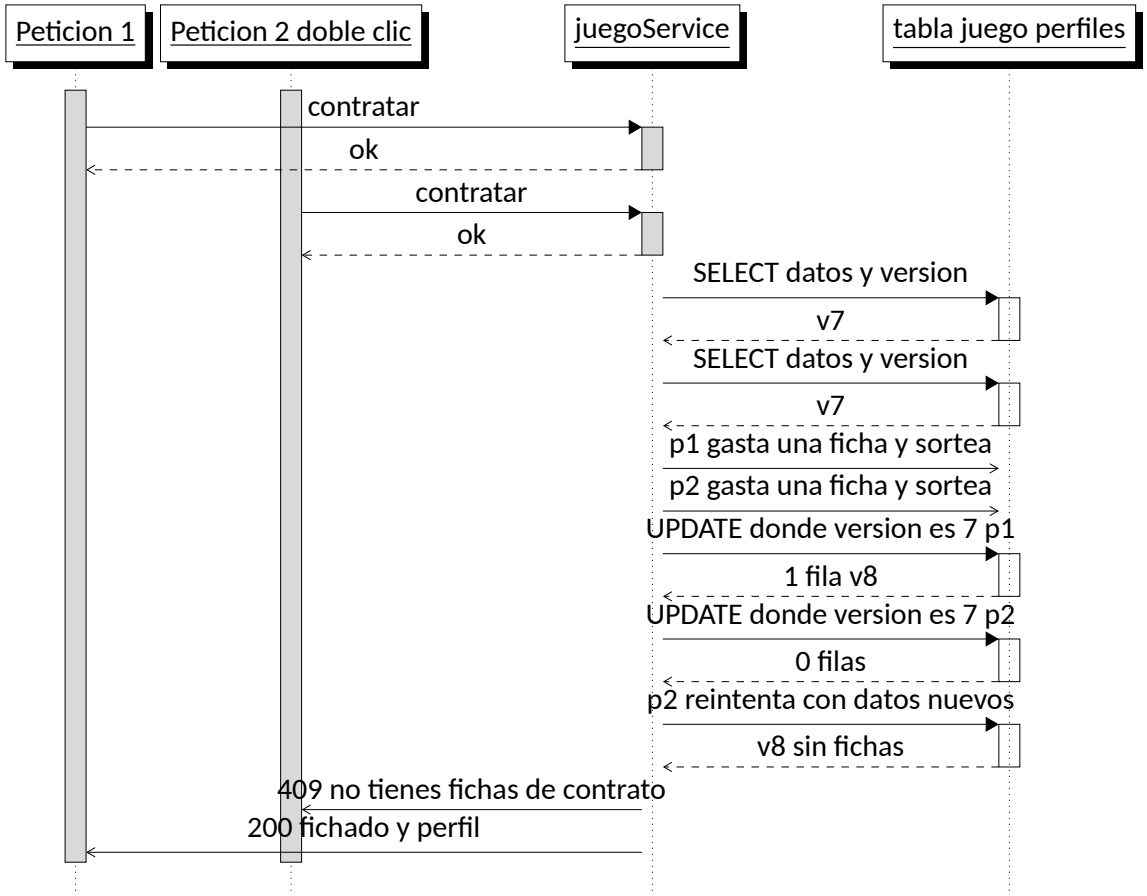


Figura 4.19: Contratar con concurrencia optimista.

4.7. Logros

La Tabla 4.2 lista los 29 logros con su condición y equipación desbloqueable.

Tabla 4.2: Logros y equipaciones.

Logro	Cómo se consigue	Equipación
Primera muesca	Vence a tu primer jefe	Segunda
Camino de la final	Llega a la Final de la Copa del Rey (minuto 5)	
Campeones de Liga	Llega a la Final de Liga (minuto 10)	Tercera eléctrica
Noche europea	Llega a la Final de la Champions (minuto 15)	Noche europea
Final del Mundial	Llega a la Final del Mundial (minuto 20)	Final del Mundial
Tercero en París	Consigue la ultra “Ganador del Balón de Oro” con Griezmann	
Bota de Oro charrúa	Consigue la ultra “Bota de Oro histórica” con Forlán	
Hamburgo, 2010	Llega a la Final de la Champions con Forlán o con el Kun	
Noche de Mónaco	Vence a 3 jefes en una partida con Falcao	
El cabezazo del Camp Nou	Consigue la combinación “Dominio aéreo” con Godín	
Brazalete eterno	Consigue la combinación “Capitán eterno” con Gabi	
El Niño	Llega a la Final de la Copa del Rey con Fernando Torres	
Del Oporto al Calderón	Consigue la combinación “Jugón” con Futre	
Doblete del 96	Llega a la Final de Liga con Simeone o con Pantić	Doblete del 96
Bernabéu, 2013	Llega a la Final de la Copa del Rey con Miranda o con Courtois	
Cinco Zamoras	Consigue la ultra “Trofeo Zamora” con Oblak	Leyenda bajo palos
Los 27 de Costa	Elimina a 270 rivales en una partida con Diego Costa	
La Araña campeona	Llega a la Final del Mundial con Julián Álvarez	
El récord de Adelardo	Elimina a 553 rivales en una sola partida	

Logro	Cómo se consigue	Equipación
El Sabio de Hortaleza	Aguanta 10 minutos con Luis Aragonés en el banquillo	
Partido a partido	Aguanta 15 minutos con el Cholo en el banquillo	
El muro	Aguanta los 5 primeros minutos sin que te toquen	
Poder de leyenda	Consigue tu primera ULTRA	
Leyenda total	Consigue tres ULTRAS en una partida	
Mil en el Metropolitano	Elimina a 1000 rivales en una partida	
El Pupas	Que te eliminen antes del minuto 2	El Pupas
Toda una vida	Juega 20 partidas con Koke	Adiós al Calderón
Azul y blanco	Ten 20 jugadores en tu plantilla	Centenario 1903
Paseo de Leyendas	Ficha a las 8 leyendas	

Cada logro esconde un dato real del club (por ejemplo, que Griezmann fue tercero en el Balón de Oro de 2018, que Oblak ganó cinco Zamoras seguidos o que el Atleti nació en 1903 vestido de azul y blanco). Los datos están en `logros.js` y se han comprobado uno a uno.

4.8. Evolución por fases

Cada fase es una etiqueta del Sprint 2 en DoneTonic. La Tabla 4.3 resume la evolución.

Tabla 4.3: Evolución del juego por fases.

Fase	Qué se añadió	Problemas que resolvió
Rediseño	Nueva interfaz (tokens, tipografía, modo oscuro rojiblanco), menú del juego como inicio con sesión	La interfaz del sprint era funcional pero sobria
1	Survivor base: simulación, oleadas, armas, mejoras, jefes y menú con el muñeco paseando	—
2	Club en el servidor (plantilla, contratos, entrenamiento), ranking de la grada, élites y poderes nuevos	Progreso ligado a la cuenta y validado
3	6 ultras, 24 combinaciones, dificultad con castigo, entrenamiento por bajas y modo pruebas	Maxear una sola cosa ya no basta
4	Mundo procedural natural, 5 ambientes por finales, mote y avatar	Mapa repetitivo; correo visible en el ranking

Fase	Qué se añadió	Problemas que resolvió
5	Rarezas por historia rojiblanca, habilidades niveladas, combinaciones a la vista, animación de la firma, colisiones exactas	Regate demasiado fuerte; esconderse detrás de los tejados
6	29 logros con historia y 10 equipaciones, estadísticas de fin, música chiptune y grada, muñecos con más detalle, jefes con escolta, furia y jugada estrella	Falta de objetivos a largo plazo; sonido pobre; jefes fáciles
7	Muñecos chibi de 32x32 (sin piernas) con rasgos propios, mapa con mucho más detalle y luz por ambiente, menú rediseñado (paneles translúcidos, centro despejado y botón de entrada), 5 habilidades (carambola, presión alta, la ola, fuera de juego y ruleta), 2 pasivas (visión de juego y partidos grandes), 5 combinaciones y mejores efectos de todas las habilidades	Muñecos casi iguales entre sí; mapa plano; menú estático; efectos poco legibles

5. Evaluación

5.1. Diseño de la evaluación

La evaluación de la calidad y del cumplimiento de la especificación se diseñó como un proceso continuo y reproducible. El marco de verificación descansa en cuatro pilares:

1. **Pruebas unitarias e integración en local:** Uso del ejecutor nativo `node:test` con el simulador de base de datos en memoria `pg-mem`, lo que permite ejecutar la suite completa en el entorno de desarrollo mediante el comando unificado `npm test`.
2. **Integración continua en GitHub Actions:** Ejecución del flujo de trabajo automatizado en cada pull request y push hacia la rama `main`. Dicho flujo despliega un servicio PostgreSQL 17 real para ejecutar las pruebas contra la base de datos auténtica y valida la construcción de la imagen de producción en Docker mediante un contenedor `node:22-alpine`.
3. **Comprobación del despliegue en Google Cloud Run:** El flujo de despliegue valida la salud del servicio desplegado mediante comprobaciones de consulta contra el endpoint `/api/health`.
4. **Revisión frase a frase contra la especificación:** Auditoría documentada en `docs/revision-entrega.md`, contrastando cada apartado del enunciado con su evidencia concreta en código y pruebas.

5.2. Estrategia de pruebas y resultados

5.2.1. Pruebas automáticas

La batería de pruebas automatizadas consta de 276 pruebas, divididas en 140 del Sprint 1 (cuentas, API, base de datos, seguridad, prueba técnica) y 136 del juego (simulación, club, movilidad, API juego, personajes, habilidades, ultras, logros, repositorio). El comando único `npm test` las ejecuta todas; en local usan `pg-mem` y en el CI PostgreSQL 17 real. La Tabla 5.1

detalla la distribución de pruebas por fichero y la funcionalidad específica comprobada en cada caso.

Tabla 5.1: Pruebas por fichero.

Fichero	Pruebas	Qué comprueba
`userService.test.js`	27	Registro, email repetido o inválido, contraseña corta, autenticación, pendiente, eliminada, volver con el mismo email tras la baja, OAuth, roles, bajas, último administrador, primer administrador
`api.auth.test.js`	16	Registro (201 sin hash), 409/400, login, mismo mensaje en 401, cookie HttpOnly, me, logout, sesión de una cuenta eliminada, confirmación, enlace no válido, URL pública, reenvío 202, correo caído (503)
`userRepository.test.js`	16	Contrato del repositorio en memoria y en PostgreSQL, incluida la cuenta eliminada que no ocupa su email
`frontend.test.js`	14	Separación interfaz/cliente, sin HTML de usuario, módulos importables
`domainPolicy.test.js`	13	Prueba técnica (8 de rechazo o error)
`api.users.test.js`	11	401, 403, listado, activo, cambio de rol, bajas propias y ajenas, último administrador
`googleOAuth.test.js`	11	Proveedores disponibles, redirección con state, cuenta nueva, existente y pendiente que se confirma, state incorrecto, ausente y reutilizado, cancelación, email sin verificar, volver a entrar con Google tras darse de baja (cuenta nueva), cliente real con fetch falso y configuración
`config.test.js`	8	Secretos obligatorios en producción, valores por defecto, .env fuera del repositorio
`accountConfirmation.test.js`	6	Correo con enlace, confirmación, enlace manipulado/caducado, eliminada, doble confirmación, reenvío
`logger.test.js`	6	JSON, niveles, ocultación de datos sensibles
`tokenSigner.test.js`	3	Firma, manipulación y caducidad
`mailer.test.js`	3	SMTP o registro
`migrate.test.js`	3	Crea las tablas, es idempotente y aplica las restricciones (email único, roles válidos)
`bootstrap.test.js`	2	Arranque con memoria y con PostgreSQL
`health.test.js`	1	/api/health

Fichero	Pruebas	Qué comprueba
`api.database.test.js`	CI	Recorrido completo con PostgreSQL real
`juego-club.test.js`	27	Club, contratos, entrenamiento, rarezas, ranking
`juego-movilidad.test.js`	22	Mundo, colisiones, mesetas, alcance del mapa, ambientes
`api.juego.test.js`	15	API del juego, anti-trampas, logros en el servidor, concurrencia, cuenta eliminada que sale del ranking
`juego-personajes.test.js`	14	Personajes, entrenadores, habilidades por jugador
`juego-habilidades.test.js`	14	Habilidades nuevas y niveles por minuto
`juego-ultras.test.js`	13	Ultras: cómo se consiguen y qué hacen
`juego-logros.test.js`	13	Logros, equipaciones, estadísticas, pantalla de fin, música y jefes más duros
`game.test.js`	10	Simulación base (paso fijo, semilla, armas, jefes, ulti, dificultad)
`juegoRepository.test.js`	8	Repositorio del juego en memoria y PostgreSQL
Total	**276**	140 del Sprint 1 y 136 del juego

5.2.2. Comprobaciones manuales y visuales

Además de las pruebas automáticas, cada cambio visible se ha comprobado en el navegador con Playwright mediante capturas automatizadas de cada pantalla y estado. El Hito 4 se probó a mano en producción: correo real recibido en Gmail, enlace manipulado, acceso con dos cuentas de Google distintas. Las capturas de la aplicación se generan con la aplicación real y datos de ejemplo.

5.3. Integración y despliegue continuos (CI/CD)

El pipeline de CI/CD se implementa con tres workflows de GitHub Actions y un webhook en Render. La Tabla 5.2 describe los workflows de integración continua y despliegue con sus eventos de activación y tareas asociadas.

Tabla 5.2: Workflows de CI/CD.

Workflow	Cuándo	Qué hace
`ci.yml` (CI)	Cada PR y push a main, hito-*/integracion	Trabajo *Pruebas*: `npm ci` + `npm test` con PostgreSQL 17. Trabajo *Imagen Docker*: construye imagen, arranca y comprueba `/api/health`
`deploy-cloud-run.yml`	CI verde en main (`workflow_run`)	Autentica en Google Cloud con Workload Identity Federation, construye y sube imagen a Artifact Registry y despliega en Cloud Run con variables y 8 secretos de Secret Manager
`memoria.yml`	Cambios en memoria/	Compila la memoria con LuaLaTeX (latexmk + biber) y publica el PDF como artefacto
Render (webhook)	Cada cambio en main	Despliegue alternativo según `render.yaml`

La Figura 5.1 ilustra el diagrama del pipeline de integración y despliegue continuos.

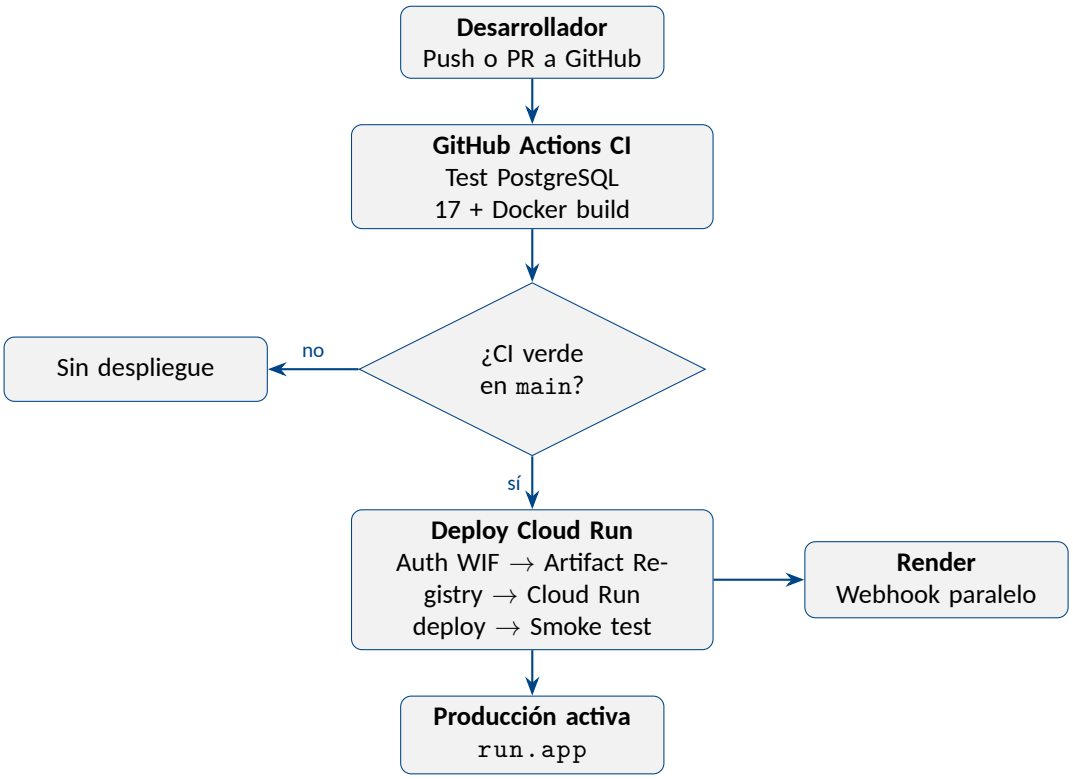


Figura 5.1: Pipeline de integración y despliegue continuos.

Como evidencia visual del flujo automatizado, la captura de la Figura 5.2 demuestra la

ejecución exitosa en verde tanto de la integración continua como del despliegue en Google Cloud Run.

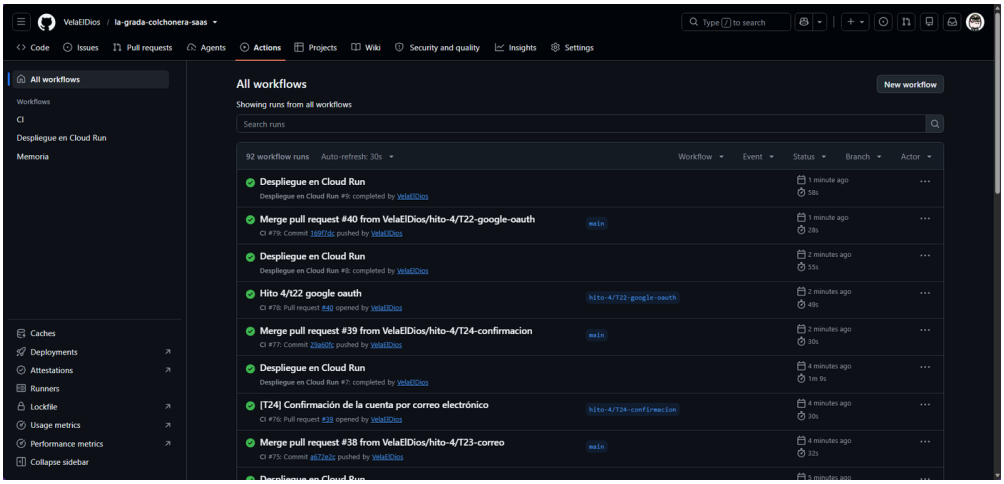


Figura 5.2: GitHub Actions: flujos de CI y despliegue en Cloud Run en verde tras los PRs del Hito 4.

5.4. Registro de actividad y manejo de errores

5.4.1. Registro de actividad (logs)

El sistema genera una línea JSON estructurada por cada evento relevante en la salida estándar (capturada por Cloud Logging en producción), incluyendo los campos `time`, `level`, `event`, `msg` y metadatos asociados. Cada petición HTTP recibe un identificador único (`X-Request-Id`) que se propaga a todas las trazas generadas en su ámbito. Los eventos tipados contemplan: `app.start`, `app.stop`, `app.storage`, `app.mail`, `admin.bootstrap`, `http.request`, `http.error`, `user.registered`, `auth.login`, `auth.logout`, `auth.login_failed`, `auth.forbidden`, `mail.confirmation_sent`, `mail.failed`, `mail.not_sent`, `user.confirmed`, `user.confirm_failed`, `auth.oauth_denied`, `auth.oauth_failed`, `user.role_changed`, `user.deleted` y `juego.identidad`. Para garantizar la confidencialidad, contraseñas, hashes, cookies, tokens y secretos se reemplazan automáticamente por el texto `[oculto]`.

5.4.2. Manejo de errores

El tratamiento de errores opera de forma uniforme desde la capa de lógica hasta la interfaz:

1. Los servicios de negocio lanzan excepciones tipadas (`ValidationError`, `Unauthorized`

Error, ForbiddenError, AccountPendingError, NotFoundError, ConflictError, ServiceUnavailableError).

2. El middleware errorHandler mapea cada error a su correspondiente código de estado HTTP y genera una respuesta estándar con formato {"error": {"code", "message"}}.
3. El cliente de comunicación api.js transforma las respuestas fallidas en objetos de error ApiError e implementa un tiempo límite de espera de 15 segundos mediante AbortController.
4. Las vistas de la interfaz muestran alertas específicas para cada caso, impidiendo que el usuario final visualice trazas internas o formatos crudos.

5.5. Revisión frente al enunciado

La auditoría frase a frase documentada en docs/revision-entrega.md confirmó el cumplimiento íntegro de las prescripciones del enunciado:

- Arquitectura obligatoria cliente-servidor con frontend servido desde el mismo origen sin acceso directo a base de datos.
- Gestión de usuarios completa: registro local, confirmación por correo, inicio de sesión local y con Google OAuth 2.0, sesiones en servidor e invalidación al cierre.
- Esquema de autorización con dos roles, listado administrativo, comprobación de estado activo, baja lógica preservando auditoría y protección del último administrador.
- Persistencia relacional en PostgreSQL con migraciones versionadas y custodia de secretos fuera del código.

5.6. Prueba técnica (RF-13)

En cumplimiento del requisito RF-13, se implementó el endpoint POST/api/prueba/dominio gobernado por domainPolicy.js. Su función consiste en validar si el dominio del correo remitido pertenece a la lista de dominios permitidos (ALLOWED_DOMAINS). La Figura 5.3 ilustra la respuesta afirmativa ante un dominio autorizado, mientras que la Figura 5.4 evidencia el rechazo de un subdominio no reconocido. Las 13 pruebas unitarias de domainPolicy.test.js verifican rechazos sintácticos, formatos inválidos y coincidencias exactas.

5.7. Comprobación en producción

El despliegue en Google Cloud Run (https://la-grada-colchonera-1081564062572.europe-southwest1.run.app) fue sometido a validación manual en condiciones reales de explotación:

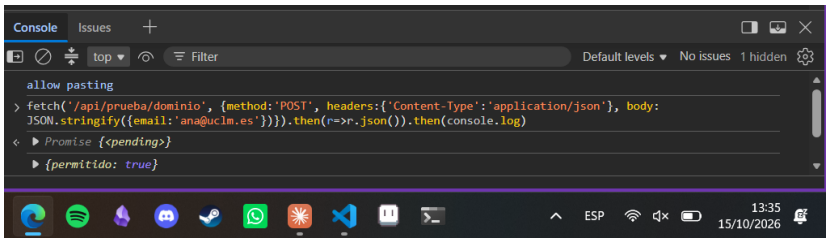


Figura 5.3: Prueba técnica: validación afirmativa de dominio permitido.

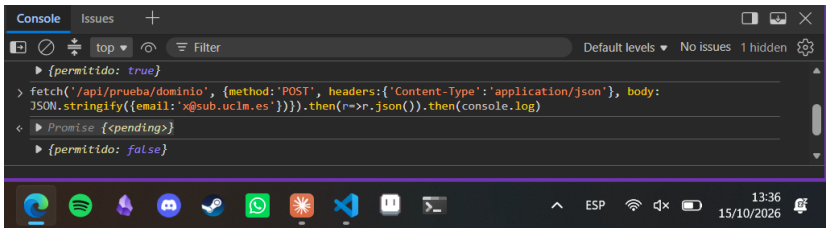


Figura 5.4: Prueba técnica: rechazo de subdominio no autorizado.

- Emisión y recepción satisfactoria de correo electrónico transaccional en buzón real de Gmail (Figura 5.5).
- Configuración de Google Auth Platform con estado publicado para usuarios externos (Figura 5.6).
- Validación de enlaces de confirmación manipulados o caducados.
- Interacción concurrente sobre perfiles y verificación de bajas de cuenta.

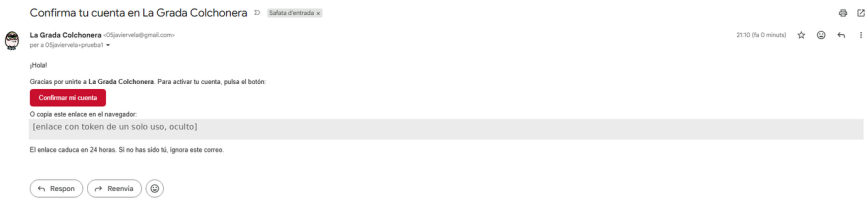


Figura 5.5: Correo de confirmación transaccional recibido en un buzón de Gmail.

5.8. Métricas del proyecto

La Tabla 5.3 sintetiza las dimensiones cuantitativas del software desarrollado a lo largo de ambos sprints.

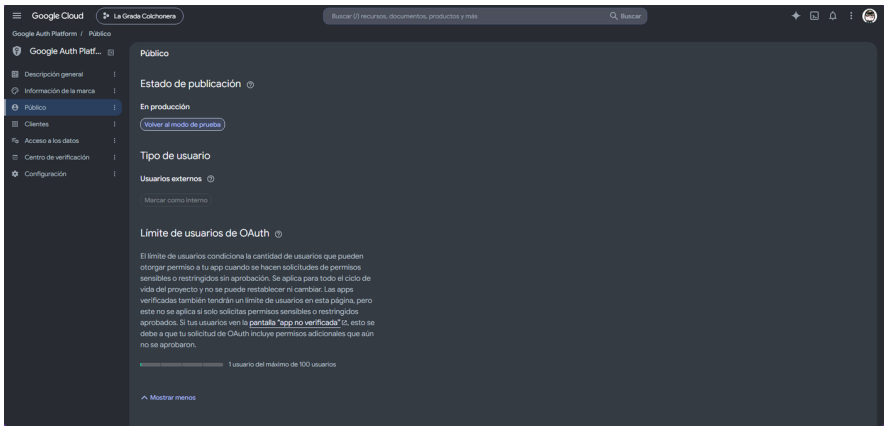


Figura 5.6: Consola de Google Cloud: aplicación OAuth publicada en producción para usuarios externos.

Tabla 5.3: Métricas globales del proyecto.

Métrica	**Valor**
PBIs y tareas del Sprint 1	19 PBIs / 32 tareas
PBIs y tareas del Sprint 2	29 PBIs / 50 tareas
Pull requests fusionados	Más de 50
Releases publicadas en GitHub	`hito-1`, `hito-2`, `hito-3`, `hito-4`, `sprint-1`
Pruebas automáticas totales	276 (más batería sobre PostgreSQL real en CI)
Endpoints expuestos en la API	23
Migraciones SQL aplicadas	5
Líneas de código del backend (`src/`, con SQL)	2.300
Líneas de interfaz web (`ui.js`, `api.js`, `ui/`)	1.150
Líneas de hojas de estilo CSS	950
Líneas del motor del juego (`public/js/game/`, 35 módulos)	14.800
Líneas de pruebas automatizadas	5.050
Dependencias directas de producción	6
Contenido jugable implementado	36 jugadores, 2 entrenadores, 24 armas, 14 pasivas, 24 combinaciones, 6 ultras, 16 equipos, 10 jefes, 5 ambientes, 26 selecciones, 29 logros, 10 equipaciones

5.9. Resultados y discusión

La evaluación confirma que la adopción de una arquitectura en capas complementada con inyección de dependencias facilitó la incorporación del juego en el Sprint 2 sin desestabilizar la base SaaS del Sprint 1. El requisito de mantener el pipeline de CI en verde antes de fusionar garantizó la ausencia de regresiones a lo largo de los más de 50 pull requests procesados. La disponibilidad de 276 pruebas automáticas aporta confianza en el comportamiento determinista del motor y en la robustez de las políticas de seguridad del servidor.

6. Conclusiones y Trabajo Futuro

6.1. Conclusiones

Se repasan los objetivos en el orden del Capítulo 1.

Objetivo principal. Se ha cumplido. La aplicación está desplegada en una URL pública de Google Cloud Run (Anexo C). Los aficionados se registran con email o entran con Google, juegan, y su club, sus partidas y sus logros se guardan en el servidor. Los dos sprints están cerrados: el Sprint 1 con sus 19 PBIs y el Sprint 2 con sus 29, todos hechos.

Producto. Los cuatro hitos del Sprint 1 están completos, desplegados y comprobados en producción (Capítulo 5), con trazabilidad desde el backlog hasta la release (Anexo B). Encima de esa base está el juego del Sprint 2 (Capítulo 4). Pasar de red social a juego fue una buena decisión: reutiliza toda la base del sprint y ha obligado a resolver problemas técnicos interesantes, como el determinismo de la simulación, el rendimiento, la validación en el servidor, la generación procedural y el audio sintetizado.

Proceso. Todo el trabajo pasó por el mismo camino: PBI en DoneTonic, tarea, rama, pull request con el CI en verde y despliegue automático. Ese camino se puede seguir hacia atrás desde cualquier pull request, y las gráficas de DoneTonic y los burndown del Anexo A muestran cómo avanzó cada sprint. El cambio de alcance del Sprint 1 (el PBI P19, para aprovechar los créditos de Google Cloud) se registró como un PBI más, en lugar de hacerse por fuera del proceso.

Calidad. La arquitectura en capas con inyección de dependencias permitió cambiar de memoria a PostgreSQL, añadir correo y Google y montar un juego entero encima sin tocar la lógica existente. Probar primero los casos de error y exigir el CI en verde para fusionar evitó regresiones durante todo el desarrollo: al cierre hay 276 pruebas automáticas.

Documentación. La gestión, la documentación técnica, la revisión de la entrega y esta memoria están en el repositorio, junto al código. Quien revise el proyecto puede ir de un requisito a su PBI, de ahí a la tarea, a la rama y al pull request, y ver las pruebas que lo cubren, sin salir de GitHub ni pedir acceso a ninguna herramienta.

6.1.1. Lecciones aprendidas

1. Dividir pronto el trabajo en tareas pequeñas: una rama y un pull request por tarea hacen que cada cambio sea fácil de revisar y de deshacer.
2. Automatizar el despliegue desde el primer día: así cada entrega es un pull request más y no un trabajo aparte al final.
3. Documentar las decisiones mientras se toman, no al final: la documentación técnica y esta memoria salen de lo que ya estaba escrito en el repositorio.
4. Comprobar visualmente cada cambio de interfaz. Así apareció, por ejemplo, el primer clic perdido en «Reenviar el correo»: al salir del campo de contraseña aparecía su error, la página se movía y el clic caía fuera del botón (Anexo A).

6.2. Trabajo futuro

1. **Multijugador:** lista de amigos y partidas en grupo (*party*) de hasta 4 jugadores, con WebSockets y un servidor autoritativo.
2. **Más contenido:** jugadores, jefes, equipaciones y logros, y mejores sprites para los jugadores menos conocidos.
3. **Ranking por temporadas y por ambiente.**
4. **Seguridad de las cuentas:** recuperación de la contraseña por correo y límite de intentos de inicio de sesión (*rate limiting*).
5. **Pruebas de extremo a extremo** automatizadas en el CI (Playwright) y medición de la cobertura.
6. **Aplicación instalable (PWA)** para jugar desde el móvil como una app.

A. Desarrollo del Proyecto

Este anexo recoge el desarrollo de los dos sprints con el mismo esquema en cada uno: planificación, desarrollo (problemas encontrados y soluciones, no un diario), revisión y retrospectiva. Los datos salen de `docs/gestion/README.md` (backlog, trazabilidad tarea a PR y evolución de cada sprint) y de `docs/documentacion-tecnica.md` (secciones 4, 13 y 14). La trazabilidad completa requisito–PBI–tarea–PR está en el Anexo B; la metodología y las herramientas, en el Capítulo 2; los requisitos, en el Capítulo 3.

A.1. Sprint 1: la base SaaS (29/09/2026–27/10/2026)

A.1.1. Planificación

El Sprint 1 tuvo como objetivo construir la base de una aplicación SaaS según los cuatro hitos del enunciado [Universidad de Castilla-La Mancha, 2026]: repositorio y flujo de trabajo, integración y despliegue continuos, lógica de usuarios con API REST y frontend separados, persistencia en PostgreSQL, secretos fuera del código, roles y administración, registro de actividad y, en el Hito 4, confirmación por correo, inicio de sesión con Google y manejo de errores de principio a fin. El alcance se repartió así: Hitos 1–3 del 29/09 al 12/10/2026, Hito 4 hasta el 23/10/2026 y cierre del sprint el 27/10/2026.

El backlog del sprint tuvo 19 PBIs (P1–P19) con 32 tareas (T1–T32), etiquetados por hito (*Hito 1* a *Hito 4*). Todos son prioridad MUST porque son requisitos obligatorios del enunciado. La Tabla A.1 muestra el sprint backlog completo.

Tabla A.1: Sprint Backlog del Sprint 1 (19 PBIs, 32 tareas).

PBI	Hito	Título	Tareas	PRs	Release
P1	H1	Repositorio GitHub y flujo GitHub Flow	T1, T2	#1, #2	hito-1
P2	H1	Integración continua (CI)	T3	#3	hito-1
P3	H1	Despliegue continuo en URL pública	T4	#4, #10	hito-1

PBI	Hito	Título	Tareas	PRs	Release
P4	H1	Lógica de gestión de usuarios (en memoria)	T5, T6	#5, #6	hito-1
P5	H1	Pruebas unitarias automatizadas de la lógica	T7	#7	hito-1
P6	H1	Capa API de usuarios	T8	#8	hito-1
P7	H2	Frontend: interfaz y cliente de comunicación separados	T9, T10	#11, #12	hito-2
P8	H2	Registro e inicio de sesión locales	T11, T12	#13, #14	hito-2
P9	H2	Cierre de sesión y mantenimiento de la sesión	T13	#15	hito-2
P10	H2	Rutas protegidas en el servidor	T14	#16	hito-2
P11	H3	Persistencia en base de datos real	T15, T16	#18, #19	hito-3
P12	H3	Gestión de secretos con variables de entorno	T17	#20	hito-3
P13	H3	Roles de usuario y administrador	T18, T19	#21, #22	hito-3
P14	H3	Registro (log) de la actividad	T20	#23	hito-3
P15	H4	Inicio de sesión con proveedor externo (OAuth)	T21, T22	#40	hito-4
P16	H4	Confirmación de cuenta por correo electrónico	T23, T24	#38, #39	hito-4
P17	H4	Manejo de errores y feedback al usuario	T25, T26	#37–#41	hito-4
P18	H4	README completo y revisión de la entrega	T27, T28, T31, T32	#25, #26, #30, #31	sprint-1
P19	H4	Despliegue continuo en Google Cloud Run	T29, T30	#27, #28	sprint-1

Cada PBI tiene en DoneTonic su historia de usuario («Como … quiero … para …») y sus criterios de aceptación, derivados del enunciado. La Definición de Hecho (DoD) de una tarea exige: código en un PR con referencia a la tarea, pruebas en verde en el CI (check *Pruebas*), PR fusionado en la rama del hito y documentación actualizada si procede. Un hito está hecho cuando su rama está en `main`, desplegada en la URL pública y publicada como release.

La Tabla A.2 recoge la trazabilidad de las 32 tareas: cada una tiene su rama `hito-N/Tx-descripcion` y su PR hacia `hito-N/integracion`.

Tabla A.2: Trazabilidad del Sprint 1: tarea → rama → PR.

PBI	Tarea	Descripción	Rama	PR
P1	T1	Crear repositorio y estructura base	`hito-1/T1-estructura-base`	#1
P1	T2	Configurar GitHub Flow	`hito-1/T2-github-flow`	#2
P2	T3	Workflow de CI con GitHub Actions	`hito-1/T3-integracion-continua`	#3

PBI	Tarea	Descripción	Rama	PR
P3	T4	Despliegue automático en proveedor cloud gratuito	`hito-1/T4-despliegue-continuo`, `hito-1/T4-url-publica`	#4, #10
P4	T5	Modelo de usuario y repositorio en memoria	`hito-1/T5-modelo-repositorio-memoria`	#5
P4	T6	Servicio de gestión de usuarios	`hito-1/T6-servicio-usuarios`	#6
P5	T7	Framework de pruebas y al menos 5 pruebas	`hito-1/T7-pruebas-unitarias`	#7
P6	T8	Endpoints de la API de usuarios	`hito-1/T8-api-usuarios`	#8
P7	T9	Cliente de comunicación con la API	`hito-2/T9-cliente-api`	#11
P7	T10	Interfaz de usuario: vistas y navegación	`hito-2/T10-vistas-navegacion`	#12
P8	T11	Registro y login en el backend (con hash)	`hito-2/T11-registro-login-backend`	#13
P8	T12	Formularios de registro y login en el frontend	`hito-2/T12-formularios-frontend`	#14
P9	T13	Mantener la sesión al recargar y cerrar sesión	`hito-2/T13-gestion-sesion`	#15
P10	T14	Middleware de autenticación y protección de rutas	`hito-2/T14-rutas-protegidas`	#16
P11	T15	Elegir y configurar la base de datos	`hito-3/T15-configurar-bd`	#18
P11	T16	Repositorio de BD sustituyendo al de memoria	`hito-3/T16-repositorio-postgres`	#19
P12	T17	Sacar los secretos a variables de entorno	`hito-3/T17-secretos-entorno`	#20
P13	T18	Roles, autorización y primer administrador (backend)	`hito-3/T18-roles-autorizacion`	#21
P13	T19	Panel de administración y baja de cuenta (frontend)	`hito-3/T19-panel-admin`	#22
P14	T20	Configurar el logging de la actividad relevante	`hito-3/T20-logging`	#23
P15	T21	Registrar la aplicación OAuth en Google	`hito-4/T22-google-oauth`	#40
P15	T22	Flujo de inicio de sesión con OAuth	`hito-4/T22-google-oauth`	#40
P16	T23	Servicio de correo	`hito-4/T23-correo`	#38
P16	T24	Flujo de confirmación de la cuenta	`hito-4/T24-confirmacion`	#39
P17	T25	Errores en el backend	`hito-4/T25-secretos-hito4` y resto de ramas del hito	#37–#41

PBI	Tarea	Descripción	Rama	PR
P17	T26	Mensajes de error en el frontend	`hito-4/T26-privacidad` y resto de ramas del hito	#37–#41
P18	T27	Completar el README	`hito-4/T27-readme-completo`	#25
P18	T28	Etiquetas de hito y revisión final	`hito-4/T28-revision-entrega`	#26
P18	T31	Revisión frase a frase contra el enunciado y README final	`hito-4/T31-revision-enunciado`	#30
P18	T32	Carpeta de la memoria en LuaLaTeX	`hito-4/T32-carpeta-memoria`	#31
P19	T29	Contenerizar la aplicación con Docker	`hito-4/T29-docker`	#27
P19	T30	Despliegue continuo en Cloud Run con GitHub Actions	`hito-4/T30-cloud-run`	#28

A.1.2. Desarrollo: problemas encontrados y soluciones

Durante el Sprint 1 surgieron los problemas de la Tabla A.3, la Tabla A.4, la Tabla A.5 y la Tabla A.6.

Tabla A.3: Problemas de desarrollo del Sprint 1 y su solución.

Problema	Causa	Solución
Un JSON mal formado devolvía la página de error HTML de Express	El error ocurría en `express.json()`, antes de la capa API	Manejador propio justo después del parser: 400 en JSON en toda la API (prueba técnica)

Tabla A.4: Problemas de despliegue y servicios externos.

Problema	Causa	Solución
El correo no salía desde Cloud Run	El SMTP sin cifrar (puerto 25/587) suele estar bloqueado	SMTP de Gmail por el puerto 465 con contraseña de aplicación, guardada en Secret Manager
`redirect_uri_mismatch` al entrar con Google	Cloud Run tiene dos formas de URL y cada una necesita su URI de redirección	Registrar las dos en el cliente de Google y usar `PUBLIC_URL`

Tabla A.5: Problemas de ramas y GitHub.

Problema	Causa	Solución
Las ramas del Hito 4 dependían unas de otras	Se desarrollaron en cadena	Reubicarlas (rebase) sobre el `main` real antes de abrir los PRs, cada una con el CI en verde

Tabla A.6: Cambio de alcance durante el sprint.

Problema	Causa	Solución
Cambiar de proveedor de despliegue a mitad del sprint	Llegaron créditos de Google Cloud para educación	Nuevo PBI P19 (cambio de alcance documentado), Docker y Cloud Run como principal, Render como alternativo

A.1.3. Revisión

La Tabla A.7 recoge la cronología del proyecto. Los cierres de hito fueron los PR #9 (Hito 1 a `main`), #17 (Hito 2), #24 (Hito 3), #29 (cierre del sprint) y #32 (revisión final y memoria). La primera entrega, el 12/10/2026, se limitó a los Hitos 1–3, P18 y P19; el Hito 4 se completó y comprobó en producción el 23/10/2026 (correo real recibido, aplicación de Google publicada). Al final del sprint se publicó la revisión frase a frase contra el enunciado en `docs/revision-entrega.md`.

Tabla A.7: Cronología del proyecto.

Fecha	Hito
29/09/2026	Empieza el Sprint 1
Primeras semanas	Hitos 1, 2 y 3 (releases `hito-1`, `hito-2`, `hito-3`)
12/10/2026	Primera entrega (Hitos 1–3, P18, P19); capturas de DoneTonic de ese día
23/10/2026	Hito 4 en producción y comprobado (correo real, Google publicado)
23/10/2026	Empieza el Sprint 2: rediseño de la interfaz y base del survivor (PR #36)
24/10/2026	Cierre del sprint; prueba técnica (PR #34). Juego: personajes, movilidad, cámara, mapa, contenido y Fases 2–4 (PRs #44–#52)
26/10/2026	Fases 5 y 6 del juego (PRs #53 y #54)
27/10/2026	Fin del Sprint 1; capturas nuevas y documentación (PR #55)

Fecha	Hito
29/10/2026	Arreglo de la baja con Google (PR #56); Sprint 2 registrado y cerrado en DoneTonic; fin del Sprint 2

A.1.4. Retrospectiva

La Tabla A.8 y la Figura A.1 muestran la evolución de los PBIs pendientes del alcance de la entrega (16 PBIs: P1–P14, P18 y P19). El escalón «+P19» refleja el cambio de alcance al añadir el despliegue en Cloud Run.

Tabla A.8: Evolución del Sprint 1 (PBIs del alcance de la entrega).

Momento	PBIs hechos	PBIs pendientes
Inicio del Sprint 1 (29/09/2026)	0	15
Cierre del Hito 1 (release `hito-1`)	6	9
Cierre del Hito 2 (release `hito-2`)	10	5
Cierre del Hito 3 (release `hito-3`)	14	1
Se añade P19 (Cloud Run)	14	2
Cierre de la primera entrega (12/10/2026)	16	0

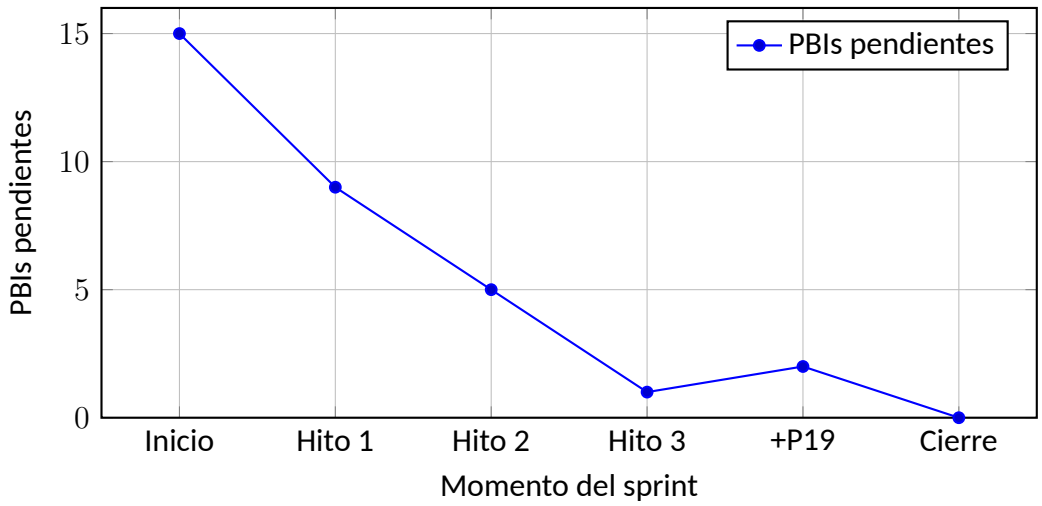


Figura A.1: Burndown del Sprint 1: PBIs pendientes del alcance de la entrega según se cerraban los hitos.

Las Figuras A.2, A.3, A.4 y A.5 son capturas de DoneTonic (tomadas el 29/10/2026, con los dos sprints cerrados): el Sprint Burndown y el Sprint Burnup del Sprint 1, el Product

Backlog con los dos sprints y el tablero del Sprint 1 al 100 %. El tablero del Sprint 1 también aparece en la Figura 2.1 del Capítulo 2.



Figura A.2: Sprint Burndown y Sprint Burnup del Sprint 1 en DoneTonic: los 19 PBIs completados, con el escalón del cambio de alcance (P19).

Cambio de alcance. El profesor repartió créditos de Google Cloud para educación durante el sprint; para aprovecharlos se añadió el PBI P19 (despliegue continuo en Google Cloud Run, tareas T29 y T30) y Render quedó como despliegue alternativo. El cambio aparece en el burndown como un aumento temporal de PBIs pendientes (de 1 a 2) antes del cierre.

Lecciones aprendidas. Dividir pronto en tareas pequeñas con rama y PR propios evitó integraciones conflictivas; automatizar el despliegue desde el primer día eliminó sorpresas; documentar las decisiones en el momento facilitó la memoria; y comprobar visualmente cada cambio de interfaz permitió detectar, por ejemplo, el clic perdido del reenvío de correo (Tabla A.11).

A.2. Sprint 2: la lógica del juego (23/10/2026–29/10/2026)

A.2.1. Planificación

Cerrado el Sprint 1, el profesor fijó que el Sprint 2 sería la lógica del juego. El juego se programó del 23 al 26/10 en seis fases, antes de que se fijara el contenido del sprint, y su backlog (29 PBIs, P20–P48, y 50 tareas, T33–T82) se registró en DoneTonic al cerrar el sprint (29/10). Todos los PBIs son MUST. Las etiquetas por fase fueron: *Base del juego* (P20–P28), *Fase 2 · Club* (P29–P32), *Fase 3 · Ultras* (P33–P34), *Fase 4 · Mundo* (P35–P37), *Fase 5 · Rarezas* (P38–P41), *Fase 6 · Logros y sonido* (P42–P46) y *Calidad* (P47–P48). La Tabla A.9 muestra el sprint backlog; «hecho» es el día en que se fusionó en `main` el último PR de cada PBI.

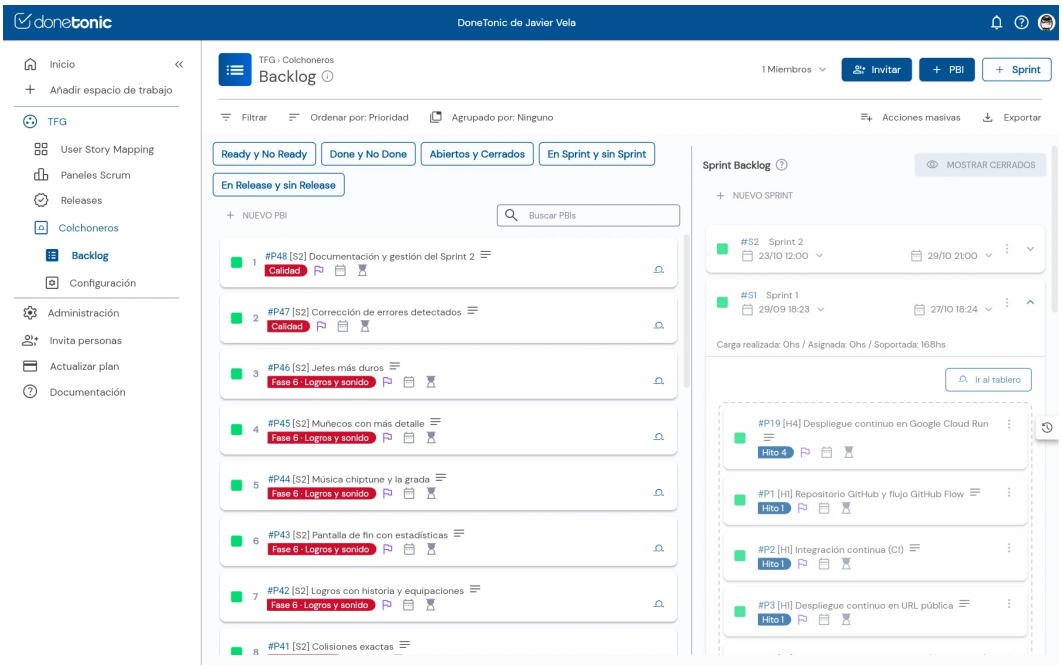


Figura A.3: Product Backlog de DoneTonic con los dos sprints (Sprint 1 desplegado con sus PBIs).

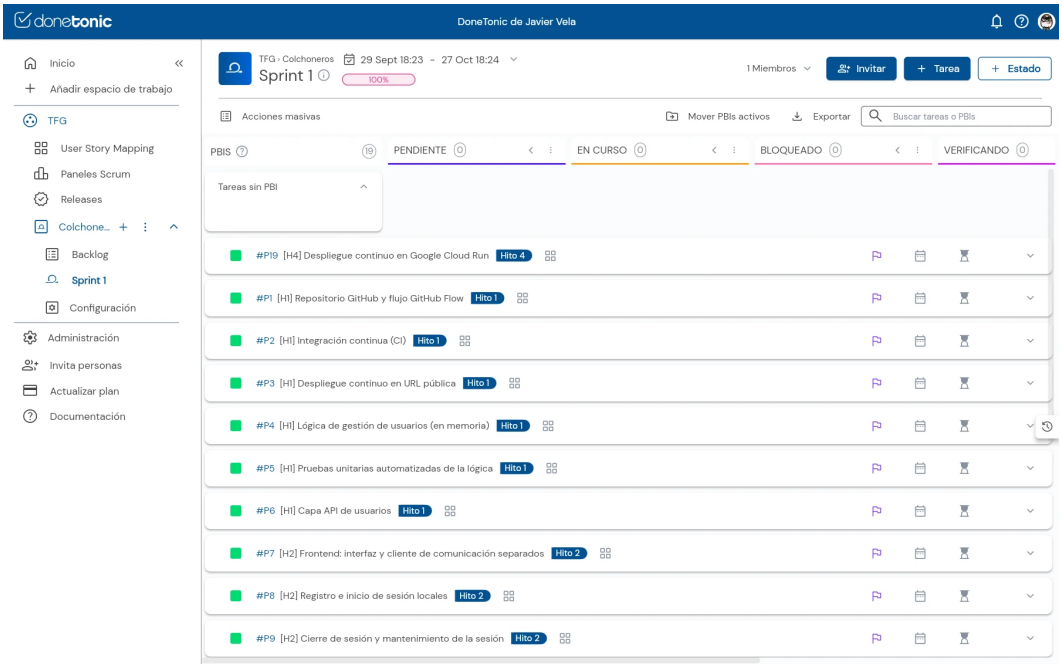


Figura A.4: Tablero del Sprint 1 en DoneTonic al cierre: los 19 PBIs en verde (hechos).



Figura A.5: Detalle de las gráficas del Sprint 1 en DoneTonic (vista ampliada del burndown y burnup).

Tabla A.9: Sprint Backlog del Sprint 2 (29 PBIs, 50 tareas).

PBI	Etiqueta	Título	Tareas	PRs	Hecho
P20	Base del juego	Rediseño de la interfaz web y menú del juego	T33, T34	#36, #45	24/10/2026
P21	Base del juego	Motor de la partida survivor	T35, T36	#36	23/10/2026
P22	Base del juego	Gráficos pixel art y cámara 3/4	T37, T38	#44, #46	24/10/2026
P23	Base del juego	Jugadores, entrenadores y Vestuario	T39–T41	#44, #48	24/10/2026
P24	Base del juego	Armas, habilidades y mejoras	T42, T43	#36, #48	24/10/2026
P25	Base del juego	Rivales con comportamiento propio	T44, T45	#44, #45, #47	24/10/2026
P26	Base del juego	Jefes con fases	T46, T47	#45, #48	24/10/2026
P27	Base del juego	Mapa con terreno y obstáculos	T48, T49	#45, #47	24/10/2026
P28	Base del juego	Controles, pantalla completa y ajustes	T50, T51	#36, #47	24/10/2026
P29	Fase 2 · Club	Progreso del juego guardado en el servidor	T52, T53	#49	24/10/2026
P30	Fase 2 · Club	Contratos: fichar jugadores	T54	#49	24/10/2026
P31	Fase 2 · Club	Entrenamiento de jugadores	T55	#49, #50	24/10/2026

PBI	Etiqueta	Título	Tareas	PRs	Hecho
P32	Fase 2 · Club	Ranking de la grada	T56	#49	24/10/2026
P33	Fase 3 · Ultras	Combinaciones y ULTRAS	T57, T58	#50	24/10/2026
P34	Fase 3 · Ultras	Dificultad equilibrada y modo pruebas	T59, T60	#50	24/10/2026
P35	Fase 4 · Mundo	Mundo procedural natural	T61	#52	24/10/2026
P36	Fase 4 · Mundo	Ambientes por finales	T62	#52	24/10/2026
P37	Fase 4 · Mundo	Mote y avatar	T63	#52	24/10/2026
P38	Fase 5 · Rarezas	Rarezas por historia rojiblanca y fichajes	T64, T65	#53	26/10/2026
P39	Fase 5 · Rarezas	Habilidades niveladas por minuto	T66	#53	26/10/2026
P40	Fase 5 · Rarezas	Combinaciones y ultras a la vista	T67	#53	26/10/2026
P41	Fase 5 · Rarezas	Colisiones exactas	T68	#53	26/10/2026
P42	Fase 6 · Logros y sonido	Logros con historia y equipaciones	T69, T70	#54	26/10/2026
P43	Fase 6 · Logros y sonido	Pantalla de fin con estadísticas	T71	#54	26/10/2026
P44	Fase 6 · Logros y sonido	Música chiptune y la grada	T72–T74	#54	26/10/2026
P45	Fase 6 · Logros y sonido	Muñecos con más detalle	T75	#54	26/10/2026
P46	Fase 6 · Logros y sonido	Jefes más duros	T76	#54	26/10/2026
P47	Calidad	Corrección de errores detectados	T77–T79	#51, #55, #56	29/10/2026
P48	Calidad	Documentación y gestión del Sprint 2	T80–T82	#55	29/10/2026

La Tabla A.10 recoge las 29 historias de usuario del Sprint 2 con sus criterios de aceptación, tal como están en DoneTonic.

Tabla A.10: Historias de usuario y criterios de aceptación del Sprint 2.

PBI	Historia de usuario	Criterios de aceptación
P20	**Rediseño de la interfaz web y menú del juego**. Como aficionado quiero una web con aspecto de juego, clara en el móvil y en el ordenador, para que entrar a jugar sea lo primero que vea al iniciar sesión.	Sistema de diseño propio (colores, espacios, tipografía Plus Jakarta Sans) en tokens CSS reutilizables; Modo oscuro rojiblanco en toda la web y diseño responsive; Con sesión iniciada el inicio es el menú del juego; la cabecera solo muestra Jugar y Mi cuenta (y Administración al admin); Las pantallas de cuentas del Sprint 1 siguen funcionando igual (pruebas en verde).
P21	**Motor de la partida survivor**. Como jugador quiero una partida de supervivencia contra oleadas infinitas, en la que mis armas ataquen solas y yo solo tenga que moverme y elegir mejoras.	Simulación pura (sin DOM) con paso fijo de 1/60 s y azar con semilla: misma semilla, misma partida; Oleadas cada 30 s, experiencia al eliminar rivales y elección de 1 de 3 mejoras al subir de nivel; Dificultad que crece sin parar (vida, daño y velocidad de los rivales); Pausa, fin de partida y vuelta al menú; la partida se pausa al cambiar de pestaña.
P22	**Gráficos pixel art y cámara 3/4**. Como jugador quiero un estilo visual propio y reconocible, con muñecos «chibi» del Atleti, para que el juego tenga personalidad sin usar imágenes oficiales.	Muñecos de 16×20 píxeles dibujados con código (sin imágenes), con caché de fotogramas; Cámara 3/4 con inercia, inclinación al correr, destellos y aplastado al recibir golpes; Se respeta prefers-reduced-motion (sin sacudidas ni partículas).
P23	**Jugadores, entrenadores y Vestuario**. Como jugador quiero elegir entre leyendas del Atleti y un entrenador, cada uno con su estilo, para que cada partida se juegue distinto.	Estadísticas por posición (portero, defensa, medio, delantero) retocadas por los rasgos de cada jugador; Cada jugador solo puede aprender las habilidades que encajan con su forma de jugar; Dos entrenadores (Simeone y Luis Aragonés) con bonificaciones y una ulti propia; Vestuario con filtro por posición y ficha del jugador.
P24	**Armas, habilidades y mejoras**. Como jugador quiero muchas armas y habilidades que suban de nivel y se puedan combinar, para construir una estrategia distinta en cada partida.	Armas de disparo y cuerpo a cuerpo que atacan solas, con 5 niveles; Pasivas (botas, imán, pasión…) y mejoras extra sin límite cuando todo está al máximo; Los porteros no aprenden armas de disparo.

PBI	Historia de usuario	Criterios de aceptación
P25	**Rivales con comportamiento propio**. Como jugador quiero rivales variados que me obliguen a moverme, para que la partida no sea solo aguantar en un sitio.	16 equipos con jugadores genéricos que se desbloquean por oleadas; Unos embisten avisando con un «!» y otros disparan balones; Los rivales rodean gradas y paredes para llegar al jugador.
P26	**Jefes con fases**. Como jugador quiero jefes famosos con ataques propios cada pocas oleadas, para tener retos claros dentro de la partida.	Un jefe cada 4 oleadas, al azar entre los disponibles y sin repetir hasta que salgan todos; Tres fases por vida (66 % y 33 %): más ataques, menos espera y más velocidad; Cada ataque fuerte se avisa en el suelo antes de golpear.
P27	**Mapa con terreno y obstáculos**. Como jugador quiero un campo con barro, lluvia, gradas y estructuras, para que moverse por el mapa sea parte del juego.	Barro que frena, lluvia que acelera los balones y gradas que no se atraviesan; Mapa procedural con biomas y estructuras distinto en cada partida; Premios por el campo señalados con flechas en el borde de la pantalla.
P28	**Controles, pantalla completa y ajustes**. Como jugador quiero jugar con teclado o con el dedo y ajustar el sonido y los gráficos, para jugar cómodo en cualquier dispositivo.	Teclado (WASD/flechas) y joystick táctil flotante en el móvil; Pantalla completa (botón y tecla F) y menú de pausa con ajustes dentro de la partida; Ajustes guardados en el navegador: volumen, efectos, números de daño, zoom y joystick.
P29	**Progreso del juego guardado en el servidor**. Como jugador quiero que mi club y mis partidas se guarden en mi cuenta, para no perder el progreso al cambiar de dispositivo.	Tablas juego_perfiles y juego_records (migración 003) ligadas al usuario; API /api/juego protegida con sesión: perfil, empezar y terminar partida; El servidor valida cada resultado con su propio reloj (anti-trampas) y rechaza resultados imposibles; Concurrencia optimista: dos peticiones a la vez no se pisan.
P30	**Contratos: fichar jugadores**. Como jugador quiero ganar fichas de contrato venciendo jefes y canjearlas por jugadores nuevos, para ampliar mi plantilla.	Plantilla inicial de 4 jugadores comunes (uno por posición); Una ficha por cada 3 jefes vencidos en la cuenta; el contrato nunca repite jugador; La probabilidad depende de la rareza del jugador.
P31	**Entrenamiento de jugadores**. Como jugador quiero entrenar a mis jugadores con los puntos que ganan, para que mejoren partida a partida.	Puntos por jefes (1 cada 3 con ese jugador) y por bajas (cada vez más caros); Cuatro estadísticas (Vida, Defensa, Disparo y Suerte) hasta 20 puntos; Solo se gastan puntos libres; validado en el servidor.

PBI	Historia de usuario	Criterios de aceptación
P32	**Ranking de la grada**. Como jugador quiero compararme con el resto de aficionados, para tener un motivo para mejorar mi marca.	Una fila por usuario con su mejor partida (tiempo; en empate, nivel); Nunca se muestra el correo de nadie; Pestañas Mis partidas y La grada.
P33	**Combinaciones y ULTRAS**. Como jugador quiero que juntar ciertas habilidades desbloquee combinaciones y poderes definitivos, para premiar las estrategias pensadas.	24 combinaciones con nombre propio y bonus; 6 ULTRAS que se consiguen solas al completar combinaciones (Premio Puskás, Balón de Oro, Zamora…); Efecto visual propio y daño en parte porcentual (reducido contra jefes).
P34	**Dificultad equilibrada y modo pruebas**. Como jugador quiero que el juego sea difícil pero justo, y como administrador quiero poder probarlo todo sin jugar horas.	Tramo final más duro desde el minuto 9 y rivales curtidos desde el 12; Ayuda inicial que se apaga en 8 minutos para poder vencer al primer jefe; Modo pruebas (solo admin): todos los jugadores y puntos sin tocar el ranking.
P35	**Mundo procedural natural**. Como jugador quiero un mundo distinto en cada partida, con barrios, estadios, parques y mesetas, para que explorar tenga gracia.	Ruidos suaves para distrito, altura y humedad: todo encaja sin cortes; Estadios con gradas que miran al campo, plazas con tiendas, mesetas con escaleras; Casi todo el mapa es alcanzable desde la salida (con pruebas).
P36	**Ambientes por finales**. Como jugador quiero que cada 5 minutos la partida cambie de escenario, para sentir que avanzo de final en final.	Pretemporada → Copa del Rey → Liga → Champions → Mundial; Cambian la luz, el césped, la afición y los carteles; los rivales salen más fuertes; En el Mundial todos llevan la camiseta de su selección.
P37	**Mote y avatar**. Como jugador quiero un mote y una cara propios, para que en el ranking y en la cabecera no salga mi correo.	Mote de 3 a 16 caracteres, único sin distinguir mayúsculas ni tildes (409 si está cogido); Avatar con la cara de un jugador fichado o de un entrenador; Migración 004 y PUT /api/juego/identidad.
P38	**Rarezas por historia rojiblanca y fichajes**. Como aficionado quiero que la rareza de cada jugador refleje lo que fue en el Atleti, y que fichar se sienta especial.	36 jugadores en cuatro rarezas (Común, Raro, Épico, Leyenda) según su historia en el club; Más rareza = más habilidades aprendibles y menos probabilidad en un contrato; Animación de la firma del contrato en pixel art.
P39	**Habilidades niveladas por minuto**. Como jugador quiero que las habilidades más fuertes no rompan la partida al principio.	Niveles que no salen hasta cierto minuto (desdeMinuto) y recarga mínima (cdMin); Regate, entrada rasante y contragolpe reequilibrados.

PBI	Historia de usuario	Criterios de aceptación
P40	**Combinaciones y ultras a la vista**. Como jugador quiero ver qué combinaciones y ultras existen y qué me falta, para planificar la partida.	Pantalla Combinaciones con nombre, bonus, requisitos y ultra a la que cuentan; Filtros Todas / Con tu jugador / Conseguidas.
P41	**Colisiones exactas**. Como jugador quiero que lo que veo sea lo que choca, para que nadie se esconda detrás de un tejado.	Edificios, gradas y bancos chocan en todo su dibujo; Los muñecos chocan por los pies (vista 3/4); la copa de los árboles se vuelve transparente.
P42	**Logros con historia y equipaciones**. Como aficionado quiero logros con datos curiosos del Atleti que desbloqueen equipaciones, para tener objetivos a largo plazo.	29 logros con un dato real que se descubre al conseguirlo; El servidor solo apunta los logros que puede comprobar; 10 equipaciones que se eligen en el Vestuario.
P43	**Pantalla de fin con estadísticas**. Como jugador quiero ver al final de la partida qué ha funcionado, para mejorar en la siguiente.	Tiempo, ambiente, bajas, jefes, daño total, golpe máximo, críticos, daño recibido y vida recuperada; Barras de daño por habilidad (armas, ultras, ulti y quemaduras); Jefes vencidos con lo que costaron y logros nuevos.
P44	**Música chiptune y la grada**. Como jugador quiero música y ambiente de estadio, para que la partida emocione.	Fanfarria propia por competición y canción de fondo que se anima con el tiempo y con los jefes; La grada murmura y canta cuando te acercas a ella; Volumen de música aparte del de efectos; todo sintetizado (sin ficheros).
P45	**Muñecos con más detalle**. Como aficionado quiero reconocer a los jugadores y a los jefes en el campo.	Sombreado, orejas, nariz y peinados nuevos; Rasgos propios: brazalete de capitán, cinta, pecas, barba, pendientes, tatuajes.
P46	**Jefes más duros**. Como jugador quiero que los jefes sean un reto de verdad a partir del segundo.	Escolta de élites desde el segundo jefe y refuerzos al cambiar de fase; Furia si tardas en ganarles; Una jugada estrella por jefe en la fase 3; el primero sigue siendo asequible.
P47	**Corrección de errores detectados**. Como usuario quiero que los errores que encuentren los probadores se arreglen con pruebas que eviten que vuelvan.	Cada error se reproduce con una prueba antes de arreglarlo; Los arreglos pasan el CI y se despliegan.
P48	**Documentación y gestión del Sprint 2**. Como responsable del proyecto quiero que todo el Sprint 2 se pueda revisar desde el repositorio.	README con capturas del juego y de DoneTonic; Documentación técnica para la memoria con diagramas; Backlog, tablero y gráficas del Sprint 2 en DoneTonic.

A.2.2. Desarrollo: problemas encontrados y soluciones

Durante el Sprint 2 se resolvieron los problemas de la Tabla A.11.

Tabla A.11: Problemas del Sprint 2 (juego y cuenta) y su solución.

Problema	Causa	Solución
El juego no cargaba al entrar	Una variable se usaba antes de existir en el módulo de arranque	Inicializar la variable antes de su primer uso (PR #51)
El primer clic en «Reenviar el correo» se perdía tras un error	El botón quedaba deshabilitado tras el primer intento fallido	Rehabilitar el botón tras mostrar el error (PR #55)
No se podía volver tras eliminar la cuenta con Google	El flujo de baja con OAuth no limpiaba la sesión local	Cerrar sesión local tras la baja federada (PR #56)

A.2.3. Revisión

El Sprint 2 se cerró el 29/10/2026, día en que se registró el backlog completo en DoneTonic y se fusionó el último PR (#56). Las entregas intermedias del juego fueron los PR #36 (23/10, base), #44–#52 (24/10, Fases 1–4), #53 y #54 (26/10, Fases 5–6) y #55 (27/10, documentación). La revisión del enunciado siguió vigente con la misma evidencia de `docs/revision-entrega.md`, ampliada con la prueba técnica (PR #34).

A.2.4. Retrospectiva

La Figura A.6 muestra el burndown real del Sprint 2 según las fechas de fusión de los PR. Como el backlog se registró en DoneTonic al cerrar el sprint (29/10/2026), las gráficas de DoneTonic del Sprint 2 salen en recta: los 29 PBIs aparecen como pendientes hasta el 29/10 y pasan a hechos ese mismo día.

Las Figuras A.7, A.8 y A.9 son las capturas de DoneTonic del Sprint 2 (tomadas el 29/10/2026): las gráficas en recta, el backlog con los 29 PBIs hechos y el tablero al 100 %.

Explicación de la recta. El backlog del Sprint 2 se registró en DoneTonic el 29/10/2026, al cerrar el sprint, con todos los PBIs ya en estado hecho; por eso el Sprint Burndown aparece como una línea recta que cae de 29 a 0 el último día, en lugar de una curva repartida en el tiempo.

Lecciones aprendidas. Registrar el backlog al inicio del sprint habría dado una gráfica real de seguimiento; aun así, la trazabilidad tarea–PR del Anexo B documenta las fechas reales de cada entrega (23, 24, 26, 27 y 29 de octubre).

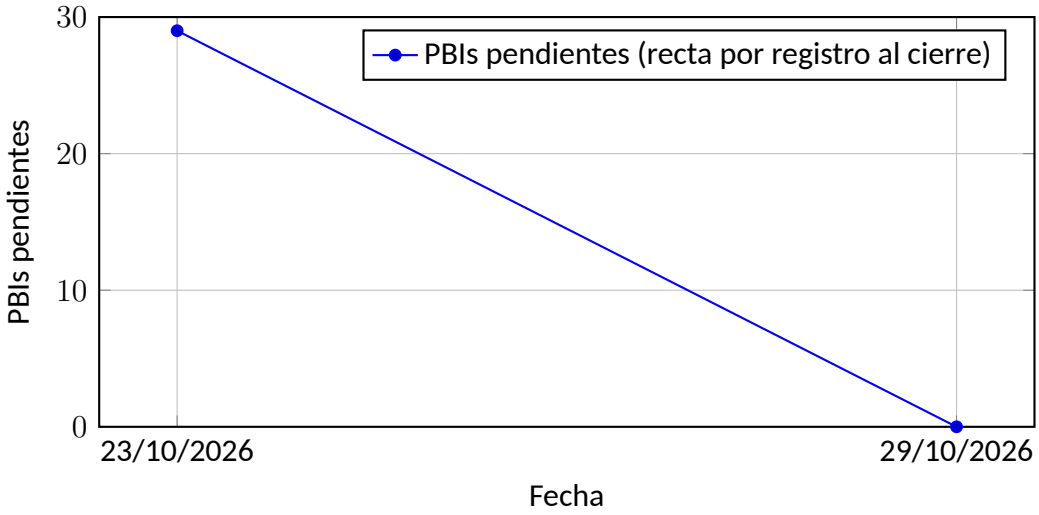


Figura A.6: Burndown del Sprint 2: recta porque el backlog se registró al cerrar el sprint (29/10/2026).

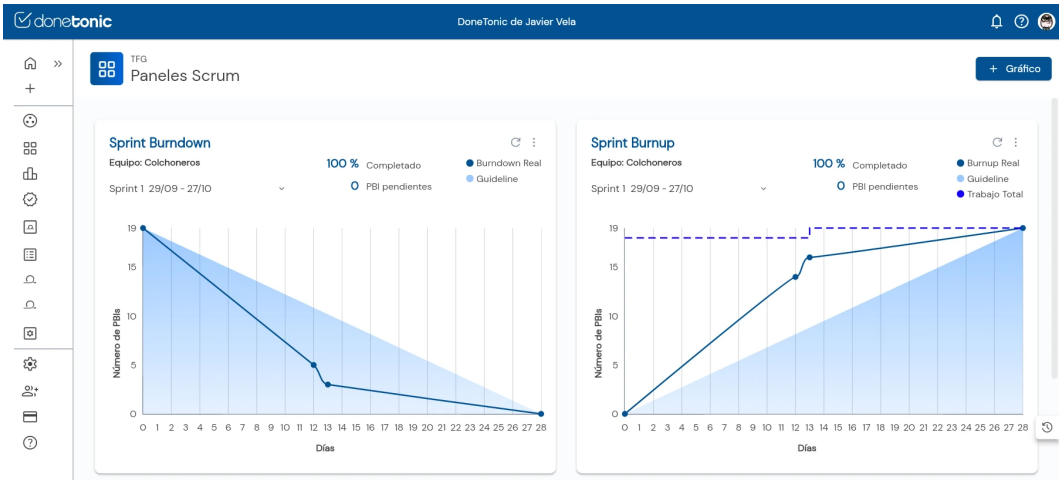


Figura A.7: Gráficas del Sprint 2 en DoneTonic: recta porque el backlog se registró al cerrar el sprint (29/10/2026).

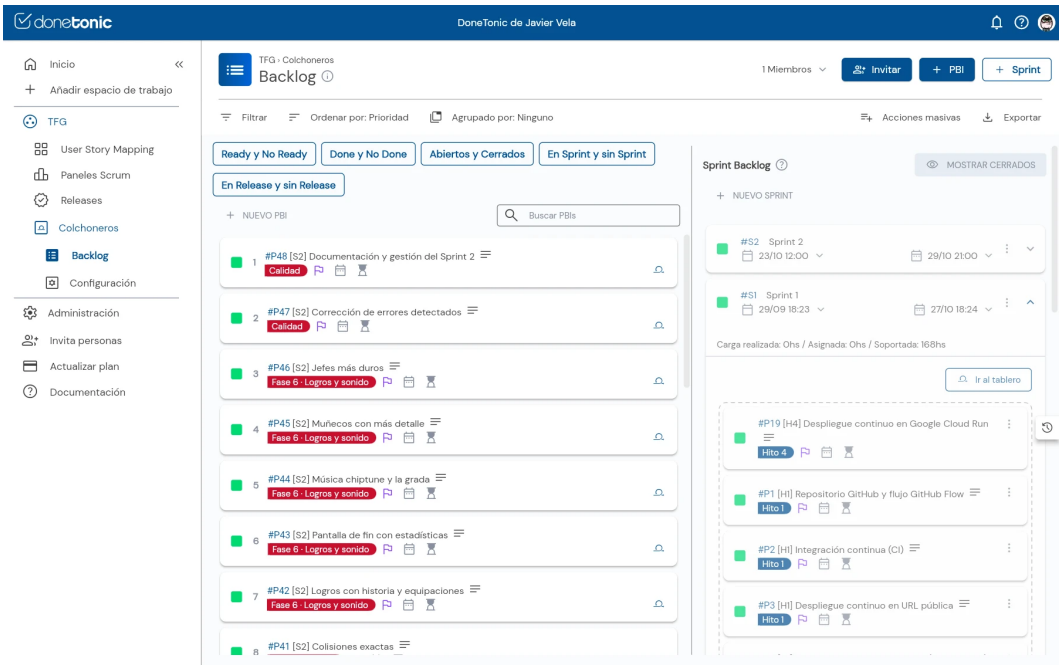


Figura A.8: Product Backlog de DoneTonic con el Sprint 2 cerrado: 29 PBIs (P20–P48) en verde.

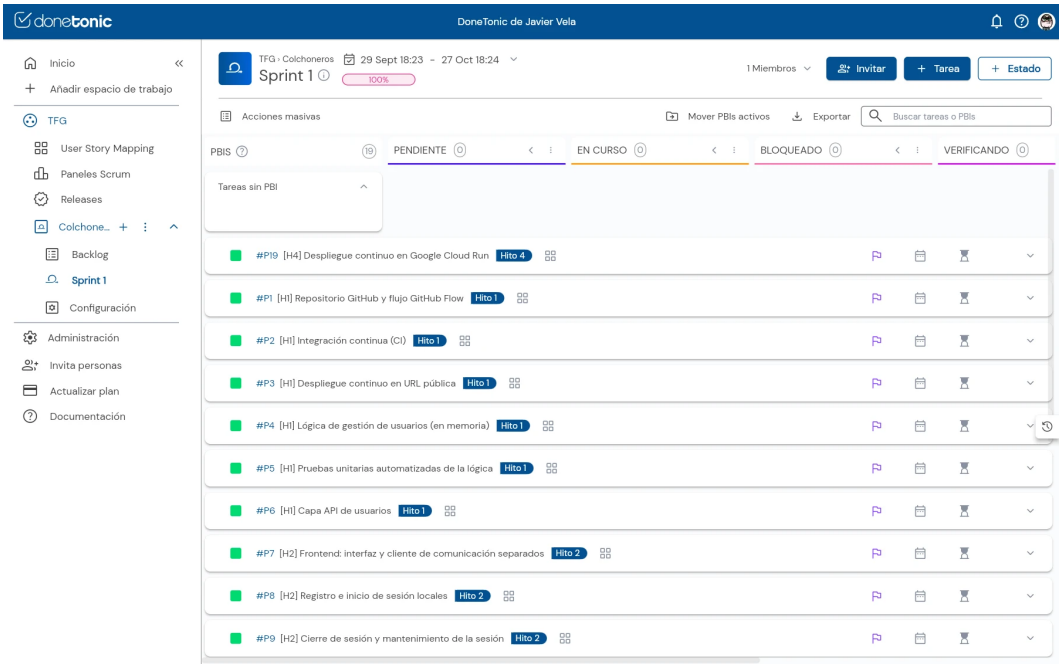


Figura A.9: Tablero de DoneTonic al cierre de los dos sprints: Sprint 1 y Sprint 2 al 100 %.

B. Trazabilidad

Este anexo presenta la trazabilidad completa desde los requisitos hasta los pull requests que los implementan, pasando por los PBIs y las tareas técnicas de los dos sprints: Sprint 1 (Hitos 1–4, PBIs P1–P19, tareas T1–T32) y Sprint 2 (PBIs P20–P48, tareas T33–T82). La correspondencia requisito–PBI sale de la sección 3 de `docs/documentacion-tecnica.md` (tablas de RF, RJ y RNF); la correspondencia PBI–tarea–PR sale de las tablas de trazabilidad de `docs/gestion/README.md`. En el Sprint 1 cada tarea tiene su rama propia y su PR individual hacia la rama del hito; en el Sprint 2 cada tarea indica en su descripción de DoneTonic el PR donde está (un PR por fase y uno por arreglo). Los requisitos no funcionales son transversales y se resumen al principio de la tabla; después, una fila por cada una de las 82 tareas. La Tabla B.1 es la matriz completa.

Tabla B.1: Trazabilidad completa: requisito → PBI → tarea → PR.

Requisito	PBI	Tarea	Descripción de la tarea	PR
RNF-01	P8, P9, P10, P13	T11–T14, T18, T19	Seguridad de credenciales y sesiones: bcrypt, cookie httpOnly/secure/sameSite, regeneración de sesión, permisos en el servidor, consultas parametrizadas	#13–#16, #21, #22
RNF-02	P12	T17	Secretos fuera del código: variables de entorno y Secret Manager	#20
RNF-03	P4, P6, P11	T5, T6, T8, T15, T16	Separación en capas API/lógica/datos con inyección de dependencias	#5, #6, #8, #18, #19
RNF-04	P2, P5	T3, T7	Pruebas automáticas con `node:test` y CI obligatorio en ramas protegidas	#3, #7
RNF-05	P3, P19	T4, T29, T30	Despliegue continuo en URL pública (Render y Cloud Run)	#4, #10, #27, #28

Requisito	PBI	Tarea	Descripción de la tarea	PR
RNF-06	P3, P19	T4, T29, T30	Disponibilidad y coste: escala a cero, capas gratuitas y créditos de educación	#4, #10, #27, #28
RNF-07	P7, P8, P17	T9, T10, T12, T25, T26	Accesibilidad: HTML semántico, foco, `aria-live`, `aria-describedby`, teclado, `prefers-reduced-motion`	#11, #12, #14, #37–#41
RNF-08	P7, P28	T9, T10, T50, T51	Responsive: panel de administración en tarjetas en el móvil y joystick táctil	#11, #12, #36, #47
RNF-09	P21, P22, P27	T35–T38, T48, T49	Rendimiento del juego: paso fijo, búsqueda espacial por celdas, texturas cacheadas	#36, #44, #46, #45, #47
RNF-10	Todos	T1–T82	Trazabilidad requisito–PBI–tarea–rama–PR–release	#1–#56
RNF-11	P16, P17, P32	T23, T24, T26, T56	Privacidad: política pública, ranking sin correo, logs que ocultan datos sensibles	#38, #39, #37–#41, #49
RNF-12	P4, P6, P11, P24	T5, T6, T8, T15, T16, T42, T43	Mantenibilidad: sin frameworks en el frontend, equilibrio en `config.js`, módulos pequeños	#5, #6, #8, #18, #19, #36, #48
—	P1	T1	Crear repositorio y estructura base	#1
—	P1	T2	Configurar GitHub Flow	#2
RNF-04	P2	T3	Workflow de CI con GitHub Actions	#3
RNF-05, RNF-06	P3	T4	Despliegue automático en proveedor cloud gratuito	#4, #10
RNF-03	P4	T5	Modelo de usuario y repositorio en memoria	#5
RNF-03	P4	T6	Servicio de gestión de usuarios	#6
RNF-04	P5	T7	Framework de pruebas y al menos 5 pruebas	#7
RNF-03	P6	T8	Endpoints de la API de usuarios	#8
RNF-07, RNF-08	P7	T9	Cliente de comunicación con la API	#11
RNF-07, RNF-08	P7	T10	Interfaz de usuario: vistas y navegación	#12
RF-01, RF-02	P8	T11	Registro y login en el backend (con hash)	#13

Requisito	PBI	Tarea	Descripción de la tarea	PR
RF-01, RF-02	P8	T12	Formularios de registro y login en el frontend	#14
RF-03	P9	T13	Mantener la sesión al recargar y cerrar sesión	#15
RF-04	P10	T14	Middleware de autenticación y protección de rutas	#16
RF-05	P11	T15	Elegir y configurar la base de datos	#18
RF-05	P11	T16	Repositorio de BD sustituyendo al de memoria	#19
RNF-02	P12	T17	Sacar los secretos a variables de entorno	#20
RF-06, RF-07, RF-08	P13	T18	Roles, autorización y primer administrador (backend)	#21
RF-06, RF-07, RF-08	P13	T19	Panel de administración y baja de cuenta (frontend)	#22
RF-09	P14	T20	Configurar el logging de la actividad relevante	#23
RF-11	P15	T21	Registrar la aplicación OAuth en Google	#40
RF-11	P15	T22	Flujo de inicio de sesión con OAuth	#40
RF-10	P16	T23	Servicio de correo	#38
RF-10	P16	T24	Flujo de confirmación de la cuenta	#39
RF-12	P17	T25	Errores en el backend	#37–#41
RF-12	P17	T26	Mensajes de error en el frontend	#37–#41
—	P18	T27	Completar el README	#25
—	P18	T28	Etiquetas de hito y revisión final	#26
—	P18	T31	Revisión frase a frase contra el enunciado y README final	#30
—	P18	T32	Carpeta de la memoria en LuaLaTeX	#31
RNF-05, RNF-06	P19	T29	Contenerizar la aplicación con Docker	#27
RNF-05, RNF-06	P19	T30	Despliegue continuo en Cloud Run con GitHub Actions	#28
RF-13	—	—	Prueba técnica: solo dominios de correo permitidos (`domainPolicy.js`)	#34
RJ-01	P20	T33	Sistema de diseño: tokens, tipografía y modo oscuro rojiblanco	#36
RJ-01	P20	T34	Menú del juego como inicio con sesión (hub con el jugador paseando)	#36, #45
RJ-02	P21	T35	Simulación de paso fijo con semilla (`mundo.js`)	#36
RJ-02	P21	T36	Oleadas, experiencia y mejoras al subir de nivel	#36
RJ-03	P22	T37	Muñecos chibi generados por código (`sprites.js`)	#44

Requisito	PBI	Tarea	Descripción de la tarea	PR
RJ-03	P22	T38	Cámara 3/4, inercia y sensación de movimiento y golpes	#46
RJ-04	P23	T39	Jugadores con estadísticas por posición y habilidades propias	#44, #48
RJ-04	P23	T40	Entrenadores con ulti y pantalla Dirección	#44
RJ-04	P23	T41	Vestuario renovado con filtros y ficha	#48
RJ-02, RJ-05	P24	T42	Armas con 5 niveles: disparo, cuerpo a cuerpo y movilidad	#36, #48
RJ-02, RJ-05	P24	T43	Pasivas y mejoras extra	#36
RJ-06	P25	T44	16 equipos rivales que se desbloquean por oleadas	#44
RJ-06	P25	T45	Rivales que embisten, disparan y rodean obstáculos	#45, #47
RJ-07	P26	T46	Jefes con tres fases y avisos de ataque	#45
RJ-07	P26	T47	Seis jefes nuevos con habilidades propias	#48
RJ-08	P27	T48	Barro, lluvia, gradas y premios	#45
RJ-08	P27	T49	Mapa procedural con biomas y estructuras	#47
RJ-10	P28	T50	Entrada por teclado y joystick táctil	#36
RJ-10	P28	T51	Pantalla completa, pausa y ajustes en partida	#47
RJ-01, RJ-11	P29	T52	Migración 003 y repositorios del juego (memoria y PostgreSQL)	#49
RJ-01, RJ-11	P29	T53	API del juego con validación anti-trampas	#49
RJ-12	P30	T54	Fichas por jefes y sorteo de contratos por rareza	#49
RJ-13	P31	T55	Puntos y estadísticas de entrenamiento	#49, #50
RJ-14	P32	T56	Ranking multijugador con la mejor partida de cada usuario	#49
RJ-15	P33	T57	Combinaciones con nombre y bonus	#50
RJ-15	P33	T58	Seis ULTRAS con efectos visuales	#50
RJ-02, RJ-16	P34	T59	Curvas de dificultad calibradas con un bot	#50
RJ-02, RJ-16	P34	T60	Modo pruebas para administradores	#50
RJ-08	P35	T61	Generación del terreno por ruidos (`terreno.js`)	#52
RJ-09	P36	T62	Cinco ambientes con cambio de luz, afición y fuerza	#52
RJ-17	P37	T63	Migración 004, API de identidad y pantalla Tu ficha	#52
RJ-04, RJ-12	P38	T64	Rarezas por historia y 36 jugadores	#53
RJ-04, RJ-12	P38	T65	Animación de la firma del contrato	#53

Requisito	PBI	Tarea	Descripción de la tarea	PR
RJ-05	P39	T66	Niveles bloqueados por minuto y recarga mínima	#53
RJ-15	P40	T67	Pantalla de combinaciones y ultras con filtros	#53
RJ-08	P41	T68	Hitbox por los pies y obstáculos macizos	#53
RJ-18	P42	T69	Logros validados en el servidor (`logros.js`)	#54
RJ-18	P42	T70	Pantalla Logros y equipaciones en el Vestuario	#54
RJ-19	P43	T71	Atribución del daño por fuente y pantalla de fin (`fin-ui.js`)	#54
RJ-20	P44	T72	Secuenciador chiptune y fanfarrias (`musica.js`)	#54
RJ-20	P44	T73	La grada: murmullo y cánticos según la cercanía	#54
RJ-20	P44	T74	Efectos de sonido nuevos y ajuste de música	#54
RJ-03	P45	T75	Rasgos y sombreado en `sprites.js`	#54
RJ-07	P46	T76	Escolta, furia y jugada estrella	#54
—	P47	T77	El juego no cargaba (variable usada antes de existir)	#51
—	P47	T78	El primer clic en «Reenviar el correo» se perdía tras un error	#55
—	P47	T79	No se podía volver tras eliminar la cuenta (Google)	#56
—	P48	T80	README del Sprint 2 con capturas del juego y de DoneTonic	#55
—	P48	T81	Documentación técnica para la memoria	#55
—	P48	T82	Backlog y seguimiento del Sprint 2 en DoneTonic	— (DoneTonic)

La Tabla B.1 cubre los 48 PBIs (P1–P48) y las 82 tareas (T1–T82) de los dos sprints. Los cierres de hito del Sprint 1 fueron los PR #9 (Hito 1), #17 (Hito 2), #24 (Hito 3), #29 (cierre del sprint) y #32 (revisión final y memoria); las releases publicadas son `hito-1`, `hito-2`, `hito-3`, `hito-4` y `sprint-1` (Anexo C).

C. Material Adicional

C.1. Enlaces del proyecto

- **Repositorio principal:**
  `https://github.com/VelaElDios/la-grada-colchonera-saas`
- **Aplicación desplegada (Google Cloud Run):**
  `https://la-grada-colchonera-1081564062572.europe-southwest1.run.app`
- **Releases publicadas** (`hito-1`, `hito-2`, `hito-3`, `hito-4` y `sprint-1`):
  `https://github.com/VelaElDios/la-grada-colchonera-saas/releases`
- **Documentación del repositorio:**
  - `README.md`: visión general, tecnologías, ejecución local, pruebas, variables de entorno, API, despliegue y acceso como administrador.
  - `CONTRIBUTING.md`: flujo de trabajo con ramas, commits y pull requests.
  - `docs/revision-entrega.md`: revisión frase a frase del enunciado con su evidencia.
  - `docs/gestion/README.md`: backlog de los dos sprints, trazabilidad tarea → rama → PR y capturas de DoneTonic.
  - `docs/documentacion-tecnica.md`: documentación técnica completa (requisitos, arquitectura, flujos, seguridad, juego, pruebas, CI/CD, métricas).
  - `docs/hito4-configuracion.md`: configuración del correo (SMTP) y del cliente OAuth de Google.
- **Gestión del proyecto (DoneTonic)**, espacio de trabajo *TFG*, equipo *Colchoneros*:
  `https://app.donetonic.com`

C.2. Capturas de la gestión en DoneTonic

Las Figuras C.1, C.2, C.3, C.4 y C.5 son capturas tomadas en DoneTonic el 29/10/2026, con los dos sprints cerrados: el tablero y las gráficas del Sprint 1, el Product Backlog con los dos sprints, y el tablero y las gráficas del Sprint 2. El detalle de la evolución de cada sprint está en el Anexo A.

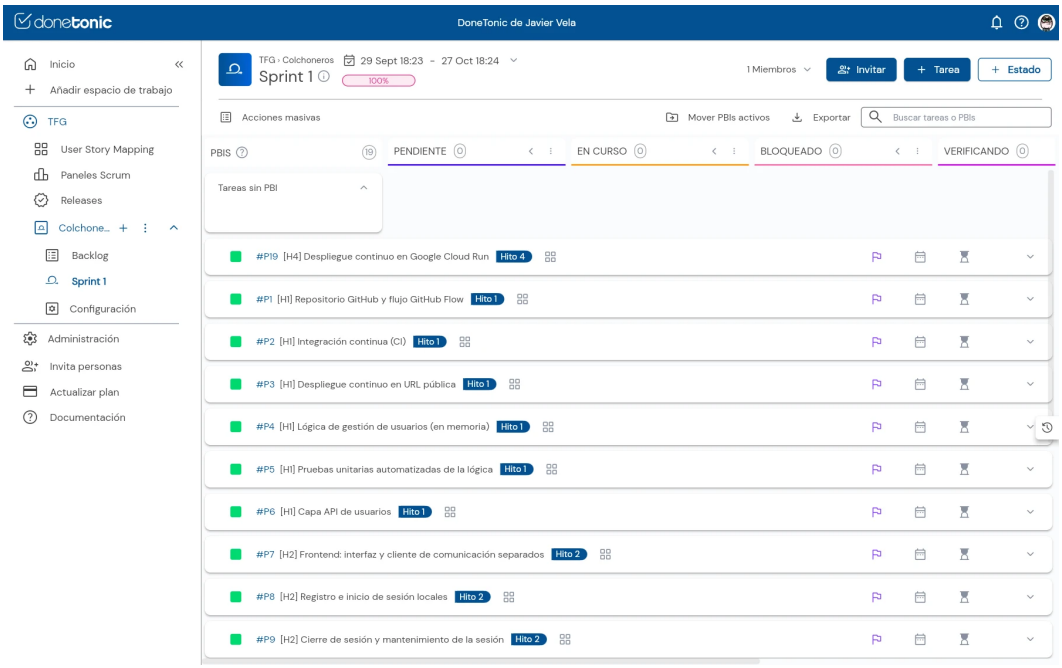


Figura C.1: Tablero del Sprint 1 en DoneTonic al 100 %: los 19 PBIs (P1–P19) hechos.

C.3. Credenciales de administrador para la revisión

Las credenciales del primer administrador (valores de `ADMIN_EMAIL` y `ADMIN_PASSWORD`) se envían junto con la entrega, tal como se indica en el `README.md` (sección «Acceso como administrador para la revisión»). No están en el repositorio ni en esta memoria.



Figura C.2: Sprint Burndown y Sprint Burnup del Sprint 1 en DoneTonic.

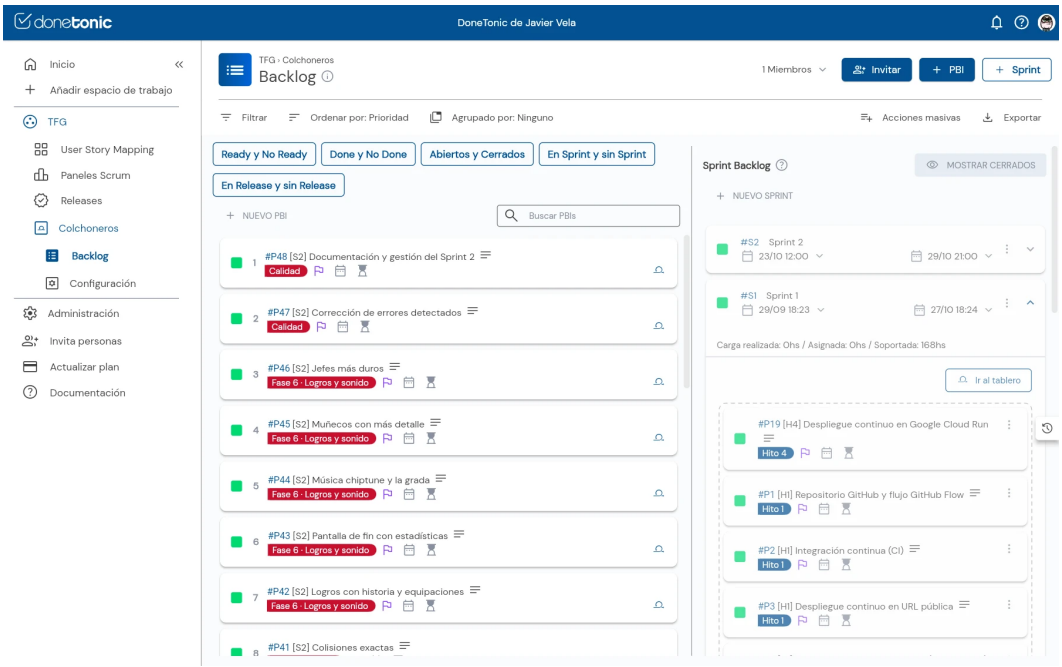


Figura C.3: Product Backlog de DoneTonic con los dos sprints del proyecto.

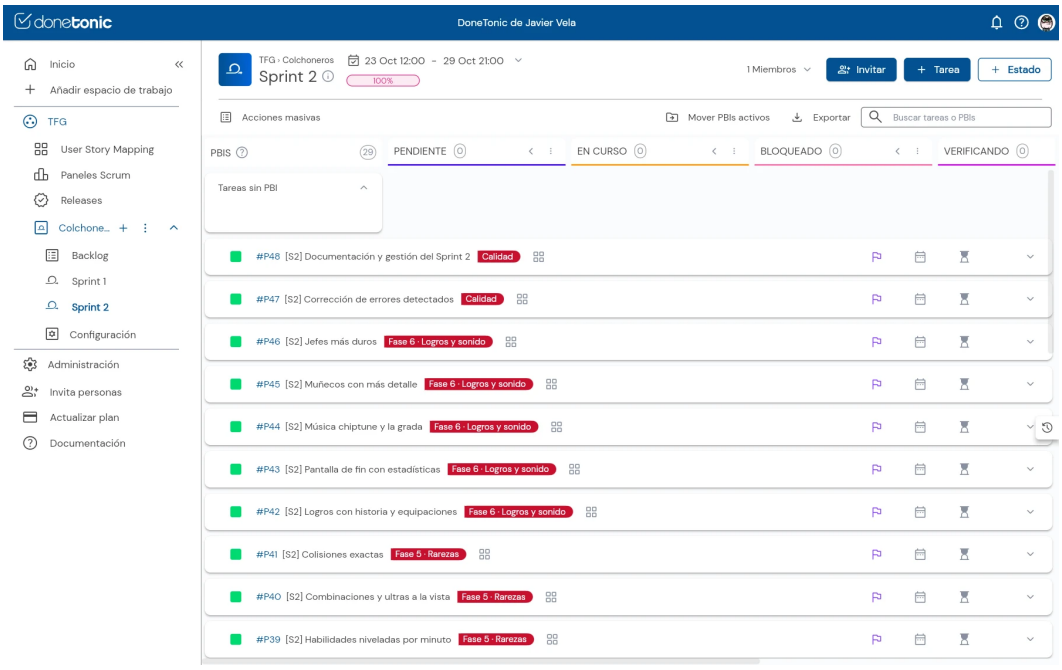


Figura C.4: Tablero del Sprint 2 en DoneTonic al 100 %: los 29 PBIs (P20–P48) con sus etiquetas de fase.

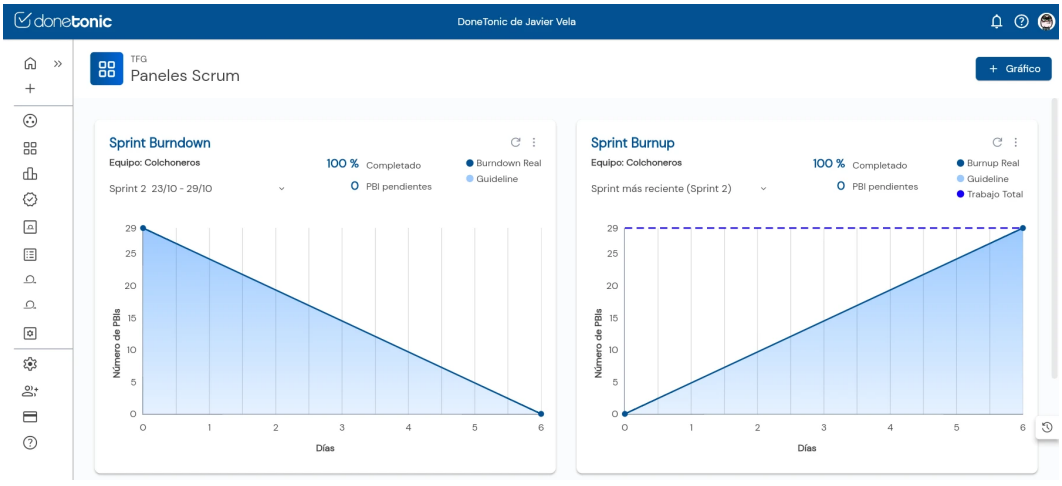


Figura C.5: Sprint Burndown y Sprint Burnup del Sprint 2 en DoneTonic.

D. Uso Responsable de la Inteligencia Artificial

Este anexo explica qué ha hecho la inteligencia artificial en el proyecto y en esta memoria, qué ha hecho el alumno y cómo se ha comprobado el resultado.

D.1. Uso de la inteligencia artificial en el desarrollo del software

Todo el proyecto, los dos sprints (incluido el juego), se hizo con Claude (Cowork) como asistente, con el modelo Claude Opus 5.5 en esfuerzo alto. Claude escribió el código, los documentos y los mensajes de los commits, que quedaron con la autoría del alumno y el trailer `Co-Authored-By`. También creó y fusionó los pull requests y actualizó el tablero de DoneTonic desde el navegador.

El alumno se encargó de lo que no se delega:

- Creó y configuró las cuentas y las credenciales de los servicios externos (Neon, Render y Google Cloud). Ninguna credencial está en el repositorio.
- Revisó la entrega del Sprint 1 frase a frase contra el enunciado (`docs/revision-entrega.md`). Esa revisión llevó a corregir el permiso de «comprobar si un usuario está activo» (PR #30).
- Probó a mano cada cambio de la interfaz y del juego en el navegador antes de darlo por hecho.

La calidad del código no se apoya en la palabra de la IA, sino en las pruebas automáticas, que se ejecutan en el CI en cada pull request y que tienen que pasar para poder fusionar (Capítulo 5).

D.2. Uso de la inteligencia artificial en la redacción de la memoria

El borrador de esta memoria lo redactaron IAs gratuitas dentro de «Cerebro Universal», un sistema del alumno que funciona en GitHub Actions. En cada paso, una IA constructo-

ra escribe y otra IA revisora comprueba el resultado contra las fuentes. Se usaron modelos gratuitos de varias empresas: Gemini (Google), modelos de la API de NVIDIA (Mistral Large, MiniMax y Nemotron) y modelos gratuitos de OpenRouter; si un modelo falla, entra el siguiente.

Para que el texto no se inventara nada se tomaron estas medidas:

- Las IAs solo podían usar como fuente la documentación del repositorio, que se descargaba en cada paso, y unas notas del alumno.
- Un test automático, que las IAs no podían modificar, comprobaba que el documento compila, que no quedan huecos de la plantilla, que cada cita existe, que están todos los requisitos, PBIs y tareas, y que ningún número mayor que 20 falta en las fuentes. Los números pequeños los comprobaba la IA revisora.
- Claude preparó la plantilla, las fuentes y ese test, revisó el resultado compilándolo y corrigió a mano los últimos fallos de maquetación.

El alumno revisa y corrige el resultado final antes de entregarlo y es responsable de su contenido.

Referencia bibliográfica

[DoneTonic, sf] DoneTonic (s.f.). Herramienta de gestión de proyectos ágiles usada para el backlog. https://app.donetonic.com.

[Express, sf] Express (s.f.). Express, node.js web application framework. https://expressjs.com/.

[GitHub, sfa] GitHub (s.f.a). Github actions documentation. https://docs.github.com/en/actions.

[GitHub, sfb] GitHub (s.f.b). Github flow. https://docs.github.com/en/get-started/using-github/github-flow.

[Google Cloud, sf] Google Cloud (s.f.). Cloud run documentation. https://cloud.google.com/run/docs.

[Hardt, 2012] Hardt, D. e. (2012). The oauth 2.0 authorization framework. RFC 6749, Internet Engineering Task Force (IETF), https://www.rfc-editor.org/rfc/rfc6749.

[OpenJS Foundation, sf] OpenJS Foundation (s.f.). Node.js. https://nodejs.org/.

[poncle, 2022] poncle (2022). Vampire survivors [videojuego]. Acceso anticipado 17/12/2021; versión 1.0 20/10/2022, https://store.steampowered.com/app/1794680/Vampire_Survivors/.

[Provos and Mazières, 1999] Provos, N. and Mazières, D. (1999). A future-adaptable password scheme. Proceedings of the FREENIX Track, 1999 USENIX Annual Technical Conference, https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf.

[Schwaber and Sutherland, 2020] Schwaber, K. and Sutherland, J. (2020). The scrum guide. https://scrumguides.org/scrum-guide.html.

[The PostgreSQL Global Development Group, sf] The PostgreSQL Global Development Group (s.f.). Postgresql. https://www.postgresql.org/.

[Universidad de Castilla-La Mancha, 2026] Universidad de Castilla-La Mancha (2026). Especificación del proyecto. procesos de ingeniería del software, curso 2026-2027 (sprint 1).